

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO
ĐỒ ÁN CHUYÊN NGÀNH

Lập lịch Phân bổ Tài nguyên trong
Datacenter sử dụng
Học Tăng Cường Đa Mục Tiêu
Ngành: Khoa học Máy tính

HỘI ĐỒNG:

GVHD 1: NGUYỄN QUANG HÙNG

—o0o—

SVTH 1: VŨ TRẦN DUY NGUYỄN - 2212338

TP. HỒ CHÍ MINH, THÁNG 05 NĂM 2026

Lời cảm ơn

Đồ án này là kết quả của quá trình học tập và nghiên cứu của em trong học kỳ vừa qua. Để hoàn thành báo cáo, em đã nhận được nhiều sự hướng dẫn, hỗ trợ từ phía Thầy và Khoa, và em xin được gửi lời cảm ơn chân thành ở những trang đầu tiên này.

Trước hết, em xin cảm ơn **Thầy Nguyễn Quang Hùng** — giảng viên hướng dẫn của em. Thầy đã giúp em định hình hướng nghiên cứu, gợi mở những góc nhìn kỹ thuật mà bản thân em khó tự nhận ra, đồng thời theo sát tiến độ và đưa ra phản hồi kịp thời ở từng giai đoạn. Những trao đổi với Thầy đã giúp em rèn luyện tư duy độc lập khi đối diện với các vấn đề kỹ thuật phức tạp.

Em cũng xin gửi lời cảm ơn đến **Khoa Khoa học và Kỹ thuật Máy tính**, Trường Đại học Bách Khoa — ĐHQG TP. HCM, vì môi trường học thuật và cơ sở vật chất nghiên cứu mà Khoa đã xây dựng. Phần kiến thức nền tảng về hệ thống phân tán, học máy và thiết kế phần mềm tích lũy qua các học phần của Khoa chính là cơ sở để em triển khai được hệ thống trong báo cáo này.

Em xin cảm ơn **gia đình** đã luôn là chỗ dựa trong suốt quá trình em học tập và thực hiện đồ án.

Dù đã cố gắng, đồ án chắc chắn còn nhiều điểm chưa hoàn thiện. Em rất mong nhận được góp ý từ Hội đồng và Quý Thầy Cô để có thể tiếp tục hoàn chỉnh ở giai đoạn sau.

TP. Hồ Chí Minh, tháng 05 năm 2026

Vũ Trần Duy Nguyên

Tóm tắt

Tên đề tài: Lập lịch Phân bổ Tài nguyên trong Datacenter sử dụng Học Tăng Cường Đa Mục Tiêu

Sinh viên thực hiện: Vũ Trần Duy Nguyên — MSSV: 2212338

Giảng viên hướng dẫn: Nguyễn Quang Hùng

Sự phát triển nhanh của các mô hình AI quy mô lớn (LLM, Diffusion Model) đang đặt hạ tầng datacenter trước áp lực chưa từng có: khối lượng tác vụ GPU vừa tăng nhanh về số lượng, vừa đa dạng về yêu cầu tài nguyên và ràng buộc thời gian. Trong khi đó, hai mục tiêu vận hành cốt lõi — *tiết kiệm năng lượng* và *đảm bảo Chất lượng Dịch vụ (SLA)* — thường xuyên xung đột với nhau. Các bộ lập lịch truyền thống như First-Fit hay Kubernetes Default chỉ tối ưu một mục tiêu duy nhất, do đó không phản ánh được sự đánh đổi (*trade-off*) giữa hai chiều này.

Trên cơ sở đó, đề án đề xuất và xây dựng **ELDAS** (*Energy-aware Learning-based Datacenter Auto-Scheduler*) — một nền tảng mô phỏng và lập lịch tự động dựa trên **Học Tăng Cường Đa Mục Tiêu (MORL)**. Hệ thống được tổ chức theo bốn lớp: (1) hạ tầng Docker Compose cho phép tái lập kết quả, (2) bộ mô phỏng datacenter CloudSim Plus có theo dõi GPU và mô hình năng lượng tuyến tính, (3) cầu nối RPC giữa Java và Python qua Py4J, và (4) môi trường học tăng cường MO-Gymnasium tích hợp agent PPO với Action Masking.

Dữ liệu đầu vào được lấy từ **Alibaba GPU Cluster Trace v2023** — bộ trace workload thực với hơn 6.000 pod — và được tách thành ba kịch bản: Low Load, High Load và Burst. Tín hiệu thưởng giữ dạng vector hai chiều $\mathbf{r} = (R_{\text{energy}}, R_{\text{SLA}})$, trong đó hệ số phạt SLA được điều chỉnh theo lớp QoS của tác vụ ($\lambda \in \{1,0; 3,0\}$). Để xấp xỉ Pareto front, đề án dùng chiến lược *linear scalarization* kết hợp với việc quét trọng số $w \in [0,1; 0,9]$.

Giai đoạn 1 đã hoàn tất toàn bộ nền tảng kỹ thuật. Vòng lặp RL được kiểm chứng đầu cuối (reset $\rightarrow$ step $\rightarrow$ reward $\rightarrow$ done); năm bộ lập lịch baseline cổ điển (FirstFit, BestFit, RoundRobin, Kubernetes LRP, Random) cùng một agent MORL PPO tối thiểu (MaskablePPO, 100k step) đã được tích hợp, tạo thành ba cụm Pareto rõ rệt để làm mốc tham chiếu. Giai đoạn 2 sẽ tập trung vào tinh chỉnh siêu tham số, quét trọng số, đối chuẩn với hai thuật toán MORL hiện đại, song song với việc tối ưu hiệu năng hệ thống và bổ sung giao diện quản lý.

Từ khóa: Học tăng cường đa mục tiêu, lập lịch tác vụ datacenter, tối ưu hóa năng lượng, SLA, CloudSim Plus, MO-Gymnasium, PPO, Alibaba Trace.

Mục lục

Lời cảm ơn	1
Tóm tắt	2
1 Giới thiệu	12
1.1 Bối cảnh và động lực nghiên cứu	12
1.2 Phát biểu bài toán	13
1.2.1 Đầu vào và đầu ra	13
1.2.2 Hai mục tiêu cần tối ưu cùng lúc	13
1.2.3 Vì sao hai mục tiêu này xung đột?	14
1.2.4 Độ phức tạp tính toán	15
1.3 Mục tiêu đề tài	15
1.3.1 Mục tiêu tổng thể	15
1.3.2 Mục tiêu Giai đoạn 1 (phạm vi báo cáo này)	16
1.3.3 Mục tiêu Giai đoạn 2 (kế hoạch tiếp nối)	16
1.4 Phạm vi và giới hạn	16
1.5 Cấu trúc báo cáo	18
2 Cơ sở lý thuyết và nghiên cứu liên quan	20
2.1 Lập lịch trong trung tâm dữ liệu	20
2.1.1 Các thuật toán heuristic truyền thống	20
2.1.2 Meta-heuristic: Ant Colony Optimization (ACO)	21
2.1.3 Kubernetes Scheduler	22
2.1.4 Lập lịch dựa trên học tăng cường: Deep Q-Network (DQN)	22
2.2 Mô hình tiêu thụ năng lượng datacenter	24
2.2.1 Mô hình tuyến tính SPECpower	24
2.2.2 Tính tổng năng lượng theo thời gian rời rạc	24
2.2.3 Mở rộng cho GPU	24
2.2.4 Power Usage Effectiveness (PUE)	24
2.2.5 Lý do chọn mô hình tuyến tính cho Phase 1	25
2.3 Học tăng cường đa mục tiêu (MORL)	25
2.3.1 Markov Decision Process (MDP)	25
2.3.2 Multi-Objective MDP (MOMDP)	25
2.3.3 Pareto Optimality	25

2.3.4	Proximal Policy Optimization (PPO)	26
2.3.5	Linear Scalarization và quét trọng số	26
2.3.6	Action Masking	26
2.4	Nghiên cứu liên quan	26
2.4.1	DeepRM (Mao và cộng sự, 2016)	27
2.4.2	Decima (Mao và cộng sự, 2019)	27
2.4.3	NSGA-II cho điều phối đám mây tiết kiệm năng lượng	27
2.4.4	Beware of Fragmentation (Weng và cộng sự, 2023)	27
2.4.5	MO-Gymnasium (Alegre và cộng sự, 2022)	27
2.5	Công nghệ nền tảng	28
2.5.1	CloudSim Plus	28
2.5.2	Py4J – cầu nối Java–Python	30
2.5.3	MO-Gymnasium	31
2.5.4	Docker Compose	31
2.5.5	So sánh với KWOK	31
2.6	Kết luận chương	31
3	Thiết kế kiến trúc hệ thống	32
3.1	Tổng quan kiến trúc bốn lớp	32
3.1.1	Vì sao chia thành bốn lớp?	32
3.1.2	Vai trò từng lớp	33
3.1.3	Một bước trong vòng lặp RL diễn ra như thế nào?	33
3.2	Lựa chọn giải pháp và biện luận	34
3.2.1	Lựa chọn công cụ mô phỏng: KWOK vs. CloudSim Plus	34
3.2.2	Lựa chọn cầu nối ngôn ngữ: Py4J vs. gRPC vs. REST	35
3.2.3	Lựa chọn thuật toán RL: PPO vs. DQN vs. A2C	35
3.2.4	Lựa chọn chiến lược MORL: Linear Scalarization vs. NSGA-II	35
3.2.5	Bảng tổng hợp ma trận quyết định	36
3.3	Thiết kế Lớp 2 — CloudSim Plus	36
3.3.1	SimulationConfig — cấu hình tập trung	37
3.3.2	DatacenterFactory và lớp theo dõi GPU	37
3.3.3	AlibabaTraceReader — đọc trace	38
3.3.4	ScenarioFilter — ba kịch bản tải	38
3.3.5	SimulationManager — ghép mô phỏng vào vòng lặp RL	38
3.4	Thiết kế Lớp 3 — Py4J Gateway	39
3.4.1	Bề mặt API tối giản	39
3.4.2	Khởi động Gateway và kết nối client	40
3.5	Thiết kế Lớp 4 — MO-Gymnasium Environment	40
3.5.1	Observation space	40
3.5.2	Action space và action masking	41
3.5.3	Vector reward và linear scalarization	41
3.5.4	Điều kiện kết thúc episode	41
3.5.5	Tóm tắt vòng lặp	41

3.6	Thiết kế Lớp 1 — Docker Compose Infrastructure	41
3.6.1	Bố trí container và volume	41
3.6.2	Multi-stage build và tối ưu kích thước image	42
3.6.3	Healthcheck và thứ tự khởi động	42
3.6.4	Khả năng tái lập kết quả	42
3.7	Thiết kế Lớp 5 — Observability stack (tùy chọn)	42
3.7.1	Sơ đồ tích hợp	43
3.7.2	Mô hình dữ liệu metric	43
3.7.3	Bốn nguyên tắc an toàn	44
3.8	Tổng kết chương	44
4	Hiện thực hệ thống	46
4.1	Hiện thực phân hệ Java	46
4.1.1	Cấu hình dự án Maven	46
4.1.2	DatacenterFactory – xây dựng datacenter và registry GPU	47
4.1.3	AlibabaTraceReader – parser chịu lỗi	48
4.1.4	Năm baseline schedulers	48
4.1.5	SimulationManager – chi tiết kỹ thuật quan trọng	50
4.1.6	MetricsExporter — xuất CSV và JSON, không cần thư viện ngoài	50
4.2	Hiện thực phân hệ Python	51
4.2.1	state_builder.py — chuyển đổi an toàn, có thể parse được	51
4.2.2	reward.py – vector, normalisation, scalarisation	51
4.2.3	environment.py – gắn kết với Gymnasium API	52
4.2.4	tracker.py – WandB online/offline tự động	53
4.2.5	baseline_eval.py — so sánh công bằng	53
4.3	Hiện thực hạ tầng Docker	54
4.3.1	Pipeline build hai stage	54
4.3.2	Quản lý volume	55
4.3.3	Vấn đề kỹ thuật gặp phải và giải pháp	55
4.4	Hiện thực Lớp 5 — Observability stack	56
4.4.1	MetricsRegistry — façade tĩnh, an toàn khi tắt	56
4.4.2	Bốn điểm móc nối trong SimulationManager	57
4.4.3	Bật/tắt và truyền tham số	57
4.4.4	Cấu hình Prometheus	58
4.4.5	Provisioning Grafana và dashboard ELDAS Live	58
4.4.6	Script kiểm tra T5.7	59
4.5	Smoke test end-to-end	59
4.6	Tổng kết chương	59
5	Kết quả và đánh giá	61
5.1	Cấu hình thực nghiệm	61
5.2	Kết quả smoke test end-to-end	62
5.2.1	Throughput của vòng lặp RL	63

5.2.2	Tính chất reward đo được	63
5.2.3	Kết quả smoke test lớp Observability (T5.7)	63
5.3	Kết quả baseline trên ba kịch bản tải	64
5.3.1	Phương pháp đo	64
5.3.2	Bảng kết quả tổng hợp	64
5.3.3	Phân tích định tính	64
5.3.4	Trực quan hoá	65
5.4	Phân tích đặc trưng dữ liệu Alibaba Trace	65
5.4.1	Thống kê tổng hợp	66
5.4.2	Phân bố ba kịch bản sau lọc	66
5.5	Đánh giá tổng thể Phase 1	68
5.5.1	Đối chiếu sáu mục tiêu MT1–MT6	68
5.5.2	Hạn chế thiết kế còn lại	68
5.6	Kết quả MORL PPO tối thiểu	69
5.6.1	Cấu hình và quy trình training	69
5.6.2	Sẵn sàng cho Phase 2	70
6	Kế hoạch Giai đoạn 2	71
6.1	Mục tiêu cụ thể của Phase 2	71
6.2	Tiêu chí chấp nhận định lượng	73
6.3	Quản lý rủi ro	73
6.4	Hướng mở rộng dài hạn	73
7	Kết luận	75
7.1	Tóm tắt đóng góp Phase 1	75
7.2	Bài học kinh nghiệm	76
7.3	Lời kết	76
A	Hướng dẫn cài đặt và chạy ELDAS	81
B	Mã nguồn quan trọng	84
C	Bảng từ viết tắt	86

Danh sách hình vẽ

1.1	Xu hướng tiêu thụ điện năng của trung tâm dữ liệu, AI và tiền mã hóa trên toàn cầu giai đoạn 2018–2026. Nguồn: IEA [3].	12
1.2	Trade-off giữa 2 mục tiêu. Mỗi điểm là một chính sách lập lịch; đường cong nối các điểm không bị thống trị (Pareto front) là kết quả mà Phase 2 hướng tới xấp xỉ.	15
3.1	Kiến trúc bốn lớp của ELDAS. Mỗi đường đứt nét là một biên cô lập về ngôn ngữ, container, hoặc hạ tầng. Mũi tên đậm đánh dấu kênh giao tiếp hai chiều của vòng lặp RL.	33
3.2	Sequence diagram cho một vòng lặp RL. Hai <code>SynchronousQueue</code> ở phía Java đảm bảo simulation thread chỉ tiến bước khi nhận được hành động hợp lệ.	34
3.3	Class diagram CloudSim Plus. Mũi tên đậm biểu diễn quan hệ <i>depends-on</i> ; <code>GatewayEntryPoint</code> là điểm vào duy nhất của Python.	37
3.4	Vòng đời <code>SimulationManager</code> trong một episode. Hai vùng “ <code>actionQueue.take</code> ” và “ <code>stepResultQueue.put</code> ” là hai điểm đồng bộ giữa thread mô phỏng và thread Py4J.	39
3.5	Cấu trúc rút gọn vector quan sát $(6H+4)$ chiều: ba khối H chiều cho CPU/RAM/GPU utilisation, một khối $3H$ chiều cho host state one-hot (<code>SUSPENDED</code> , <code>IDLE</code> , <code>ACTIVE</code> cho mỗi host), và bốn ô cuối mô tả tác vụ kế tiếp cần xếp lịch.	40
3.6	Topology Docker Compose. <code>rl-agent</code> không expose port nào (client thuần); <code>cloudsim-java</code> chỉ expose <code>\$PY4J_PORT</code>	42
3.7	Topology lớp Observability. Mũi tên liền là dữ liệu trên đường tới hạn (Py4J); mũi tên đứt là metric scrape, chạy ngoài đường tới hạn. Hai service <code>prometheus</code> và <code>grafana</code> thuộc Docker Compose profile <code>monitoring</code> – mặc định không khởi động.	43
4.1	Kết quả <code>docker compose build</code> hoàn tất cho cả hai service trên máy phát triển.	54
5.1	Hai container hoạt động ổn định và <code>cloudsim-java</code> vượt qua healthcheck Py4J, điều kiện tiên quyết cho mọi smoke test và baseline run sau đây.	61
5.2	Smoke test tám pha hoàn tất thành công.	62
5.3	Phân bố số GPU yêu cầu trên trace (7 255 tác vụ). 84,5% tác vụ yêu cầu đúng 1 GPU; 14,5% không cần GPU; <1% yêu cầu ≥ 2 GPU. Trục y thang log để các thanh thiếu số vẫn nhìn được.	67

5.4	Phân bố lớp QoS toàn trace. Tỷ lệ <i>LatencySensitive</i> cao (57,8%) là cơ sở để thiết kế λ ở §3.3: bất kỳ vi phạm SLA nào ở Phase 2 cũng có trọng số phạt khuếch đại $3\times$ đáng kể.	67
5.5	Scatter Pareto sáu chính sách trên HIGH. Trục x : năng lượng tổng (thấp hơn = tốt hơn). Trục y : tổng R_{sla} (cao hơn – ít vi phạm hơn = tốt hơn). PPO-min (chấm đỏ) nằm <i>trên-trái</i> Random (xám) ở cả hai trục $\Rightarrow$ strict Pareto dominance. Nó không dominate được hai cụm cực {BestFit, FirstFit} (energy thấp hơn) và {K8s, RoundRobin} (SLA tốt hơn) – là điểm Pareto-interior balanced.	70

Danh sách bảng

1.1	Hệ số phạt λ ứng với từng lớp QoS — vi phạm SLA của tác vụ tương tác bị phạt gấp ba lần tác vụ thường.	14
2.1	So sánh các thuật toán lập lịch truyền thống.	23
2.2	Bảng tổng hợp so sánh các nghiên cứu liên quan.	28
2.3	So sánh quy mô datacenter ELDAS với các mức công nghiệp.	30
2.4	So sánh các tùy chọn cầu nối Java–Python cho ELDAS.	30
2.5	So sánh KWOK và CloudSim Plus theo các tiêu chí của ELDAS.	31
3.1	Ma trận quyết định bốn lựa chọn kiến trúc của ELDAS. Ký hiệu: $\checkmark$ = đáp ứng tốt, $\sim$ = chấp nhận được, $\times$ = không đáp ứng.	36
3.2	Ảnh xạ trường Alibaba trace $\rightarrow$ đối tượng <code>TaskRecord</code>	38
3.3	API công khai của <code>GatewayEntryPoint</code> . Mọi tham số/giá trị trả về đều là kiểu nguyên thủy, mảng nguyên thủy, hoặc Java <code>record</code> đơn giản – thân thiện với Py4J và serializable cô lập.	39
3.4	Mô hình dữ liệu metric của ELDAS Java exporter.	44
3.5	Ảnh xạ ngược từ mục tiêu Phase 1 (§1.3) sang thành phần kiến trúc.	45
4.1	Quy mô mã nguồn Phase 1 (đếm bằng <code>wc -l</code> , không trừ comment/blank line). Tổng 3188 dòng.	46
4.2	Kích thước Docker image cuối cùng.	54
4.3	Sáu vấn đề kỹ thuật đáng ghi nhận trong Phase 1 và giải pháp.	55
4.4	Bốn điểm móc Prometheus trong <code>SimulationManager</code>	57
4.5	Năm bước của <code>check-monitoring.sh</code>	59
5.1	Cấu hình môi trường thực nghiệm Phase 1.	61
5.2	Kết quả tám pha smoke test (kịch bản LOW, seed = 42). Thời gian <i>tổng</i> đo bằng wall-clock; thời gian từng pha không được instrument riêng nên cột bên phải ghi n/a.	62
5.3	Kết quả <code>check-monitoring.sh</code> sau khi đã chạy 80 step baseline K8s. Tất cả các đối tượng kiểm tra đều đạt.	63
5.4	Kết quả năm baseline trên ba kịch bản (cùng seed = 42, cùng trace). Số liệu trích trực tiếp từ <code>summary.json</code> của mỗi run.	64

5.5	Thống kê mô tả Alibaba PAI Trace <code>openb_pod_list_default.csv</code> sau khi qua <code>AlibabaTraceReader.read</code> (chỉ giữ các pod <i>không</i> ở trạng thái <code>Pending</code> ; tổng 7255 tác vụ).	66
5.6	Số lượng tác vụ, năng lượng tiêu thụ (K8s LRP) và makespan của ba kịch bản sau khi <code>ScenarioFilter</code> lọc. Makespan phản ánh thời gian cluster còn bận đến khi task cuối hoàn thành (qua <code>flushTillEnd</code> trong piecewise energy integrator).	66
5.7	Checklist hoàn thành sáu mục tiêu Phase 1 với artifact bằng chứng có thể kiểm tra độc lập.	68
5.8	Bảng so sánh sáu chính sách trên kịch bản HIGH, sắp xếp theo năng lượng tăng dần. Dòng PPO-min in đậm. Cột R_{sla} là tổng cộng dồn vector reward thành phần SLA (giá trị càng cao $\Rightarrow$ ít vi phạm hơn). Số liệu sinh tự động bởi <code>scripts/phase1-comparison.py</code>	69
6.1	Tám hạng mục công việc của Phase 2 và đầu ra dự kiến.	72
6.2	Bốn tiêu chí chấp nhận của Phase 2. Tất cả phải đạt ✓ mới coi là Phase 2 thành công.	73
6.3	Phân tích rủi ro Phase 2. Thang P/I : thấp / trung / cao.	73

Listings

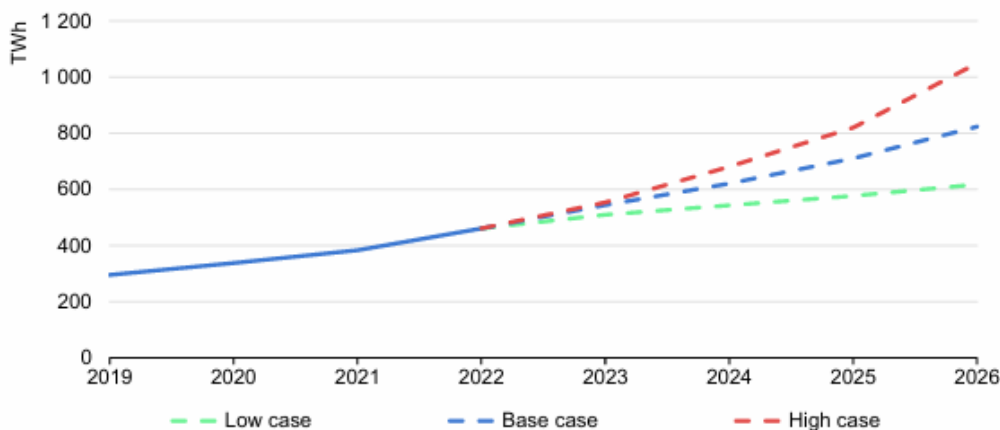
2.1	Khởi chạy hệ thống.	31
4.1	<code>pom.xml</code> – trích đoạn hai dependency chính.	47
4.2	<code>AlibabaTraceReader.read()</code> – tolerant CSV parsing.	48
4.3	<code>VmAllocationPolicyK8sDefault</code> – Filter → Score → Select.	49
4.4	Manual resource accounting. Đây là <i>nguồn sự thật</i> duy nhất cho utilisation trong Phase 1.	50
4.5	<code>CloudSimEnv.reset</code> – tránh reset thừa lần đầu.	52
4.6	<code>action_masks()</code> – fail-open để không khoá episode.	52
4.7	<code>baseline_eval.evaluate_baseline</code> – tận dụng chung pipeline với RL.	53
4.8	Trick layer cache: <code>pom.xml</code> trước, <code>src/</code> sau.	54
4.9	Bộ khung <code>MetricsRegistry</code> : cờ <code>enabled</code> kiểm soát no-op, mọi mutator wrap try/catch.	56
4.10	Trích <code>docker-compose.yml</code> : profile monitoring.	57
4.11	<code>prometheus.yml</code> – hai job, cho phép <code>rl_agent</code> down.	58
4.12	Lệnh chạy smoke test.	59
5.1	Lệnh chạy smoke test.	62
5.2	Lệnh chạy năm baseline trên ba kịch bản.	64
A.1	Quy trình tối thiểu để dựng lại toàn bộ hệ thống.	81
A.2	Layout of generated data (Phase 1.8).	82
B.1	Placeholder for <code>docker-compose.yml</code>	84
B.2	Placeholder for <code>SimulationManager.java</code>	84
B.3	Placeholder for <code>environment.py</code>	84
B.4	Placeholder for <code>reward.py</code>	85

Chương 1

Giới thiệu

1.1 Bối cảnh và động lực nghiên cứu

Bước vào nửa sau thập kỷ 2020, các mô hình ngôn ngữ lớn (*Large Language Models* – LLM) như GPT-4, LLaMA-3, Gemini hay Claude đã đẩy nhu cầu tính toán trên bộ tăng tốc đồ họa (GPU) lên gấp nhiều lần. Một phiên huấn luyện mô hình tiên tiến hiện nay có thể cần đến hàng nghìn GPU NVIDIA H100 hoặc A100 chạy liên tục trong nhiều tuần; kéo theo đó là sự mở rộng quy mô chưa từng có của các trung tâm dữ liệu (*datacenter*) phục vụ trí tuệ nhân tạo (AI) [1], [2]. Theo báo cáo *Electricity 2024* của Cơ quan Năng lượng Quốc tế (IEA) [3], riêng năm 2022, tổng điện năng mà các trung tâm dữ liệu, AI và tiền mã hoá tiêu thụ đã đạt khoảng 460 TWh — tương đương 1,5% nhu cầu điện toàn cầu — và được dự báo sẽ tăng gấp đôi, chạm mốc 1 000 TWh vào năm 2026, vượt qua tổng tiêu thụ điện của cả nước Nhật Bản.



Hình 1.1: Xu hướng tiêu thụ điện năng của trung tâm dữ liệu, AI và tiền mã hóa trên toàn cầu giai đoạn 2018–2026. Nguồn: IEA [3].

Đi cùng tốc độ mở rộng đó lại là một nghịch lý về hiệu suất sử dụng tài nguyên. Phân tích vết tải (*trace*) công khai của Alibaba PAI Cluster v2023 [4] cho thấy: dù GPU chiếm hơn 80% chi phí phần cứng của một AI cluster, hiệu suất sử dụng GPU trung bình chỉ vào khoảng 25–40%. Nguyên nhân chủ yếu đến từ hiện tượng **phân mảnh tài nguyên** (*Resource Fragmentation*): một host có thể còn dư GPU nhưng hết CPU hoặc RAM để nhận thêm tác vụ, hoặc ngược lại. Hệ quả là xuất hiện những GPU “mồ côi” (*orphan GPUs*) — tiêu tốn điện ở trạng thái *idle* mà

không tạo ra giá trị tính toán nào [5], [6].

Bài toán càng phức tạp khi nhà vận hành đồng thời phải theo đuổi **hai mục tiêu vốn đối nghịch**:

1. **Tiết kiệm năng lượng** — giảm tổng điện năng tiêu thụ và lượng phát thải CO₂ tương ứng;
2. **Đảm bảo Thỏa thuận Mức Dịch vụ** (*Service Level Agreement* – SLA) — hoàn thành tác vụ trước hạn chót, đặc biệt với các tác vụ *LatencySensitive*.

Hai mục tiêu này hiếm khi đi cùng chiều. Gom (*consolidate*) tác vụ vào ít host để tắt bớt máy nhàn rỗi sẽ tiết kiệm điện, nhưng đồng thời làm tăng độ trễ xếp hàng và nguy cơ vi phạm SLA. Ngược lại, dàn tản ra nhiều host giúp giữ SLA, nhưng lại khiến phần lớn máy vận hành ở mức tải thấp và tiêu hao công suất nhàn rỗi.

Riêng tại Việt Nam, khi giá điện công nghiệp tiếp tục được điều chỉnh tăng và các cam kết Net-Zero 2050 ngày càng gây sức ép lên ngành tính toán, việc có một nền tảng nghiên cứu để đánh giá và phát triển các thuật toán lập lịch **vừa tiết kiệm năng lượng vừa đảm bảo chất lượng dịch vụ** trở thành một nhu cầu thực tế, không còn dừng ở mức học thuật.

1.2 Phát biểu bài toán

1.2.1 Đầu vào và đầu ra

Đầu vào. Hệ thống nhận hai loại dữ liệu:

- Một cụm gồm m **host** vật lý. Mỗi host có dung lượng cố định về CPU (đơn vị MIPS), RAM (MB), GPU (số chiếc) và băng thông (Mbps).
- Một dòng **tác vụ** đến theo thời gian. Mỗi tác vụ τ_i đi kèm bộ thông tin: nhu cầu tài nguyên $(r_i^{\text{cpu}}, r_i^{\text{ram}}, r_i^{\text{gpu}})$, thời hạn hoàn thành d_i và nhãn chất lượng dịch vụ $q_i \in \{\text{Burstable}, \text{LatencySensitive}, \text{BestEffort}\}$, tất cả lấy trực tiếp từ Alibaba trace.

Đầu ra. Tại mỗi bước, scheduler đưa ra một quyết định: *đặt tác vụ τ_i lên host nào, hoặc hoãn lại sang bước kế tiếp*. Phép gán này được ký hiệu là $f(\tau_i)$.

Ràng buộc cứng. Tại mọi thời điểm, tổng nhu cầu tài nguyên của các tác vụ đang chạy trên một host không được vượt quá dung lượng host đó:

$$\text{Tổng } r_i^{(k)} \text{ trên host } h_j \leq c_j^{(k)}, \quad k \in \{\text{cpu}, \text{ram}, \text{gpu}\}. \quad (1.1)$$

Đây là điều kiện không cho phép vi phạm: một phép gán vi phạm capacity là phép gán không khả thi, chứ không đơn thuần là phép gán “tệ hơn”. Trong hệ thống, ràng buộc này được thực thi ngay từ không gian hành động thông qua cơ chế **action masking** (xem Chương 3) — RL agent không thể chọn host không đủ tài nguyên.

1.2.2 Hai mục tiêu cần tối ưu cùng lúc

Mục tiêu 1 — Tiết kiệm năng lượng. Tại thời điểm t , mỗi host h_j tiêu thụ một công suất $P_j(t)$ phụ thuộc vào mức sử dụng $U_j(t) \in [0, 1]$. ELDAS áp dụng *mô hình tuyến tính*

SPECpower [7]: công suất bằng P^{idle} khi host nhàn rỗi và tăng tuyến tính lên P^{max} khi $U_j = 1$:

$$P_j(U_j) = P_j^{\text{idle}} + U_j \cdot (P_j^{\text{max}} - P_j^{\text{idle}}). \quad (1.2)$$

Tổng năng lượng của một episode mô phỏng được tính bằng cách cộng dồn công suất theo thời gian trên toàn bộ host:

$$E = \sum_{\text{host}} \sum_{\text{tick}} P_j(t) \Delta t. \quad (1.3)$$

Mục tiêu đặt ra là làm cho E *nhỏ nhất có thể*.

Mục tiêu 2 — Hoàn thành tác vụ trước hạn (SLA). Một tác vụ được coi là “trễ hạn” khi thời điểm hoàn thành C_i vượt qua deadline d_i . Mức độ trễ tương ứng là $\max(0, C_i - d_i)$, nhận giá trị 0 nếu tác vụ xong đúng hạn hoặc xong sớm. Tổng vi phạm trên toàn episode là tổng các mức trễ này, có trọng số theo lớp QoS:

$$V = \sum_{\text{task}} \lambda(q_i) \max(0, C_i - d_i), \quad (1.4)$$

Nhãn QoS	Hệ số phạt λ
LatencySensitive (tác vụ tương tác)	3,0
Burstable (tác vụ thường)	1,0
BestEffort (tác vụ nền)	0,3

Bảng 1.1: Hệ số phạt λ ứng với từng lớp QoS — vi phạm SLA của tác vụ tương tác bị phạt gấp ba lần tác vụ thường.

Tương tự mục tiêu 1, đích cần hướng tới là làm V *nhỏ nhất có thể*.

1.2.3 Vì sao hai mục tiêu này xung đột?

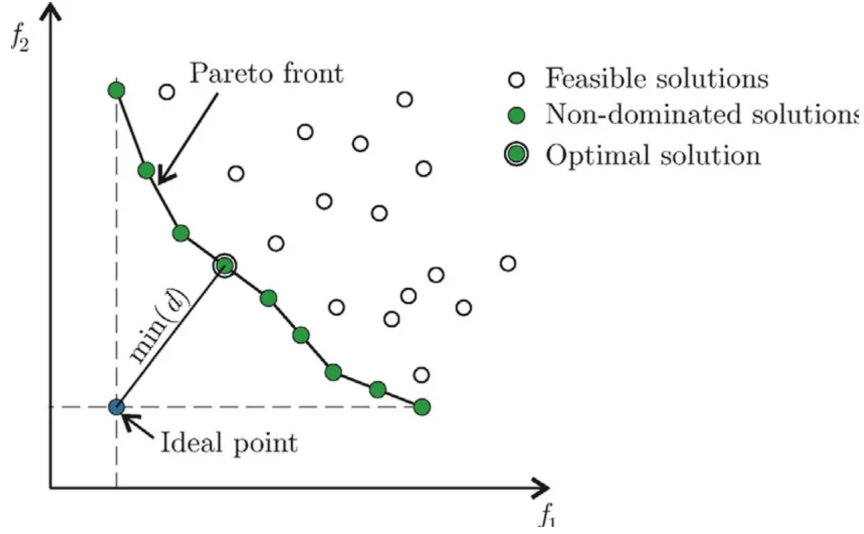
Hai mục tiêu trên không thể cùng đạt cực trị, vì chúng kéo scheduler theo *hai chiều ngược nhau*:

- **Hướng tiết kiệm năng lượng:** dồn tải vào ít host hơn, tắt bớt host nhàn rỗi $\Rightarrow$ các host còn lại bận hơn, hàng đợi dài hơn, độ trễ tăng $\Rightarrow$ *vi phạm SLA tăng theo*.
- **Hướng đảm bảo SLA:** trải tải ra nhiều host, mỗi host chỉ nhận vài tác vụ $\Rightarrow$ phần lớn host vận hành ở mức tải thấp, vẫn phải gánh P^{idle} ngay cả khi gần như không làm gì $\Rightarrow$ *tổng năng lượng tăng lên*.

Vì lý do đó, bài toán không có một “phương án tốt nhất tuyệt đối”, mà chỉ có *một họ các phương án đánh đổi*: phương án A tiết kiệm điện hơn thì thường trễ hơn; phương án B giữ SLA tốt hơn thì lại tốn điện hơn.

Cách so sánh hai phương án. Quy ước thường dùng: phương án A được coi là *tốt hơn* phương án B nếu A *không thua kém* ở cả hai mục tiêu, đồng thời *tốt hơn rõ rệt ở ít nhất một mục tiêu*. Nếu A tốt hơn ở mục tiêu này nhưng kém hơn ở mục tiêu kia, hai phương án không thể so sánh trực tiếp; cả hai đều là *ứng cử viên hợp lệ*, việc chọn phương án nào tùy thuộc chính

sách của nhà vận hành. Tập tất cả các phương án không bị bất kỳ phương án nào khác “vượt qua” theo nghĩa trên chính là **Pareto front** — đường biên mà ELDAS đặt mục tiêu xấp xỉ.



Hình 1.2: Trade-off giữa 2 mục tiêu. Mỗi điểm là một chính sách lập lịch; đường cong nối các điểm không bị thống trị (Pareto front) là kết quả mà Phase 2 hướng tới xấp xỉ.

1.2.4 Độ phức tạp tính toán

Ngay ở dạng đơn giản nhất — gán n tác vụ lên m host với một chiều tài nguyên duy nhất — bài toán đã tương đương với *Bin Packing*, vốn là một bài toán NP-hard [8]. Khi mở rộng sang nhiều chiều tài nguyên, nhiều mục tiêu và đặc tính *online* (tác vụ đến theo thời gian thực), việc tìm lời giải tối ưu chính xác trong thời gian đa thức là không khả thi. Đây cũng là lý do nhiều nghiên cứu gần đây chuyển sang hướng **học tăng cường đa mục tiêu** (*Multi-Objective Reinforcement Learning* — MORL), trong đó agent học một chính sách (*policy*) gần tối ưu thông qua tương tác với môi trường mô phỏng [9], [10].

1.3 Mục tiêu đề tài

1.3.1 Mục tiêu tổng thể

Đề tài **ELDAS** (*Energy- and Latency-aware Datacenter AI Scheduler*) hướng đến việc xây dựng một **nền tảng mô phỏng và huấn luyện thuật toán MORL** cho bài toán lập lịch tác vụ AI trong trung tâm dữ liệu. Nền tảng này cần đáp ứng được các yêu cầu sau:

- Mô phỏng đủ trung thực hành vi của một cụm máy chủ AI ở quy mô vừa;
- Nạp được dữ liệu workload thực, thay vì phải sinh ngẫu nhiên;
- So sánh công bằng nhiều thuật toán lập lịch (heuristic và RL) trên cùng một bộ kịch bản;
- Cung cấp các chỉ số định lượng đa mục tiêu để dựng Pareto front.

1.3.2 Mục tiêu Giai đoạn 1 (phạm vi báo cáo này)

Báo cáo này tập trung trình bày sáu sản phẩm kỹ thuật của Giai đoạn 1, mỗi sản phẩm là một kết quả có thể kiểm chứng độc lập:

1. **MT1 — Bộ mô phỏng datacenter có theo dõi GPU.** Dựng môi trường mô phỏng cụm máy chủ trên nền CloudSim Plus, bổ sung lớp theo dõi GPU (CloudSim Plus chưa hỗ trợ sẵn) cùng mô hình công suất tuyến tính cho cả CPU lẫn GPU.
2. **MT2 — Tích hợp workload thực.** Bộ parser đọc trực tiếp Alibaba PAI Cluster Trace v2023, kèm bộ lọc tạo ba kịch bản tải *Low* / *High* / *Burst* để phục vụ so sánh.
3. **MT3 — Cầu nối Java–Python.** Dùng Py4J cho phép phía Python điều khiển toàn bộ vòng đời mô phỏng (reset / step / pause / resume), phù hợp với đặc thù tương tác từng bước của RL.
4. **MT4 — Môi trường RL theo chuẩn MO-Gymnasium.** Tín hiệu thưởng được giữ dưới dạng **vector hai chiều** (R^{energy} , R^{SLA}), không gộp về một đại lượng vô hướng; ràng buộc capacity được áp đặt cứng thông qua action masking.
5. **MT5 — Năm baseline cổ điển và một agent MORL PPO tối thiểu.** Năm baseline trải đều ba cụm chiến lược — pack-tight (*FirstFit*, *BestFit*), trung gian (*Random*) và spread (*Kubernetes LRP*, *RoundRobin*) — tạo ba cụm Pareto rõ rệt làm *mốc tham chiếu* cho Phase 2. Bên cạnh đó, một agent MORL PPO tối thiểu (MaskablePPO, 100k step) đặt một điểm nằm bên trong Pareto front nhằm xác nhận pipeline RL hoạt động đúng.
6. **MT6 — Đóng gói cho tái lập kết quả.** Toàn bộ hệ thống được đóng gói trong Docker Compose; với cùng seed, kết quả phải khớp đến từng bit, kể cả khi chạy trên máy khác.

1.3.3 Mục tiêu Giai đoạn 2 (kế hoạch tiếp nối)

Giai đoạn 2 kế thừa nền tảng MT1–MT6, mở rộng theo bốn hướng:

1. Huấn luyện agent PPO + MLP, đưa hai mục tiêu về một đại lượng vô hướng bằng tổng có trọng số: $R = w \cdot R^{\text{energy}} + (1 - w) \cdot R^{\text{SLA}}$;
2. Quét trọng số w qua các giá trị $\{0,1; 0,2; \dots; 0,9\}$ để thu được nhiều điểm trên Pareto front;
3. So sánh agent MORL với năm baseline cổ điển trên cả ba kịch bản tải, theo các chỉ số: năng lượng, makespan, tỉ lệ vi phạm SLA và số lần wake-up host;
4. Tinh chỉnh siêu tham số qua Weights & Biases; thử thay MLP bằng Graph Neural Network nếu điều kiện thời gian cho phép.

1.4 Phạm vi và giới hạn

Để bảo đảm tính khả thi trong khuôn khổ một đề án chuyên ngành, đề tài giới hạn phạm vi như sau:

Về quy mô. Cụm mô phỏng có 5 host (mỗi host gồm 32 vCPU, 128 GB RAM, 4 GPU NVIDIA A100); workload đầu vào trích từ Alibaba trace với cỡ 1 000–5 000 tác vụ. Quy mô này vừa đủ để bộc lộ hiện tượng phân mảnh tài nguyên và xung đột giữa hai mục tiêu, đồng thời giữ cho thời gian huấn luyện mỗi episode ở mức chấp nhận được trên phần cứng phòng thí nghiệm.

Về môi trường. Toàn bộ thực nghiệm được tiến hành trong môi trường **mô phỏng** với CloudSim Plus, không triển khai trên cụm Kubernetes thật. Nguyên nhân là một chu trình huấn luyện RL cần hàng triệu bước tương tác, không thể thực hiện trên hạ tầng vật lý trong khoảng thời gian của đề án.

Về những vấn đề không nằm trong phạm vi. Đề án **không xét** các nội dung sau: (i) di chuyển máy ảo trực tiếp (*live VM migration*); (ii) lập lịch liên trung tâm dữ liệu (*multi-datacenter scheduling*); (iii) mô hình mạng chi tiết (topology, mô phỏng tắc nghẽn); (iv) lập lịch các tác vụ phân tán có giao tiếp tập thể (*collective communication* trong distributed training).

Về tập ràng buộc được thực thi trong Phase 1. Phát biểu bài toán ở §1.2 có nhắc đến năm chiều ràng buộc tài nguyên (CPU, RAM, GPU, băng thông, nhiệt độ). Trong Phase 1, đề án **chỉ áp đặt cứng** ba chiều CPU/RAM/GPU (xem §4.1.4); hai chiều còn lại được *hoãn có chủ đích* sang Phase 2, với lý do cụ thể như sau:

- **Băng thông mạng.** Bản thân Alibaba PAI v2023 không cung cấp trường `bandwidth_demand` ở mức task. Muốn thêm ràng buộc băng thông ngay từ Phase 1 thì phải tự sinh giá trị giả định (*synthetic*, ví dụ lấy tỉ lệ theo `cpu_milli`); cách làm này vừa làm giảm khả năng tái lập kết quả, vừa đưa thêm một biến số không kiểm soát vào hệ thống, trong khi mục tiêu của Phase 1 là *kiểm chứng kiến trúc bốn lớp*. Bên cạnh đó, mỗi chiều ràng buộc bổ sung sẽ kéo theo H feature mới trong observation (một feature/host); với $H = 10$, điều này làm chậm quá trình hội tụ của PPO mà không thực sự cần thiết khi thiết kế MORL còn đang được hiệu chỉnh. Phase 2 sẽ kích hoạt `host.getBwProvisioner()` sẵn có của CloudSim Plus (bổ sung khoảng 20 dòng vào `VmAllocationPolicy.find()`) sau khi đã có dữ liệu bandwidth thực hoặc một mô hình synthetic được biện luận đầy đủ.
- **Nhiệt độ host.** Một mô hình nhiệt sơ bộ cần ba thành phần: (a) mạng RC nhiệt — dạng đơn giản nhất là $T_j(t) = T_{\text{amb}} + \theta_{\text{JA}} \cdot P_j(t)$; (b) đường cong fan kèm ngưỡng throttle; (c) tương tác hai chiều với DVFS. Cả ba thành phần này tự thân đã là một hướng nghiên cứu phụ. Quan trọng hơn, mô hình năng lượng ở Phase 1 vẫn còn tuyến tính (xem §2.2) và chưa có DVFS; nếu thêm thermal lúc này, kết quả sẽ là *một lớp giả định chồng lên lớp giả định khác*, không mang giá trị khoa học. Phase 2 dự kiến đưa DVFS và thermal vào cùng một gói nhất quán: trước hết thay (1.2) bằng mô hình bậc hai có DVFS, sau đó bổ sung $T_j(t)$ vào observation cùng một filter `HostFitsThermal` theo phong cách K8s predicate (cấm gán task lên host có $T_j > T_{\text{max}}$).

Cách phân chia này dựa trên nguyên tắc *minimum viable scope*: Phase 1 ưu tiên các ràng buộc **có dữ liệu thực** (CPU/RAM/GPU đều có sẵn trong trace), nhờ đó mọi con số trong báo cáo đều có thể truy nguồn được; những ràng buộc **phải dựa trên giả định** chỉ được đưa vào

sau khi mô hình nền (power, DVFS) đủ chính xác để không xảy ra tình trạng giả định chồng lên giả định.

Về mô hình năng lượng. Đồ án sử dụng mô hình tuyến tính (1.2) cho cả CPU và GPU. Các mô hình phi tuyến (bậc ba, logarit) cùng cơ chế DVFS (*Dynamic Voltage and Frequency Scaling*) được ghi nhận như hướng mở rộng cho những giai đoạn sau.

Về thuật toán. Phase 1 hiện thực năm baseline cổ điển (FirstFit, BestFit, RoundRobin, K8s LRP, Random) cùng một agent MORL PPO tối thiểu. Phase 2 sẽ mở rộng PPO theo hai hướng — tinh chỉnh siêu tham số và quét trọng số cho linear scalarization — đồng thời đối chuẩn với hai thuật toán MORL hiện đại (PCN, Envelope Q-learning hoặc CAPQL — lựa chọn cụ thể sẽ được chốt sau bước khảo sát literature). Các biến thể dùng Graph Neural Network được giữ ở phần *stretch goals*.

1.5 Cấu trúc báo cáo

Báo cáo gồm bảy chương, được tổ chức như sau:

Chương 1 — Giới thiệu. Trình bày bối cảnh, phát biểu bài toán lập lịch đa mục tiêu một cách hình thức, kèm mục tiêu cụ thể, phạm vi và giới hạn của đề tài.

Chương 2 — Cơ sở lý thuyết và nghiên cứu liên quan. Tổng quan ba mảng kiến thức: (i) các thuật toán lập lịch truyền thống và Kubernetes scheduler; (ii) mô hình tiêu thụ năng lượng trong datacenter; (iii) học tăng cường đơn và đa mục tiêu, kèm khảo sát ít nhất ba công trình tiêu biểu.

Chương 3 — Kiến trúc hệ thống. Mô tả kiến trúc bốn lớp của ELDAS (Docker Compose, CloudSim Plus, Py4J Gateway, MO-Gymnasium), các luồng dữ liệu chính và những quyết định thiết kế then chốt.

Chương 4 — Hiện thực. Trình bày chi tiết quá trình hiện thực Phase 1: phía Java gồm SimulationConfig, DatacenterFactory, AlibabaTraceReader, SimulationManager (lifecycle pause/resume/reset), GatewayEntryPoint và hai VmAllocationPolicy baseline; phía Python gồm state_builder, reward, environment và tracker.

Chương 5 — Kết quả Phase 1. Báo cáo kết quả của *smoke test* vòng RL hoàn chỉnh, kiểm chứng tính đúng của cầu nối Java–Python, đồng thời đo các chỉ số ban đầu (năng lượng, SLA) của hai baseline trên cả ba kịch bản tải.

Chương 6 — Kế hoạch Phase 2. Lộ trình huấn luyện PPO, kế hoạch weight sweep, tinh chỉnh siêu tham số và đánh giá Pareto front, kèm sơ đồ Gantt và phân tích rủi ro.

Chương 7 — Kết luận. Tổng kết đóng góp của Phase 1, các bài học kinh nghiệm và hướng phát triển tiếp theo.

Cuối báo cáo, phần *Phụ lục* cung cấp toàn bộ tệp cấu hình (`docker-compose.yml`, `.env`), các đoạn mã đại diện và hướng dẫn chạy lại thực nghiệm.

Chương 2

Cơ sở lý thuyết và nghiên cứu liên quan

Chương này tập hợp nền tảng lý thuyết cho ELDAS theo bốn mảng: (i) các thuật toán lập lịch trong trung tâm dữ liệu; (ii) mô hình tiêu thụ năng lượng; (iii) học tăng cường đa mục tiêu (MORL); và (iv) các công trình nghiên cứu liên quan. Phần trình bày được tiết chế ở mức vừa đủ để người đọc nắm được các quyết định thiết kế ở Chương 1 cũng như phần hiện thực ở các chương sau, đồng thời định vị được đóng góp của đề tài trong tương quan với các nghiên cứu đã có.

2.1 Lập lịch trong trung tâm dữ liệu

2.1.1 Các thuật toán heuristic truyền thống

Bài toán lập lịch tài nguyên đã được nghiên cứu từ rất sớm, dưới dạng *Bin Packing*: xếp n vật phẩm có kích thước cho trước vào số thùng nhỏ nhất, mỗi thùng có dung lượng cố định [8]. Một số *heuristic* kinh điển vẫn được sử dụng phổ biến cho đến nay:

- **First-Fit (FF)**. Duyệt danh sách host theo thứ tự cố định, đặt tác vụ lên host đầu tiên còn đủ tài nguyên. Cách làm đơn giản, độ phức tạp $O(n \cdot m)$, nhưng dễ gây phân mảnh.
- **Best-Fit (BF)**. Đặt tác vụ lên host *vừa khít nhất* — tức host có tài nguyên dư còn lại nhỏ nhất sau khi đặt. Giảm phân mảnh nhưng tốn chi phí sắp xếp.
- **Worst-Fit (WF)**. Ngược lại với BF: chọn host còn dư nhiều nhất để cân bằng tải, đổi lại làm tăng số host phải bật.
- **Round-Robin (RR)**. Phân phối tuần tự, bảo đảm công bằng nhưng bỏ qua sự khác biệt về dung lượng giữa các host.

Điểm chung của các heuristic này là **đơn mục tiêu** (single-objective): hoặc tối thiểu số host, hoặc cân bằng tải, không tính đến năng lượng hay SLA. Đây là hạn chế đáng kể khi áp dụng cho các datacenter AI hiện đại.

2.1.2 Meta-heuristic: Ant Colony Optimization (ACO)

Để vượt qua hai giới hạn cố hữu của các heuristic “nhìn một bước” (FF, BF) là *đơn mục tiêu* và *dễ kẹt ở cực tiểu địa phương*, nhiều nghiên cứu chuyển sang nhóm **meta-heuristic**; trong đó *Ant Colony Optimization* (ACO) là đại diện được khảo sát rộng rãi nhất cho bài toán lập lịch tài nguyên [11].

Ý tưởng cốt lõi. ACO mô phỏng cách đàn kiến tìm đường ngắn nhất tới nguồn thức ăn dựa trên *pheromone trail*: mỗi con kiến đi qua một đoạn đường sẽ để lại một lượng pheromone; đoạn nào tích lũy nhiều pheromone hơn sẽ được các con kiến sau ưu tiên chọn, tạo thành vòng phản hồi dương và dần hội tụ về đường tối ưu. Khi ánh xạ sang bài toán scheduling, mỗi “con kiến” xây một schedule hoàn chỉnh bằng cách chọn host cho từng task theo xác suất tỉ lệ thuận với lượng pheromone đã tích lũy và chất lượng ước tính của cặp (*task*, *host*). Sau mỗi vòng, pheromone bay hơi một phần rồi được củng cố theo mức độ tốt của schedule vừa tìm ra, hình thành “bộ nhớ tập thể” để định hướng cho các kiến ở vòng sau. Liu và cộng sự áp dụng biến thể *Energy-aware ACO* cho bài toán VM placement và giảm được 9–12% năng lượng so với First-Fit Decreasing trên cùng một bộ workload trace [12].

Ưu điểm. (i) khả năng *thoát khỏi cực tiểu địa phương* nhờ tính ngẫu nhiên trong lựa chọn của từng kiến; (ii) hỗ trợ multi-objective một cách tự nhiên thông qua hàm chi phí tổng hợp; (iii) không cần gradient, có thể hoạt động trên không gian rời rạc rộng.

Hạn chế đối với ELDAS. Trong phạm vi đề án này, ACO bộc lộ ba nhược điểm đáng kể:

1. **Chi phí đánh giá rất lớn.** ACO thuộc nhóm population-based: mỗi iteration cần K kiến ($K \approx 20\text{--}50$) $\times$ một episode mô phỏng đầy đủ. Với throughput đo được khoảng 65 step/s (§5.2.1) và ~ 2000 task/episode, một iteration ACO tiêu tốn khoảng $30\text{ s} \times 30$ kiến ≈ 15 phút; 100 iteration hội tụ tương đương ~ 25 giờ *cho một bộ siêu tham số duy nhất*.
2. **Khó khái quát hoá.** Ma trận pheromone τ_{ij} được học riêng cho một workload cụ thể; khi gặp workload mới, phải chạy lại từ đầu. Ngược lại, một chính sách RL đã huấn luyện có khả năng khái quát hoá nhờ hàm xấp xỉ.
3. **Nhiều siêu tham số.** ACO có khá nhiều tham số cần chỉnh ($\alpha, \beta, \rho, Q, K$, chiến lược elitism, ...), khiến khả năng tái lập kết quả thấp và việc so sánh công bằng giữa các nghiên cứu trở nên khó khăn.

So sánh với PPO (định hướng của ELDAS). PPO khắc phục được cả ba điểm yếu kể trên: (i) tuy là on-policy, PPO cập nhật ngay sau *mỗi episode* thay vì sau *cả một quần thể episode*, tiết kiệm khoảng K lần thời gian mô phỏng; (ii) khái quát hoá qua mạng nơ-ron — một chính sách có thể dùng cho nhiều workload; (iii) chỉ có một vài siêu tham số chính (learning rate, clip ϵ , GAE λ). Do đó, ACO chỉ được nhắc đến như *tham chiếu so sánh* trong phần khảo sát, không được hiện thực trong ELDAS.

2.1.3 Kubernetes Scheduler

Kubernetes (K8s) là hệ điều phối container phổ biến nhất hiện nay [13]. Bộ lập lịch mặc định (*kube-scheduler*) hoạt động theo *pipeline* ba bước cho mỗi *pod*:

1. **Filter (Predicate).** Loại bỏ các node không đủ điều kiện. Một số bộ lọc tiêu biểu: *PodFitsResources* (đủ CPU/RAM/GPU), *NodeAffinity*, *Taints/Tolerations*.
2. **Score (Priority).** Chấm điểm từng node còn lại trên thang $[0, 10]$ bằng các hàm tính điểm. Hai hàm được dùng trong ELDAS để mô phỏng K8s:

$$\text{LRP}(h_j) = 10 \cdot \frac{(c_j^{\text{cpu}} - u_j^{\text{cpu}}) + (c_j^{\text{ram}} - u_j^{\text{ram}})}{2(c_j^{\text{cpu}} + c_j^{\text{ram}})},$$

gọi là *LeastRequestedPriority* – ưu tiên host có nhiều tài nguyên trống (cân bằng tải). Hàm thứ hai *BalancedResourceAllocation* ưu tiên host có tỉ lệ CPU/RAM sử dụng đồng đều, nhằm tránh nghẽn cổ chai một chiều tài nguyên.

3. **Bind.** Gắn pod vào node có điểm số cao nhất.

Hạn chế. Bộ lập lịch của K8s không có hàm chấm điểm theo năng lượng, và bản thân nó không hỗ trợ tối ưu đa mục tiêu một cách nguyên thủy. Đây cũng chính là khoảng trống mà các phương pháp dựa trên học tăng cường có thể khai thác.

2.1.4 Lập lịch dựa trên học tăng cường: Deep Q-Network (DQN)

Kết quả đột phá của Mnih và cộng sự trên bộ trò chơi Atari năm 2015 [14] đã đưa *Deep Q-Network* (DQN) trở thành một trong những thuật toán RL được khảo sát phổ biến nhất cho bài toán scheduling trong giai đoạn sau đó. DQN học hàm giá trị **Q-function** $Q(s, a)$ — ước lượng tổng reward kỳ vọng khi thực hiện hành động a từ trạng thái s . Thay vì tính trực tiếp, DQN dùng một mạng nơ-ron để xấp xỉ Q và cập nhật theo nguyên lý Bellman: phần thưởng hiện tại cộng với giá trị tốt nhất có thể đạt được ở bước kế tiếp. Hai kỹ thuật then chốt giúp quá trình học ổn định là **target network** (một bản sao mạng được đóng băng định kỳ để tránh hiện tượng “mục tiêu di động”) và **replay buffer** (lưu lại kinh nghiệm rồi lấy mẫu ngẫu nhiên, phá vỡ tương quan thời gian giữa các bước liên tiếp).

Ưu điểm. (i) **Tiết kiệm mẫu hơn so với nhóm on-policy:** cùng một mẫu kinh nghiệm có thể được tái sử dụng nhiều lần qua replay buffer — một lợi thế rõ rệt khi mỗi episode mô phỏng CloudSim đều tốn kém; (ii) **phù hợp tự nhiên với không gian hành động rời rạc:** phép $\arg \max_a Q$ là toán tử kiểm soát mặc định, ăn khớp với $H = 10$ lựa chọn host; (iii) biến thể *Rainbow DQN* (kết hợp PER, Double DQN, Dueling, Distributional) hiện đang giữ SOTA trên nhiều benchmark.

Hạn chế đối với ELDAS. Có bốn lý do khiến DQN *không* phải lựa chọn tối ưu cho phạm vi Phase 2:

1. **Reward lệch biên độ dẫn tới ước lượng Q thiên dương.** Reward energy trong ELDAS luôn âm và có biên độ vào khoảng 10^3 Ws, trong khi reward SLA thừa hơn và biên độ phụ thuộc vào mức vi phạm. DQN chọn hành động dựa trên $\arg \max_a Q$, nên dễ bị *thiên dương* (overestimation) khi phân phối Q có hai thành phần chênh nhau tới hai bậc độ lớn. PPO xử lý được vấn đề này bằng *advantage normalisation* — chuẩn hoá A_t về trung bình 0, độ lệch chuẩn 1 trước mỗi bước cập nhật.
2. **Action masking phức tạp hơn.** Các host không đủ tài nguyên cần bị cấm tuyệt đối. Trong DQN, điều này phải được thực thi ở hai vị trí — bước chọn hành động theo ϵ -greedy và bước $\max_{a'}$ trong Bellman backup — với nhiều biến thể chưa được chuẩn hoá trong literature. Trong khi đó, với PPO chỉ cần mask một lần tại policy head, và thư viện `sb3-contrib` đã có sẵn `MaskablePPO`.
3. **Không hỗ trợ vector reward một cách nguyên thủy.** Q -function trả về vô hướng cho mỗi cặp (s, a) ; muốn dùng cho MORL thì phải *scalarize trước khi học*, đồng nghĩa với việc phải huấn luyện lại từ đầu cho từng trọng số $\mathbf{w}$ — mức chi phí ngang bằng với linear scalarization của PPO. Các biến thể MORL chuyên dụng như *Envelope-Q* hay *Pareto-DQN* có tồn tại, nhưng phức tạp hơn, chưa có triển khai sẵn trên SB3 và do đó nằm ngoài phạm vi Phase 2.
4. **Mô hình off-policy + replay buffer khó song song hoá.** Phase 2 dự kiến chạy weight sweep ($K = 9$) qua `SubprocVecEnv` để tận dụng nhiều core CPU; PPO on-policy ăn khớp tự nhiên với kiến trúc này, trong khi DQN đòi hỏi các kiến trúc phức tạp hơn nhiều như Ape-X hay R2D2.

So sánh với PPO (định hướng của ELDAS). Trước phổ reward lệch biên độ, PPO ổn định hơn DQN nhờ hai cơ chế mà DQN không có tương đương: *advantage normalisation* — triệt tiêu chênh lệch biên độ trước mỗi lần cập nhật — và *trust region thông qua clipping* — giới hạn mức thay đổi của chính sách ở mỗi bước, tránh hiện tượng diverge khi vi phạm SLA bất ngờ tăng vọt. Phần chi tiết về PPO được trình bày ở §2.3.

Bảng 2.1: So sánh các thuật toán lập lịch truyền thống.

Thuật toán	Mục tiêu	Đa mục tiêu?	Năng lượng?
First-Fit	Min #host	Không	Không
Best-Fit	Min phân mảnh	Không	Không
Round-Robin	Cân bằng tải	Không	Không
K8s LRP	Cân bằng tải	Không	Không
NSGA-II	Energy + QoS	Có	Có
DRL (DeepRM)	Slowdown	Không	Không
MORL (ELDAS)	Energy + SLA	Có	Có

2.2 Mô hình tiêu thụ năng lượng datacenter

2.2.1 Mô hình tuyến tính SPECpower

Phép đo thực nghiệm trên hơn 200 máy chủ thương mại do tổ chức SPEC công bố (SPECpower_{ssj2008}) cho thấy công suất tiêu thụ của một máy chủ phụ thuộc gần như tuyến tính vào mức sử dụng CPU [7]. Mô hình đơn giản và được dùng rộng rãi nhất là:

$$P(U) = P_{\text{idle}} + U \cdot (P_{\text{max}} - P_{\text{idle}}), \quad (2.1)$$

trong đó:

- $U \in [0, 1]$ – mức sử dụng CPU trung bình của host;
- P_{idle} – công suất ở trạng thái rảnh ($U = 0$);
- P_{max} – công suất ở trạng thái tải tối đa ($U = 1$).

Có một đặc điểm đáng chú ý: P_{idle} thường chiếm tới 50–70% của P_{max} . Nói cách khác, một host ở trạng thái *rảnh* vẫn tiêu tốn hơn phân nửa công suất so với khi chạy hết tải. Đây cũng là cơ sở để các thuật toán *consolidation* ưu tiên gom tác vụ về ít host nhất có thể rồi tắt số host còn lại.

2.2.2 Tính tổng năng lượng theo thời gian rời rạc

Mô phỏng trong ELDAS chạy theo **thời gian rời rạc** với bước $\Delta t = 1$ s. Tổng năng lượng tiêu thụ của một host trong K bước được tính bằng phép cộng dồn:

$$E_j = \sum_{k=1}^K P_j(U_j^{(k)}) \cdot \Delta t, \quad (2.2)$$

và năng lượng toàn cụm là $E = \sum_{j=1}^m E_j$. Cách rời rạc hoá này ánh xạ trực tiếp xuống *event loop* của CloudSim Plus (xem Chương 4), nên không cần đến phép tích phân giải tích.

2.2.3 Mở rộng cho GPU

CloudSim Plus phiên bản 8.5.7 chưa có khái niệm GPU một cách nguyên thủy. ELDAS bổ sung thêm một lớp theo dõi GPU riêng, dùng chung dạng công thức tuyến tính với hai mức công suất $P_{\text{idle}}^{\text{gpu}}$ và $P_{\text{max}}^{\text{gpu}}$ lấy từ tài liệu kỹ thuật của NVIDIA A100 [15].

2.2.4 Power Usage Effectiveness (PUE)

Để phản ánh chi phí năng lượng *vận hành* (làm mát, chiếu sáng, mạng), ngành công nghiệp dùng chỉ số PUE:

$$\text{PUE} = \frac{E_{\text{total}}}{E_{\text{IT}}}, \quad (2.3)$$

với E_{IT} là năng lượng tiêu thụ bởi thiết bị tính toán và E_{total} là tổng năng lượng đầu vào của datacenter. PUE lý tưởng là 1,0; trung bình toàn ngành 2023 vào khoảng 1,58 [16]. ELDAS sử dụng giá trị mặc định $\text{PUE} = 1,5$ để quy đổi E_{IT} sang chi phí thực tế khi báo cáo.

2.2.5 Lý do chọn mô hình tuyến tính cho Phase 1

Việc dừng lại ở mô hình tuyến tính trong Phase 1 dựa trên ba lý do:

1. **Đủ chính xác cho mục đích so sánh tương đối.** Sai số trung bình so với phép đo thực nghiệm chỉ vào khoảng 5–7% [7] — đủ để xếp hạng các scheduler.
2. **Đơn giản và dễ tái lập.** Mỗi loại thiết bị chỉ cần hai tham số, vừa dễ tinh chỉnh, vừa minh bạch khi viết lại trong báo cáo.
3. **Tránh phụ thuộc vào DVFS.** Các mô hình phi tuyến (bậc ba, logarit) và cơ chế *Dynamic Voltage and Frequency Scaling* (DVFS) cần dữ liệu tần số/điện áp theo thời gian thực — Alibaba trace không cung cấp các trường này, do đó phần mở rộng được giữ lại cho Phase 2 hoặc cho hướng nghiên cứu sau.

2.3 Học tăng cường đa mục tiêu (MORL)

2.3.1 Markov Decision Process (MDP)

Học tăng cường (*Reinforcement Learning* — RL) mô hình hoá bài toán ra quyết định tuần tự dưới dạng *Markov Decision Process* [17]. Ở mỗi bước, **agent** quan sát **trạng thái** của môi trường, chọn một **hành động**, nhận về một **phần thưởng** rồi chuyển sang trạng thái mới; quá trình này được lặp lại cho đến khi episode kết thúc. Mục tiêu của agent là học một **chính sách** — một ánh xạ từ trạng thái sang hành động — sao cho tổng phần thưởng tích lũy (có chiết khấu theo thời gian) đạt giá trị lớn nhất.

2.3.2 Multi-Objective MDP (MOMDP)

Khi xuất hiện **nhiều mục tiêu xung đột**, phần thưởng không còn là một số vô hướng đơn lẻ mà trở thành một **vector phần thưởng** [9], [18]. Trong ELDAS, ở mỗi bước agent nhận đồng thời hai phần thưởng riêng biệt: R^{energy} — mức năng lượng tiêu thụ trong bước đó, luôn mang giá trị âm — và R^{SLA} — mức vi phạm deadline, bằng 0 nếu tác vụ xong đúng hạn.

2.3.3 Pareto Optimality

Do không thể cộng trực tiếp hai mục tiêu khác đơn vị (Joule và giây trễ), khái niệm tối ưu phải được mở rộng thành **Pareto optimality** (đã được định nghĩa ở §1.2). Trong không gian giá trị kỳ vọng của vector phần thưởng, tập các chính sách không bị thống trị tạo thành *Pareto front* $\mathcal{P}^*$.

Vì sao cần vector reward thay vì giá trị vô hướng? Nếu cộng gộp ngay từ đầu $R = w_1 R_1 + w_2 R_2$ với một $\mathbf{w}$ cố định, agent chỉ học được *một điểm* duy nhất trên Pareto front. Giữ phần thưởng ở dạng vector mang lại hai lợi ích: (i) có thể quét trọng số sau khi huấn luyện để dựng lại nhiều điểm trên Pareto front; (ii) có thể áp dụng các thuật toán MORL chuyên biệt như *Envelope Q-Learning* hay *Pareto-DQN* trong các nghiên cứu kế tiếp.

2.3.4 Proximal Policy Optimization (PPO)

PPO [19] hiện là một trong những thuật toán *policy gradient* ổn định nhất, và cũng là lựa chọn cho Phase 2 của ELDAS. Ý tưởng chính của PPO khá đơn giản: ở mỗi vòng cập nhật, giữ độ *lệch* của chính sách mới so với chính sách cũ trong một *vùng tin cậy*. Thay vì đi giải một bài toán ràng buộc phức tạp, PPO áp dụng cơ chế **cắt tỉ số** (clipping): tỉ số xác suất giữa hai chính sách bị giới hạn trong khoảng $[1 - \varepsilon, 1 + \varepsilon]$ (thường lấy $\varepsilon = 0,2$), nhờ đó mỗi bước cập nhật không thể kéo chính sách thay đổi quá lớn và gây *diverge*.

Vì sao chọn PPO? (i) ổn định nhất trong nhóm policy gradient; (ii) ít siêu tham số, dễ tinh chỉnh; (iii) được hỗ trợ tốt trong `stable-baselines3`; (iv) đã chứng minh hiệu quả trên các bài toán scheduling tương tự [10], [20].

2.3.5 Linear Scalarization và quét trọng số

Cách đơn giản nhất để tối ưu một vector phần thưởng là **cộng có trọng số**:

$$R_t^{\text{scalar}}(\mathbf{w}) = w \cdot R_t^{\text{energy}} + (1 - w) \cdot R_t^{\text{SLA}}, \quad w \in [0, 1]. \quad (2.4)$$

Mỗi giá trị w tương ứng với một bài toán RL đơn mục tiêu chuẩn, agent PPO sẽ hội tụ về một chính sách π_w^* tối ưu cục bộ tại điểm vận hành đó. Bằng cách **quét trọng số** $w \in \{0,1; 0,2; \dots; 0,9\}$ và huấn luyện độc lập chín agent, đồ án thu được chín điểm xấp xỉ trên Pareto front. Phương pháp này có hai lợi thế: (i) tận dụng được mọi thuật toán RL đơn mục tiêu sẵn có; (ii) dễ song song hoá.

Hạn chế. Linear scalarization không thể khôi phục được những đoạn *lõm* (concave) của Pareto front. Đây là điều cần lưu ý khi diễn giải kết quả ở Phase 2.

2.3.6 Action Masking

Trong môi trường ELDAS, không phải hành động nào cũng hợp lệ tại mỗi trạng thái: nếu host h_j không đủ tài nguyên cho tác vụ τ_i , hành động “đặt τ_i lên h_j ” phải bị cấm. Cách xử lý phổ biến trong RL là **action masking** [21]: ở mỗi bước, môi trường trả về kèm theo một danh sách các host khả thi, và những host không hợp lệ bị loại khỏi phân phối xác suất *trước khi* agent chọn hành động — nói cách khác, xác suất chọn một host không đủ tài nguyên bằng đúng 0. ELDAS sử dụng *MaskablePPO* trong `sb3-contrib` — thư viện đã hiện thực sẵn cơ chế này. Cách làm đem lại hai lợi ích: (i) loại bỏ hoàn toàn khả năng vi phạm ràng buộc capacity; (ii) agent học nhanh hơn nhờ không phải lãng phí mẫu vào các hành động sẽ bị phạt cứng.

2.4 Nghiên cứu liên quan

Phần này khảo sát năm công trình tiêu biểu, thuộc ba hướng nghiên cứu liên quan trực tiếp đến ELDAS: RL cho lập lịch tài nguyên, lập lịch đa mục tiêu kèm yếu tố năng lượng, và các framework mô phỏng / MORL.

2.4.1 DeepRM (Mao và cộng sự, 2016)

DeepRM [10] là một trong những công trình đầu tiên áp dụng *Deep Reinforcement Learning* cho bài toán cấp phát tài nguyên cluster. Đóng góp chính của bài báo gồm: (i) biểu diễn trạng thái cluster bằng *ảnh hai chiều* (thời gian $\times$ tài nguyên), từ đó tận dụng được kiến trúc CNN; (ii) chỉ ra rằng agent DRL có thể hội tụ và vượt heuristic Shortest-Job-First trong môi trường mô phỏng. **Hạn chế:** chỉ tối thiểu một mục tiêu duy nhất là *average job slowdown*, không xét đến năng lượng và chỉ thử nghiệm ở quy mô nhỏ (≤ 20 tác vụ đồng thời).

2.4.2 Decima (Mao và cộng sự, 2019)

Decima [20] mở rộng hướng đi của DeepRM, dùng *Graph Neural Network* (GNN) để mô hình hoá Directed Acyclic Graph của các Spark job. Hệ thống đã được triển khai thực tế trên cụm 25 máy và giảm *average job completion time* 21% so với bộ lập lịch mặc định của Spark. **Liên hệ với ELDAS:** cùng triết lý “RL + simulation + deployment”; việc đưa GNN vào ELDAS được xếp ở mục *stretch goal* trong Phase 2, sẽ chỉ tiến hành sau khi MLP đã hội tụ.

2.4.3 NSGA-II cho điều phối đám mây tiết kiệm năng lượng

Một hướng nghiên cứu khác dùng *Non-dominated Sorting Genetic Algorithm II* (NSGA-II) [22] để giải bài toán đa mục tiêu năng lượng + QoS. Ưu điểm của hướng này là thuật toán tiến hoá có thể sinh ra cả Pareto front trong một lần chạy mà không cần quét trọng số. Tuy nhiên, mỗi lần ra quyết định lại đòi hỏi đánh giá hàng nghìn cá thể, nên không phù hợp với *online scheduling* thời gian thực. Đây cũng chính là khoảng trống mà RL có lợi thế: chính sách đã huấn luyện chỉ cần một bước forward pass duy nhất khi suy luận.

2.4.4 Beware of Fragmentation (Weng và cộng sự, 2023)

Công trình [4] công bố Alibaba PAI Cluster Trace v2023 và đề xuất *Fragmentation Gradient Descent* — một heuristic giúp giảm phân mảnh GPU tới 10%. **Liên hệ trực tiếp với ELDAS:** đây chính là nguồn dữ liệu workload mà ELDAS sử dụng, đồng thời bài báo cũng cung cấp một baseline heuristic đủ mạnh để so sánh ở Phase 2.

2.4.5 MO-Gymnasium (Alegre và cộng sự, 2022)

MO-Gymnasium [23] là framework chuẩn cho các benchmark MORL, mở rộng API `step()` của Gymnasium để trả về vector phần thưởng. Thư viện cung cấp một số môi trường tham chiếu (*deep-sea-treasure*, *minecart*, *four-room*) cùng các chỉ số đánh giá Pareto front (hypervolume, sparsity). ELDAS đăng ký một môi trường custom theo đúng chuẩn này, từ đó có thể tương thích với mọi thuật toán MORL trong các nghiên cứu sau.

Bảng 2.2: Bảng tổng hợp so sánh các nghiên cứu liên quan.

Nghiên cứu	Năm	Thuật toán	Đa mục tiêu?	Năng lượng?	Simulator	GPU?
DeepRM	2016	DQN+CNN	Không	Không	Custom	Không
Decima	2019	GNN+RL	Không	Không	Spark	Không
NSGA-II/EE	2020	GA	Có	Có	CloudSim	Không
FGD/Alibaba	2023	Heuristic	Có	Không	Trace	Có
MO-Gym	2022	Framework	Có	—	Gym	—
ELDAS	2026	MORL+PPO	Có	Có	CloudSim Plus	Có

Định vị đóng góp. Bảng 2.2 cho thấy ELDAS lấp vào khoảng giao của bốn yêu cầu: *đa mục tiêu, tích hợp năng lượng, theo dõi GPU và sử dụng workload trace thực*, tất cả trong cùng một nền tảng MORL. Theo tìm hiểu của tác giả tại thời điểm viết báo cáo (2026), chưa có công trình mã nguồn mở nào kết hợp đồng thời cả bốn đặc tính này.

2.5 Công nghệ nền tảng

2.5.1 CloudSim Plus

CloudSim Plus [24] là framework mô phỏng rời rạc sự kiện (*discrete-event simulation*) cho điện toán đám mây, viết bằng Java 21. So với CloudSim gốc (Đại học Melbourne, 2009), CloudSim Plus được tái cấu trúc lại theo các nguyên tắc kỹ thuật phần mềm hiện đại: API dạng *fluent*, tách bạch rõ ràng *Host / Vm / Cloudlet*, và tích hợp sẵn *PowerModel*.

Lý do chọn: (i) mã nguồn mở, cộng đồng còn duy trì tích cực; (ii) hoạt động *event-driven*, cho phép tạm dừng và tiếp tục mô phỏng — đây là yêu cầu bắt buộc của vòng lặp RL (xem §4.5); (iii) đã có sẵn mô hình năng lượng cho CPU. **Hạn chế:** chưa hỗ trợ GPU, do đó ELDAS bổ sung thêm lớp tracking riêng.

Bản chất mô phỏng host: POJO + DES, không Thread/Process. Khác với một số công cụ mô phỏng đời đầu (SimJava và các nguyên mẫu CloudSim thuở sơ khai) vốn ánh xạ mỗi thực thể mô phỏng (Datacenter, Broker, Host, VM) thành một `java.lang.Thread` riêng biệt, CloudSim Plus **tái thiết kế toàn bộ theo mô hình hướng đối tượng điều khiển bằng sự kiện** (*event-driven object model*). Mọi *Host*, *Vm*, *Cloudlet* thực chất chỉ là các *Plain Old Java Object* (POJO) cư trú trên Heap của JVM, *không* phải là Thread hay tiến trình hệ điều hành. Toàn bộ vòng lặp mô phỏng chạy trên **một luồng thực thi duy nhất**, liên tục rút đối tượng *SimEvent* từ một *Priority Queue* có tên *FutureQueue* (sắp xếp theo timestamp), chuyển sang *DeferredQueue* khi đã đến hạn, rồi gọi trực tiếp phương thức `processEvent()` của đối tượng đích. Đồng hồ mô phỏng (*simulation clock*) không trôi theo thời gian thực mà *nhảy vọt* từ sự kiện này sang sự kiện kế tiếp, nhờ vậy có thể “tua nhanh” qua các khoảng trống không có sự kiện. Quyết định kiến trúc này quan trọng vì ba lý do: (i) triệt tiêu chi phí *context switch* và *lock contention* của hệ điều hành khi quy mô lên tới hàng vạn host; (ii) đảm bảo **tính xác định** (determinism) — cùng một random seed luôn cho ra kết quả giống hệt nhau, điều kiện tiên quyết cho nghiên cứu khoa học; (iii) tiết kiệm bộ nhớ vì mỗi Thread JVM tốn ~ 512 KB stack, trong khi một POJO chỉ tốn vài chục byte.

Ưu điểm. (i) *Reproducibility tuyệt đối* nhờ single-threaded loop và POJO; (ii) API *fluent* và mô hình object-chaining (`cloudlet.getVm().getHost()`) giảm boilerplate; (iii) hỗ trợ sẵn `PowerModel`, các bộ lập lịch CPU (`TimeShared`, `SpaceShared`, mô phỏng CFS của Linux), auto-scaling theo *Event Listener*, và đọc trace từ Google Cluster / Alibaba dưới dạng stream; (iv) có cơ chế `startSync()` cho phép *tạm dừng* mô phỏng tại mỗi khoảng `runFor()` — đúng cơ chế ELDAS dùng để chen vòng lặp RL qua Py4J (§4.5); (v) thực nghiệm của nhóm phát triển trên kịch bản 20 000 host, 40 000 VM, 50 000 Cloudlet cho thấy CloudSim Plus chạy nhanh hơn CloudSim 4.0.0 từ 20% tới hơn 90% tùy thuật toán cấp phát [25].

Hạn chế và điểm nghẽn ở quy mô lớn. Ngược lại, kiến trúc này cũng kéo theo bốn ràng buộc cần lưu ý khi xác định phạm vi đồ án:

1. **Lỗi đơn luồng không tận dụng được nhiều core.** Cho phép chạy *song song nhiều kịch bản độc lập* (mỗi kịch bản một JVM hoặc một luồng lỗi riêng), nhưng *một* Datacenter không thể phân tán event-loop ra nhiều CPU. Trần thông lượng bị chặn bởi single-thread performance của CPU.
2. **Độ phức tạp $O(V \cdot H)$ của allocation policy.** Các chính sách scoring đầy đủ (BestFit, WorstFit, K8s LRP, meta-heuristic) phải duyệt qua mọi host khả thi cho mỗi VM cần đặt; khi V, H cùng đạt 10^4 , khâu cấp phát có thể tốn hàng giờ CPU.
3. **Áp lực *recursive update*.** Mỗi nhịp `schedulingInterval`, Datacenter gọi đệ quy `updateVmsProcessing()` qua toàn bộ $\text{Host} \rightarrow \text{VM} \rightarrow \text{Cloudlet}$. Thí nghiệm trên Google Cluster Trace cho thấy với 10 000 host, mô phỏng *hai ngày* dữ liệu thực có thể tốn tới *bảy ngày* wall-time; điểm nghẽn lớn nhất là vòng lặp cập nhật chứ không phải thuật toán cấp phát.
4. **Áp lực JVM Garbage Collector.** Một episode dài với hàng triệu `SimEvent` và `HostStateHistoryEntry` làm Heap đầy nhanh; các đợt *Stop-The-World GC* của G1GC/Shenandoah có thể đóng băng tiến trình hàng giây. Trần thực tế của một JVM 16 GB RAM nằm vào khoảng 10^4 – 10^5 host trên cùng một mô phỏng.
5. **Không có GUI.** Cấu hình hoàn toàn bằng mã Java hoặc YAML; rào cản đối với người dùng không lập trình hệ thống.

Định vị quy mô datacenter của ELDAS. Việc thấu hiểu các điểm nghẽn nói trên trực tiếp ấn định *khoảng quy mô khả thi* của một thí nghiệm CloudSim Plus, và qua đó định vị ELDAS trên thang đo công nghiệp. Bảng 2.3 so sánh ba mức quy mô tham chiếu.

Bảng 2.3: So sánh quy mô datacenter ELDAS với các mức công nghiệp.

Tiêu chí	ELDAS (Phase 1)	Normal-scale	Hyperscale
Số host vật lý	10 (mở rộng được 50)	10^2 – 10^3	10^5 – 10^6
Loại đại diện	Lab / micro-cluster	Cluster học thuật, edge DC	Google, AWS, Azure region
#vCPU/host	16 (cấu hình được)	32–128	96–192
GPU/host	0–4 (mô phỏng)	0–8	8+ (cụm AI dedicated)
#task/episode	$\sim 2 \cdot 10^3$ (Alibaba)	10^4 – 10^5	10^7 + trên ngày
PUE điển hình	1,5 (giả định)	1,5–1,8	1,10–1,15 [26]
Khả năng tái lập	Tuyệt đối (1 seed)	Khó (multi-tenant)	Không thể

ELDAS rõ ràng thuộc nhóm **lab-scale** / **micro-cluster**: 10 host, vài nghìn task/episode trên trace Alibaba đã được lọc về một window đại diện. Mức quy mô này có hai hệ quả cần khẳng định trong phần giới hạn:

- **Đủ để chứng minh thuật toán.** Với 10 host và 6 scheduler, không gian hành động (10 host) và không gian trạng thái ($6H + 4 = 64$ chiều) đủ rộng để tạo ra *Pareto front có thể quan sát*: chênh lệch năng lượng BestFit↔RoundRobin đạt ~ 20 – 26% (xem Chương 5), và PPO-min đã có vị trí interior Pareto thực sự — đủ chứng minh giả thuyết MORL khả thi trên CloudSim Plus.
- **Không đại diện cho hyperscale.** Tại quy mô 10^5 + host, các yếu tố trở nên chi phối mà ELDAS không mô phỏng: phân tầng mạng đa lớp, fault domain, multi-zone replication, năng lượng làm mát phụ thuộc nhiệt độ ngoài trời, và đặc biệt là chi phí lập lịch đệ quy phình theo $O(V \cdot H)$ đã nêu. Một thí nghiệm CloudSim Plus 10 000+ host sẽ vướng cả ba điểm nghẽn ở mục trên và đòi hỏi hạ tầng JVM riêng (≥ 32 GB Heap, GC tinh chỉnh). Đây là giới hạn được nêu rõ trong Chương 7.

2.5.2 Py4J – cầu nối Java–Python

Py4J [27] là thư viện cho phép code Python gọi trực tiếp các đối tượng Java trong cùng JVM thông qua *GatewayServer*. Cơ chế:

- Phía Java khởi tạo *GatewayServer* lắng nghe trên TCP port 25333;
- Phía Python tạo *JavaGateway()* để truy cập đối tượng *entry_point*;
- Mỗi lời gọi method được serialize qua TCP socket, kết quả trả về đồng bộ.

Bảng 2.4: So sánh các tùy chọn cầu nối Java–Python cho ELDAS.

Tiêu chí	Py4J	gRPC	REST
Độ trễ một call	$\sim 0,1$ ms	~ 1 ms	~ 5 ms
Cần định nghĩa schema?	Không	Có (<i>.proto</i>)	Có (OpenAPI)
Truyền đối tượng phức tạp	Trực tiếp	Phải serialize	JSON
Phù hợp tần suất cao?	Tốt	Khá	Kém

Do vòng lặp RL có thể gọi *step()* tới hàng nghìn lần mỗi episode, Py4J là phương án cân bằng tốt nhất giữa hiệu năng và độ phức tạp triển khai.

2.5.3 MO-Gymnasium

MO-Gymnasium là một *fork* của Gymnasium do Alegre và cộng sự duy trì [23]. Khác biệt quan trọng nhất nằm ở API `step(action)`: hàm này trả về `(obs, reward_vector, terminated, truncated, info)`, với `reward_vector` $\in \mathbb{R}^d$ thay vì một số thực. ELDAS đăng ký môi trường ELDAS-v0 kế thừa từ lớp `MOEnv`, qua đó có thể *plug-and-play* với mọi thuật toán MORL tuân theo chuẩn này.

2.5.4 Docker Compose

Toàn bộ hệ thống được đóng gói bằng Docker Compose v2, gồm hai service (`cloudsim-java`, `rl-agent`) cùng ba volume chia sẻ (`trace-data`, `metrics`, `checkpoints`). Mục tiêu là khả năng **tái lập kết quả** (*reproducibility*): bất kỳ ai có Docker đều có thể chạy lại thực nghiệm bằng một câu lệnh duy nhất:

```
1 docker compose up --build
```

Listing 2.1: Khởi chạy hệ thống.

2.5.5 So sánh với KWOK

KWOK (*Kubernetes WithOut Kubelet*) là một lựa chọn thay thế khá phổ biến để mô phỏng cluster K8s ở quy mô lớn. Bảng 2.5 tóm lược lý do ELDAS chọn CloudSim Plus thay vì KWOK ở Phase 1.

Bảng 2.5: So sánh KWOK và CloudSim Plus theo các tiêu chí của ELDAS.

Tiêu chí	KWOK	CloudSim Plus
Mô hình thời gian	Real-time	Discrete-event
Mô hình năng lượng	Không có	Có sẵn (PowerModel)
Pause/Resume mô phỏng	Khó	Hỗ trợ nguyên thủy
Quy mô tối đa	Hàng nghìn node	Hàng trăm host
Phù hợp huấn luyện RL?	Khó (không pause)	Tốt
Hỗ trợ GPU	Plugin	Phải bổ sung

Lợi thế quyết định của CloudSim Plus nằm ở cơ chế *discrete-event*: mô phỏng có thể “tua nhanh” qua các khoảng không có sự kiện, nhờ vậy một episode RL chỉ mất vài giây thay vì vài giờ thời gian thực.

2.6 Kết luận chương

Chương này đã đặt nền tảng lý thuyết cho ba quyết định thiết kế cốt lõi của ELDAS: (i) chọn **mô hình năng lượng tuyến tính** vì cân bằng được độ chính xác và khả năng tái lập; (ii) chọn **PPO kết hợp linear scalarization** làm thuật toán nền cho Phase 2 nhờ tính ổn định và dễ song song hoá; (iii) chọn ngăn xếp công nghệ **CloudSim Plus + Py4J + MO-Gymnasium** vì khả năng pause/resume mô phỏng và mức độ tương thích với chuẩn MORL. Bảng tổng hợp ở §2.4 cũng đã định rõ vị trí và đóng góp của ELDAS trong tương quan với các công trình liên quan. Chương kế tiếp sẽ trình bày kiến trúc bốn lớp hiện thực hoá các quyết định nói trên.

Chương 3

Thiết kế kiến trúc hệ thống

Chương này trình bày cách ELDAS được thiết kế để giải bài toán đã đặt ra ở Chương 1. Hệ thống được chia thành **bốn lớp**; mỗi lớp đảm nhận một vai trò riêng và giao tiếp với nhau qua giao diện được định nghĩa rõ ràng. Phần mở đầu giới thiệu tổng quan kiến trúc cùng lý do lựa chọn từng công nghệ. Các phần kế tiếp lần lượt mô tả chi tiết bốn lớp: mô phỏng (CloudSim Plus), cầu nối ngôn ngữ (Py4J), tác tử học (MO-Gymnasium) và lớp đóng gói hạ tầng (Docker Compose).

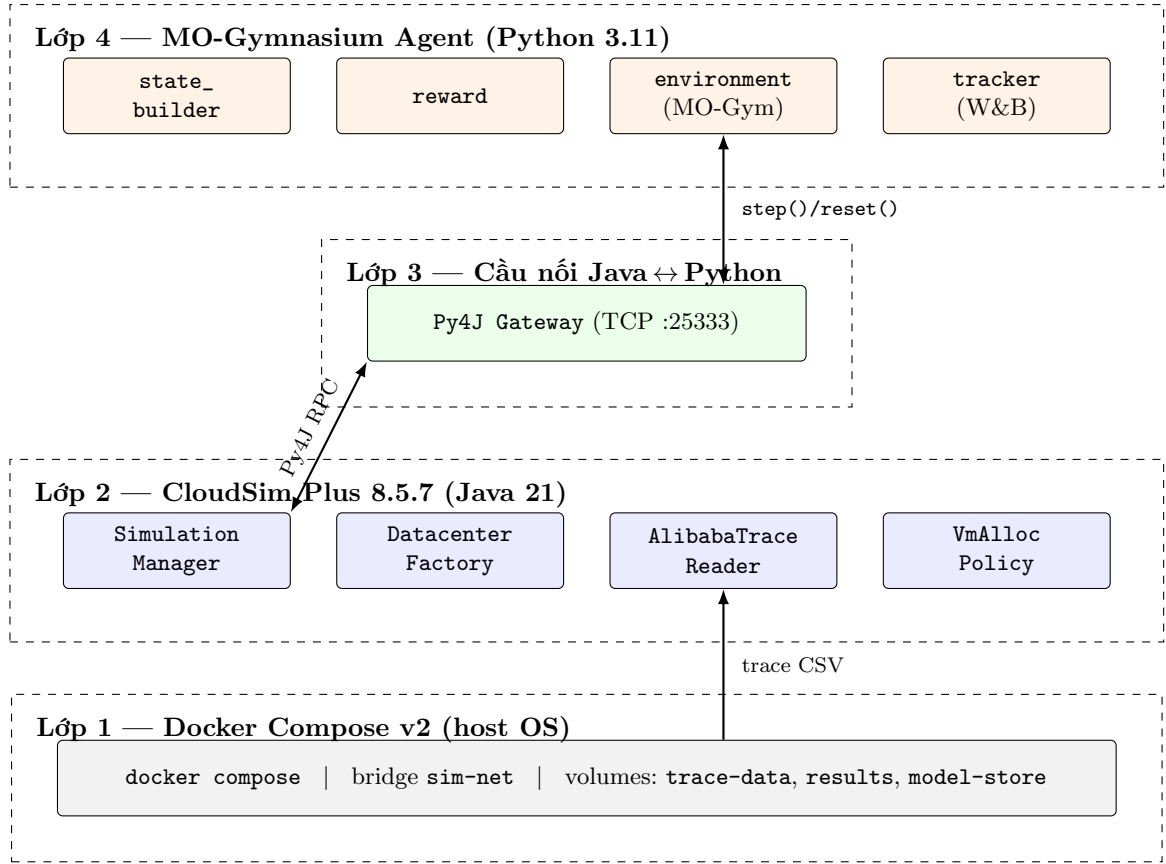
3.1 Tổng quan kiến trúc bốn lớp

3.1.1 Vì sao chia thành bốn lớp?

ELDAS phải kết hợp ba mảng rất khác nhau: **mô phỏng cluster** (CPU, RAM, GPU, năng lượng), **thuật toán học** (PPO, mạng nơ-ron) và **đóng gói triển khai** (Docker, volume, biến môi trường). Nếu dồn tất cả vào một chương trình duy nhất, ngay lập tức sẽ vấp phải ba khó khăn:

- Engine mô phỏng năng lượng datacenter tốt nhất hiện nay (CloudSim Plus) chỉ có bản Java, trong khi các thư viện RL hiện đại (PyTorch, Stable-Baselines3, MO-Gymnasium) đều dùng Python — hai ngôn ngữ không thể chạy trong cùng một process.
- Mỗi lần đổi siêu tham số huấn luyện không nên kéo theo việc khởi động lại cả engine mô phỏng, vì khâu này rất tốn thời gian.
- Người khác cần dựng lại được toàn bộ hệ thống chỉ bằng một câu lệnh, thay vì phải cài thủ công từng phần.

Lời giải là chia hệ thống thành bốn lớp (Hình 3.1). Java và Python chạy trong hai container Docker riêng, trao đổi với nhau qua một cầu nối trong cùng mạng nội bộ.



Hình 3.1: Kiến trúc bốn lớp của ELDAS. Mỗi đường đứt nét là một biên cô lập về ngôn ngữ, container, hoặc hạ tầng. Mũi tên đậm đánh dấu kênh giao tiếp hai chiều của vòng lặp RL.

3.1.2 Vai trò từng lớp

Lớp 1 — Docker Compose. Đảm bảo toàn bộ hệ thống chạy đồng nhất trên mọi máy. Hai container (`cloudsim-java` và `rl-agent`) dùng chung một mạng nội bộ cùng ba volume chứa dữ liệu trace, kết quả và checkpoint.

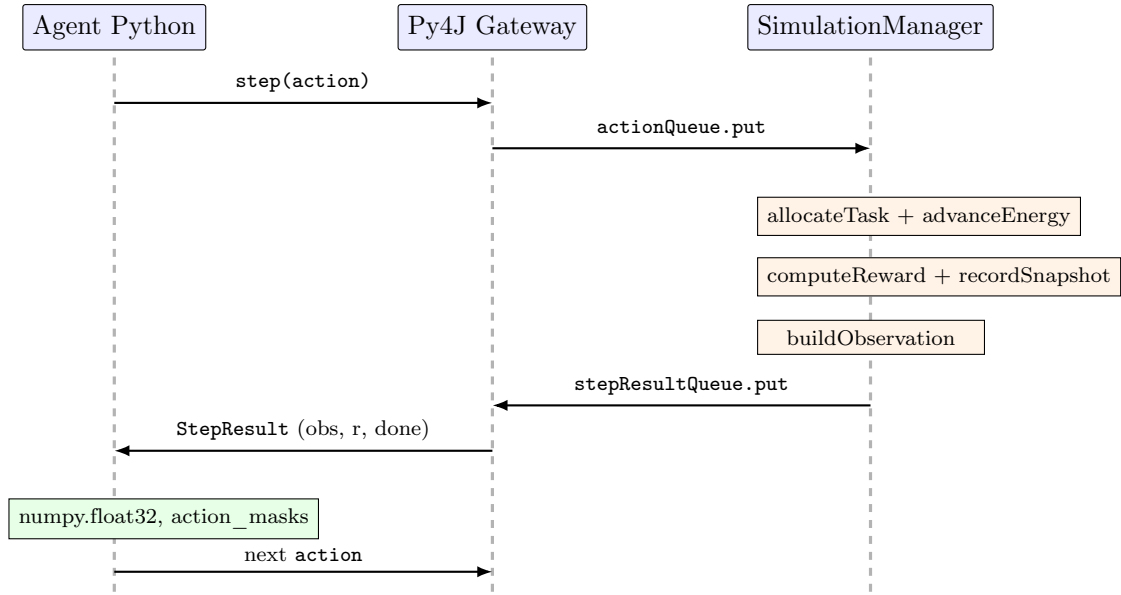
Lớp 2 — CloudSim Plus. Đóng vai trò “môi trường” theo nghĩa của RL: chứa cluster ảo (host, CPU, RAM, GPU), đọc trace Alibaba, nhận hành động từ agent và trả về trạng thái mới kèm phần thưởng. Toàn bộ mô hình năng lượng và GPU đều nằm tại lớp này.

Lớp 3 — Py4J Gateway. Đảm nhận vai trò cầu nối giữa Java và Python. Lớp này chỉ phơi ra một vài phương thức cơ bản (`reset`, `step`, `getActionMask`, `exportMetrics`). Phía Python không cần biết bất kỳ chi tiết nào về CloudSim, nên sau này hoàn toàn có thể thay engine bằng KWOK hoặc một cluster Kubernetes thật mà không phải đụng đến code Python.

Lớp 4 — MO-Gymnasium Agent. Đóng vai trò “tác tử” RL: nhận quan sát từ Java, dùng MaskablePPO để chọn hành động và ghi log lên Weights & Biases. Toàn bộ hệ sinh thái Python (PyTorch, Stable-Baselines3, MO-Gymnasium) được giữ trọn trong lớp này.

3.1.3 Một bước trong vòng lặp RL diễn ra như thế nào?

Hình 3.2 minh họa quá trình của một lần `step()`: Python gửi hành động sang Java, Java cập nhật mô phỏng rồi trả về quan sát mới kèm phần thưởng. Toàn bộ một vòng *round-trip* chỉ tốn khoảng 0,1–0,3ms khi hai container cùng chạy trên một máy.



Hình 3.2: Sequence diagram cho một vòng lặp RL. Hai **SynchronousQueue** ở phía Java đảm bảo simulation thread chỉ tiến bước khi nhận được hành động hợp lệ.

3.2 Lựa chọn giải pháp và biện luận

Bốn quyết định thiết kế lớn nhất của ELDAS đều được trình bày theo cùng một khuôn: nêu các phương án, so sánh ưu nhược điểm trên các tiêu chí thực tế, rồi chốt lại phương án kèm phần đánh đổi cần chấp nhận. Bảng 3.1 ở cuối phần này tổng hợp lại cả bốn quyết định.

3.2.1 Lựa chọn công cụ mô phỏng: KWOK vs. CloudSim Plus

Hai ứng viên cho việc mô phỏng cluster có khả năng tạm dừng phục vụ RL là CloudSim Plus và KWOK (*Kubernetes WithOut Kubelet*). Có ba điểm quyết định lựa chọn:

1. **Tốc độ.** CloudSim Plus dùng cơ chế *discrete-event simulation*, có thể tua nhanh qua các khoảng không có sự kiện; một episode 5 000 tác vụ chạy xong trong vài giây. KWOK chạy ở thời gian thực, cùng episode đó tốn tới nhiều phút — chậm hơn hàng trăm lần và không phù hợp cho việc huấn luyện hàng triệu bước.
2. **Mô hình năng lượng.** CloudSim Plus có sẵn mô hình công suất CPU tuyến tính theo SPECpower; trong khi KWOK không có khái niệm năng lượng và phải tự viết plugin để bổ sung.
3. **Tạm dừng / tiếp tục.** Vòng lặp RL cần có khả năng dừng lại tại mỗi sự kiện “tác vụ mới đến” để chờ agent ra quyết định. CloudSim Plus hỗ trợ pause/resume sẵn có; KWOK thì không.

Quyết định: chọn **CloudSim Plus 8.5.7**. **Đánh đổi cần chấp nhận:** mô hình mạng còn đơn giản và không có `kubectl` thật, nên về sau, nếu muốn kiểm chứng trên cluster thực, vẫn cần thêm bước port chính sách. Tuy vậy, do Lớp 3 đã được giữ rất gọn, việc chuyển đổi chỉ cần thay phần backend Java; phần code Python không phải sửa.

3.2.2 Lựa chọn cầu nối ngôn ngữ: Py4J vs. gRPC vs. REST

Bảng 2.4 ở §2.5 đã so sánh chi tiết. Ở đây nhắc lại ba lý do chính khiến Py4J phù hợp nhất với ELDAS:

- **Gọn nhẹ.** Mỗi bước RL chỉ trao đổi vài chục số thực — nhỏ hơn nhiều so với phần overhead header và bước parse JSON mà REST hay gRPC kéo theo.
- **Tần suất cao.** Phase 2 sẽ gọi `step()` hàng triệu lần. Với REST (khoảng 5 ms mỗi lần), riêng phần giao tiếp đã tiêu tốn vài giờ; trong khi Py4J (khoảng 0,1 ms) chỉ cần khoảng vài chục phút.
- **Truy cập trực tiếp đối tượng Java.** Py4J cho phép Python gọi thẳng phương thức của đối tượng Java, không cần định nghĩa `.proto` hay OpenAPI schema — giảm đáng kể lượng boilerplate.

Quyết định: chọn **Py4J 0.10.9.7**. **Đánh đổi cần chấp nhận:** hai container phải nằm trong cùng mạng Docker, không thể tách ra hai máy khác nhau. ELDAS chấp nhận giới hạn này vì phạm vi đề án chỉ chạy trong phòng thí nghiệm.

3.2.3 Lựa chọn thuật toán RL: PPO vs. DQN vs. A2C

Mục 2.3 đã giới thiệu PPO. Ở đây, ba ứng viên được so sánh trên ba tính chất bắt buộc của bài toán:

1. **Hỗ trợ action masking.** Ở mỗi bước, không phải host nào cũng còn đủ tài nguyên cho task hiện tại, nên agent cần bị cấm cứng khỏi các host không khả thi. PPO có biến thể `MaskablePPO` trong `sb3-contrib` hỗ trợ sẵn; DQN phải tự sửa cả bước chọn action lẫn bước Bellman; còn A2C chưa có triển khai mask sẵn dùng.
2. **Ổn định khi reward thưa và lệch biên độ.** Reward năng lượng luôn mang giá trị âm cỡ vài trăm, còn reward SLA gần như bằng 0 ở hầu hết các bước. Trước phổ reward kiểu này, DQN dễ bị overestimate giá trị Q ; trong khi PPO có hai cơ chế bù trừ là chuẩn hoá advantage và cắt tỉ số chính sách.
3. **Dễ song song hoá.** Phase 2 sẽ huấn luyện nhiều agent cùng lúc, mỗi agent ứng với một trọng số khác nhau. PPO on-policy chạy nhiều môi trường song song một cách tự nhiên; trong khi DQN với replay buffer đòi hỏi kiến trúc phức tạp hơn nhiều.

Quyết định: **PPO + MLP**, dùng `MaskablePPO` từ `sb3-contrib`. **Hướng mở rộng:** có thể thay MLP bằng GNN ở giai đoạn sau — chỉ cần đổi backbone, không phải động đến bốn lớp kiến trúc.

3.2.4 Lựa chọn chiến lược MORL: Linear Scalarization vs. NSGA-II

Linear scalarization chuyển bài toán đa mục tiêu thành nhiều bài toán đơn mục tiêu: với mỗi trọng số w , agent học một chính sách tối ưu tại điểm cân bằng tương ứng; quét nhiều trọng số rồi nối các điểm thu được sẽ thành Pareto front. **NSGA-II** [22] ngược lại là thuật toán tiến hoá, sinh ra cả Pareto front trong một lần chạy. Tuy vậy, xét trong bối cảnh của ELDAS:

- NSGA-II phải đánh giá hàng nghìn cá thể ở mỗi thế hệ, và mỗi cá thể tương ứng với một episode mô phỏng hoàn chỉnh. Tổng chi phí vượt xa ngân sách thời gian của Phase 2.
- Linear scalarization chỉ cần huấn luyện 9 chính sách độc lập (ứng với 9 giá trị w) và dễ chạy song song trên nhiều CPU. Khi triển khai thực tế, mỗi quyết định chỉ tốn một lượt forward pass của mạng — đủ nhanh cho online scheduling.
- Đúng là linear scalarization không khôi phục được những đoạn lõm của Pareto front, nhưng hạn chế này đã được nêu rõ ở §2.3 và có thể chấp nhận trong phạm vi đề án.

Quyết định: dùng **linear scalarization** với weight sweep $w \in \{0,1; 0,2; \dots; 0,9\}$. NSGA-II và các thuật toán MORL chuyên dụng (Envelope-Q, Pareto-DQN) được giữ ở phần hướng phát triển tương lai.

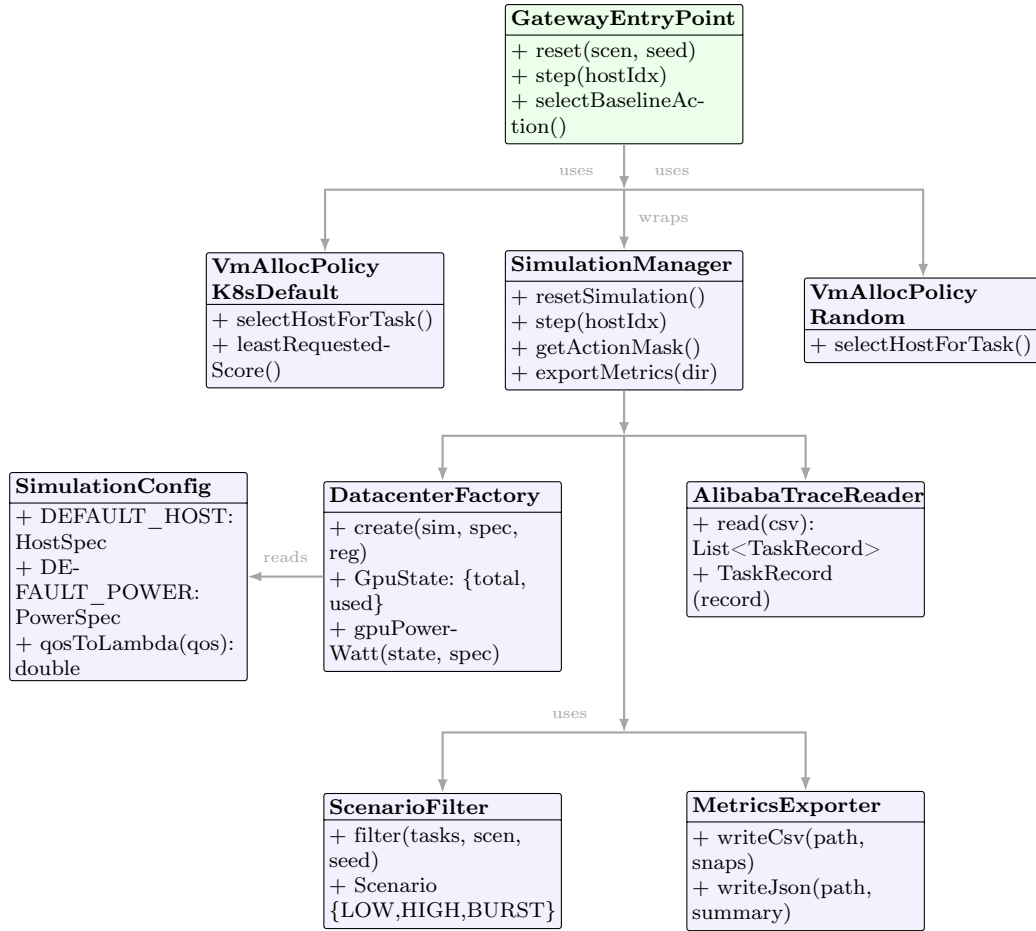
3.2.5 Bảng tổng hợp ma trận quyết định

Bảng 3.1: Ma trận quyết định bốn lựa chọn kiến trúc của ELDAS. Ký hiệu: ✓ = đáp ứng tốt, ~ = chấp nhận được, × = không đáp ứng.

Quyết định	Phương án	Latency	Năng lượng	Pause/Resume	Đa mục tiêu	Reproducible
Engine	CloudSim Plus	✓	✓	✓	✓	✓
	KWOK	~	×	×	✓	✓
Bridge	Py4J	✓	—	—	—	✓
	gRPC	~	—	—	—	✓
	REST	×	—	—	—	✓
RL Algo	PPO + MaskablePPO	—	—	—	✓	✓
	DQN	—	—	—	~	✓
	A2C	—	—	—	×	✓
MORL	Linear scalar.	—	—	—	~	✓
	NSGA-II	×	—	—	✓	✓

3.3 Thiết kế Lớp 2 — CloudSim Plus

Lớp 2 là phần lớn nhất của hệ thống, gồm khoảng 1 800 dòng Java chia thành chín lớp (Hình 3.3). Mỗi lớp đảm nhận đúng một công việc cụ thể, qua đó việc bảo trì và mở rộng đều dễ dàng hơn.



Hình 3.3: Class diagram CloudSim Plus. Mũi tên đậm biểu diễn quan hệ *depends-on*; GatewayEntryPoint là điểm vào duy nhất của Python.

3.3.1 SimulationConfig — cấu hình tập trung

Mọi tham số của cluster được gom vào một file Java duy nhất, giúp khâu tinh chỉnh và kiểm tra trở nên đơn giản. Cấu hình mặc định mô phỏng một cụm gồm 10 host; mỗi host có 64 vCPU, 256 GB RAM và 8 GPU loại A100/V100. Công suất CPU lấy ở mức 400 W khi đầy tải và 120 W khi rảnh; GPU tương ứng là 300 W và 30 W (số liệu lấy từ tài liệu kỹ thuật của nhà sản xuất). Bảng ánh xạ QoS → hệ số λ dùng trong reward SLA cũng được khai báo ở đây: bốn lớp QoS của Alibaba (LS, Guaranteed, Burstable, BE) được ánh xạ lần lượt sang các hệ số phạt 3,0 — 2,0 — 1,0 — 0,5.

3.3.2 DatacenterFactory và lớp theo dõi GPU

CloudSim Plus 8.5.7 chưa có khái niệm GPU. Để không phải sửa thư viện gốc — điều sẽ phá vỡ khả năng tái lập kết quả — ELDAS giữ một bảng riêng, ánh xạ mỗi host sang trạng thái GPU của nó (tổng số GPU và số GPU đang sử dụng). Công suất GPU của host được tính theo cùng dạng tuyến tính như CPU: số GPU đang dùng càng nhiều thì công suất càng tiến gần mức tối đa, còn các GPU rảnh đóng góp ở mức công suất idle. Phép tính này được gọi ở mỗi bước mô phỏng và được tính gộp vào tổng năng lượng của cluster.

3.3.3 AlibabaTraceReader — đọc trace

File trace `openb_pod_list_default.csv` có 11 trường, được ánh xạ sang một record Java đơn giản (Bảng 3.2). Hai trường dẫn xuất được sinh ngay trong khi parse: **deadline** bằng thời điểm K8s bind cộng thêm 110% thời gian chạy thực tế (tức cho phép trễ tối đa 10% trước khi tính là vi phạm); **slaLambda** là hệ số phạt theo QoS, lấy từ bảng đã khai báo ở §3.3.1. Những bản ghi có `podPhase = Pending` hoặc thiếu trường bắt buộc sẽ bị bỏ qua kèm cảnh báo, không làm dừng toàn bộ quá trình import.

Bảng 3.2: Ánh xạ trường Alibaba trace → đối tượng `TaskRecord`.

Trường CSV	Trường TaskRecord	Ngữ nghĩa
<code>name</code>	<code>name</code>	Mã định danh tác vụ
<code>cpu_milli</code>	<code>pesNeeded()</code>	$\lceil x/1000 \rceil$ PE
<code>memory_mib</code>	<code>memoryMib</code>	RAM yêu cầu (MiB)
<code>num_gpu</code>	<code>numGpu</code>	Số GPU nguyên
<code>gpu_milli</code>	<code>gpuMilli</code>	Tỉ phần GPU (1000 = full card)
<code>qos</code>	<code>slaLambda</code>	$\rightarrow \lambda \in \{0,5, 1, 2, 3\}$
<code>creation_time</code>	<code>creationTime</code>	Thời điểm xuất hiện (giây)
<code>deletion_time</code>	<code>deletionTime</code>	Thời điểm kết thúc thực tế
<code>scheduled_time</code> (dẫn xuất) <code>deadline</code>	<code>scheduledTime</code>	Thời điểm K8s bind ban đầu = <code>scheduledTime</code> + $1,1 \Delta_{\text{run}}$

3.3.4 ScenarioFilter — ba kịch bản tải

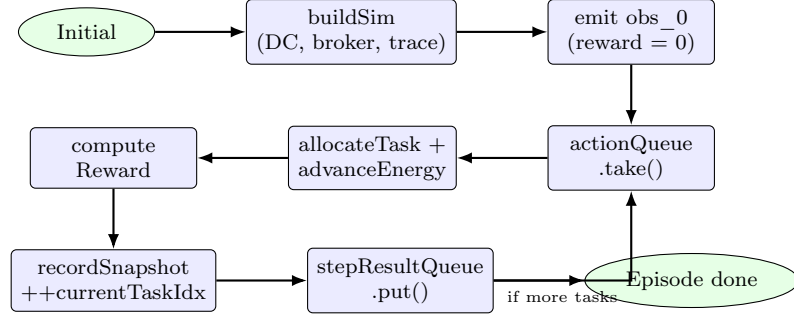
Để bộc lộ ba chế độ vận hành khác biệt, `ScenarioFilter` cắt trace đầy đủ thành ba tập con:

- **Low**: 25% tác vụ đầu tiên theo `creationTime` — dùng để kiểm tra hành vi của scheduler khi cluster ở mức tải thấp (đây cũng là vùng năng lượng kém hiệu quả nhất theo `SPECpower`).
- **HIGH**: toàn bộ tác vụ có `podPhase` $\neq$ `Pending` — kịch bản tham chiếu, đủ khả năng làm bão hoà cluster.
- **BURST**: chỉ giữ những tác vụ thuộc các cửa sổ 1 giờ có cường độ đến vượt percentile 80 — dùng để kiểm tra hành vi của cluster khi bị *shock* đột ngột.

Cả ba kịch bản đều dùng chung một `seed` cho `Random`, qua đó đảm bảo các lần chạy đều cho kết quả lặp lại được.

3.3.5 SimulationManager — ghép mô phỏng vào vòng lặp RL

Đây là lớp phức tạp nhất ở Lớp 2: lớp này có nhiệm vụ điều phối nhịp chạy của mô phỏng với nhịp ra quyết định của agent. Hình 3.4 mô tả luồng trạng thái trong một episode. Hai thread — một chạy mô phỏng phía Java, một nhận lệnh từ Python — trao đổi với nhau qua hai hàng đợi đồng bộ: một hàng để Python gửi hành động sang, một hàng để Java trả lại quan sát và phần thưởng. Hai hàng đợi được thiết kế sao cho mỗi lần `step()` là một giao dịch trọn vẹn — không cần khoá tường minh mà vẫn loại bỏ được hiện tượng đua giữa hai thread.



Hình 3.4: Vòng đời **SimulationManager** trong một episode. Hai vùng “actionQueue.take” và “stepResultQueue.put” là hai điểm đồng bộ giữa thread mô phỏng và thread Py4J.

Một lưu ý thiết kế quan trọng. Trong Phase 1, ELDAS không khởi động đồng hồ của **CloudSim**. Thay vào đó, **SimulationManager** tự theo dõi tài nguyên đã cấp (CPU và RAM) qua hai bảng băm riêng. Lý do: nếu bật đồng hồ mô phỏng khi chưa đăng ký đầy đủ các listener, JVM có thể crash âm thầm do sự kiện không có handler tương ứng. Hệ quả là Phase 1 chưa đo được thời gian hoàn thành theo đồng hồ mô phỏng — mọi deadline đều được suy ra từ trace — hạn chế này được nêu rõ ở §5.5 và sẽ được xử lý ngay đầu Phase 2.

Reward được tính ngay tại phía Java, theo đúng mô hình đã phát biểu ở Chương 1. Phần năng lượng được tính bằng công suất CPU và GPU hiện tại của host nhân với độ dài một bước scheduling (đạo dẫu để chuyển thành reward). Phần SLA được tính từ mức độ trễ ước lượng vượt deadline, nhân với hệ số λ theo QoS. Cả hai giá trị này được trả về dưới dạng mảng hai phần tử để giữ nguyên tính chất *vector reward* của MOMDP.

3.4 Thiết kế Lớp 3 — Py4J Gateway

3.4.1 Bề mặt API tối giản

Lớp Gateway cố ý chỉ phơi ra một bề mặt rất nhỏ: chỉ 9 phương thức công khai và không trả về trực tiếp bất kỳ đối tượng CloudSim nào. Bảng 3.3 liệt kê toàn bộ contract.

Bảng 3.3: API công khai của **GatewayEntryPoint**. Mọi tham số/giá trị trả về đều là kiểu nguyên thủy, mảng nguyên thủy, hoặc Java **record** đơn giản – thân thiện với Py4J và serializable cơ lập.

Phương thức	Trả về	Mô tả
<code>reset(scenario, seed)</code>	StepResult	Khởi tạo lại simulation, trả về quan sát ban đầu.
<code>step(hostIndex)</code>	StepResult	Áp hành động, trả về (s_{t+1}, r_t, d_t) .
<code>getActionMask()</code>	<code>boolean[]</code>	Mask hợp lệ cho bước hiện tại.
<code>getActionSize()</code>	<code>int</code>	$ \mathcal{A} $ = số host.
<code>getObservationSize()</code>	<code>int</code>	$6H + 4$.
<code>getTotalEnergyKwh()</code>	<code>double</code>	Năng lượng tích lũy (kWh) – dùng cho early-stop.
<code>selectBaselineAction(p)</code>	<code>int</code>	Chỉ số host do baseline (firstfit/bestfit/roundrobin/k8s/random) chọn.
<code>exportMetrics(dir)</code>	<code>void</code>	Ghi <code>metrics.csv</code> + <code>summary.json</code> .
<code>shutdown()</code>	<code>void</code>	Dừng simulation, không tắt GatewayServer .

Quy ước bất biến của hợp đồng: mọi state đều được giữ trong **SimulationManager** phía Java; phía Python *không cache* trạng thái. Mỗi **StepResult** là một **record** bất biến; sau khi

nhận, Python chỉ thực hiện đúng một lần chuyển đổi sang `numpy.ndarray` (kèm thao tác clip vào $[0, 1]$ để chống nhiễu số học) rồi đẩy thẳng cho agent. Cách làm này loại bỏ triệt để lớp bug đồng bộ trạng thái Java/Python.

3.4.2 Khởi động Gateway và kết nối client

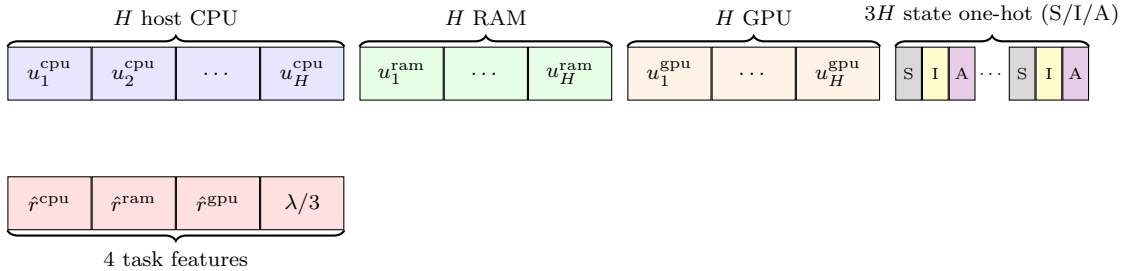
Phía Java, `GatewayServer` được bind vào địa chỉ `0.0.0.0` (không phải `localhost`) để container Python trong cùng mạng Docker có thể kết nối tới được. Cổng mặc định được đọc từ biến môi trường `PY4J_PORT` (giá trị `25333`).

Phía Python, môi trường mở kết nối tới gateway ngay khi khởi tạo và có cơ chế **retry với thời gian chờ tăng dần**, nhằm xử lý khoảng vài giây ngắt quãng giữa thời điểm Docker đánh dấu Java đã healthy và thời điểm cổng thực sự sẵn sàng. Sau 10 lần thử mà vẫn không kết nối được, hệ thống báo lỗi rõ ràng cho người dùng. Tham số `auto_convert=True` của Py4J giúp tự ép kiểu giữa Python và Java: `int` truyền thẳng, mảng `double[]` của Java về phía Python tự thành `list` — nhờ đó lượng boilerplate ở phía agent giảm đáng kể.

3.5 Thiết kế Lớp 4 — MO-Gymnasium Environment

3.5.1 Observation space

Quan sát là một vector phẳng dài $6H + 4$, bao gồm: H giá trị CPU util, H giá trị RAM util, H giá trị GPU util, $3H$ phần tử one-hot biểu diễn trạng thái host (`SUSPENDED`, `IDLE`, `ACTIVE`) và 4 task feature. Giá trị đã được Java chuẩn hoá về $[0, 1]$ và Python clip lại để chống lỗi số học (Hình 3.5). Phần one-hot trạng thái host là chỉ báo không thể thiếu: nếu bỏ đi, agent không thể phân biệt host `IDLE` (sẵn sàng nhận task miễn phí) với host `SUSPENDED` (phải trả thêm chi phí wake-up khi đặt task lên).



Hình 3.5: Cấu trúc rút gọn vector quan sát ($6H+4$) chiều: ba khối H chiều cho CPU/RAM/GPU utilisation, một khối $3H$ chiều cho host state one-hot (**S**`SUSPENDED`, **I**`IDLE`, **A**`ACTIVE` cho mỗi host), và bốn ô cuối mô tả tác vụ kế tiếp cần xếp lịch.

Việc *nhóm theo loại tài nguyên* (CPU, rồi RAM, rồi GPU, rồi state) thay vì *nhóm theo host* là một lựa chọn có chủ đích. Lớp ẩn của MLP học các tương tác *trong cùng một loại* dễ hơn (ví dụ: host nào có CPU thấp nhất, host nào đang `SUSPENDED`). Sau này, nếu chuyển sang GNN ở stretch goal, các khối CPU/RAM/GPU có thể **reshape** về ma trận $H \times 3$ một cách tự nhiên; còn khối state được đưa vào node feature dưới dạng vector one-hot 3 chiều.

3.5.2 Action space và action masking

Không gian hành động khá đơn giản: là chỉ số của host (từ 0 đến $H - 1$) mà agent muốn đặt tác vụ hiện tại lên. Ở mỗi bước, môi trường gọi sang Java để lấy về một mask boolean dài H : phần tử thứ j bằng `true` nếu host j còn đủ CPU, RAM và GPU cho tác vụ; ngược lại là `false`. MaskablePPO áp mask này lên đầu ra của policy, qua đó đảm bảo agent không bao giờ chọn host không đủ tài nguyên. Trong trường hợp đặc biệt khi không có host nào chứa nổi tác vụ (toàn bộ mask đều `false`), phía Python sẽ bật lại toàn bộ mask để tránh agent bị “khóa cứng”.

3.5.3 Vector reward và linear scalarization

Ở mỗi bước, môi trường trả về reward dưới dạng vector hai phần tử: phần năng lượng (luôn âm, bằng lượng năng lượng tiêu thụ thêm trong bước đó, đã đảo dấu) và phần SLA (âm nếu vi phạm deadline, 0 nếu đúng hạn). Để có thể chạy được với PPO chuẩn, ELDAS cung cấp một wrapper là `ScalarRewardWrapper`, có nhiệm vụ cộng hai thành phần này theo trọng số (w_e, w_s) với $w_e + w_s = 1$. Trọng số mặc định là (0,8; 0,2); Phase 2 sẽ quét $w_e \in \{0,1; \dots; 0,9\}$ để dựng Pareto front. Khi cần benchmark với một thuật toán MORL chuyên dụng, chỉ cần bỏ wrapper là môi trường trả thẳng vector reward về cho thuật toán — không phải đụng đến code Java.

3.5.4 Điều kiện kết thúc episode

`terminated=True` chỉ khi mọi tác vụ trong tập `tasks` đã được phân bổ (`currentTaskIdx == tasks.size()`). `truncated` luôn bằng `False` ở Phase 1, do bài toán có chân trời hữu hạn một cách tự nhiên (số tác vụ cố định sau khi `ScenarioFilter` cắt). Sang Phase 2, ELDAS có thể bật `truncated=True` khi số bước vượt `max_steps`, qua đó giới hạn chi phí cho các chính sách tồi.

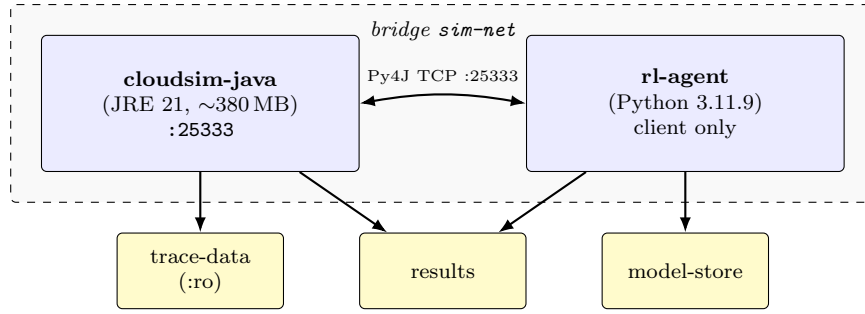
3.5.5 Tóm tắt vòng lặp

Một episode hoàn chỉnh theo đúng chuẩn Gymnasium gồm bốn bước lặp lại: (1) gọi `action_masks()` để biết các host khả thi; (2) policy chọn một host hợp lệ; (3) gọi `step(action)` để nhận quan sát và reward mới; (4) kiểm tra điều kiện kết thúc. Sau khi episode kết thúc, hàm `export_metrics()` được gọi để Java ghi `metrics.csv` và `summary.json` ra volume kết quả.

3.6 Thiết kế Lớp 1 — Docker Compose Infrastructure

3.6.1 Bố trí container và volume

Hai container được nối qua mạng nội bộ `sim-net` và chia sẻ ba volume (Hình 3.6). Volume chứa trace được mount ở chế độ **chỉ đọc** cho `cloudsim-java`, qua đó Java không có quyền ghi đè dữ liệu thực nghiệm và tránh được tình huống vô tình làm hỏng trace gốc trong khi debug.



Hình 3.6: Topology Docker Compose. **rl-agent** không expose port nào (client thuần); **cloudsim-java** chỉ expose \$PY4J_PORT.

3.6.2 Multi-stage build và tối ưu kích thước image

Cả hai Dockerfile đều dùng pattern **multi-stage**: một stage chứa các công cụ biên dịch nặng (Maven, gcc, ...), stage còn lại chỉ giữ phần runtime tối thiểu. Nhờ vậy, image **cloudsim-java** giảm từ khoảng 1,1 GB xuống chỉ còn ~ 380 MB; image **rl-agent** cũng giảm tương tự so với base **python:3.11**. Có một thủ thuật nhỏ nhưng đáng giá: copy các file khai báo dependency (**pom.xml**, **requirements.txt**) *trước* khi copy source code — khi các file này không đổi, Docker sẽ tái sử dụng layer cache và mỗi lần build lại tiết kiệm được 1–2 phút.

3.6.3 Healthcheck và thứ tự khởi động

Để tránh hiện tượng đua khi khởi động, **cloudsim-java** có một healthcheck kiểm tra cổng Py4J (25333) đã lắng nghe hay chưa; song song, **rl-agent** được khai báo phải chờ Java sẵn sàng mới được chạy. Docker Compose đảm nhiệm phần điều phối này. Cơ chế retry ở phía Python (đã mô tả ở §3.4) đóng vai trò lớp phòng vệ thứ hai, xử lý khoảng vài giây ngán giữa thời điểm cổng mở và thời điểm gateway hoàn tất bind.

3.6.4 Khả năng tái lập kết quả

ELDAS được thiết kế để bảo đảm **tái lập kết quả**: với cùng **RANDOM_SEED**, mỗi lần chạy đều cho cùng cách chia tác vụ giữa các kịch bản, cùng thứ tự lấy mẫu của các baseline ngẫu nhiên và cùng một chuỗi hành động. Mọi phụ thuộc đều được ghim phiên bản cứng: phía Java có Java 21 (eclipse-temurin), Maven 3.9.9, CloudSim Plus 8.5.7 và Py4J 0.10.9.7; phía Python có Python 3.11.9, mo-gymnasium 1.3.2 và stable-baselines3 2.7.0. Riêng **gymnasium** cố ý không pin, để pip tự giải xung đột giữa SB3 và MO-Gym.

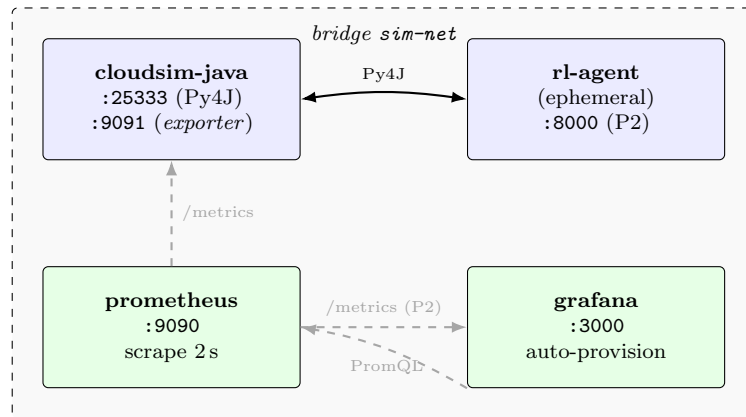
3.7 Thiết kế Lớp 5 — Observability stack (tùy chọn)

Bốn lớp ở trên đã đủ để chạy huấn luyện và xuất số liệu offline. Tuy vậy, khi cần *quan sát chiến lược đặt task của scheduler ngay trong lúc chạy* — chẳng hạn để minh họa tương phản giữa “K8s rải đều” và “MORL gom chặt” trong báo cáo hay buổi bảo vệ — thì các biểu đồ tính từ **metrics.csv** không đủ trực quan. ELDAS vì thế bổ sung thêm một **lớp Observability tùy chọn** (Phase 1.5) gồm Prometheus + Grafana, được thiết kế dưới bốn ràng buộc:

1. **Không can thiệp vào đường tới hạn (hot path).** Vòng lặp RL phải chạy bình thường ngay cả khi lớp này bị tắt hoặc gặp lỗi. Bởi vậy, Prometheus và Grafana *không* được đưa vào `depends_on` của bất kỳ service nào.
2. **Bật/tắt qua Docker Compose profile.** Mặc định, `docker compose up` chỉ khởi động hai container core; chỉ khi chạy `docker compose --profile monitoring up` thì stack quan sát mới được bật lên.
3. **Java là nguồn metric chính.** `cloudsim-java` là service *long-running* có healthcheck, nên Prometheus có thể scrape ổn định; ngược lại, `rl-agent` là *ephemeral* (chạy bằng `docker compose run --rm`) — nếu phát metric từ Python thì target sẽ chấp chớn. Mặt khác, phía Java giữ các giá trị *thực* (kWh, queue length), còn Python chỉ thấy observation đã được chuẩn hoá về $[0, 1]$.
4. **Không thay thế kết quả khoa học.** Grafana scrape theo *wall-time* chứ không phải *sim-time*, do đó trục thời gian của Grafana không tương thích với báo cáo định lượng. Vì lý do này, các figure trong Chương 5 vẫn lấy từ `metrics.csv` và WandB; Grafana chỉ phục vụ mục đích debug và demo trực tiếp.

3.7.1 Sơ đồ tích hợp

Hình 3.7 minh hoạ cách hai service mới gắn vào mạng `sim-net` mà vẫn không chạm vào đường dữ liệu Py4J giữa hai container core.



Hình 3.7: Topology lớp Observability. Mũi tên liền là dữ liệu trên đường tới hạn (Py4J); mũi tên đứt là metric scrape, chạy ngoài đường tới hạn. Hai service `prometheus` và `grafana` thuộc Docker Compose profile `monitoring` – mặc định không khởi động.

3.7.2 Mô hình dữ liệu metric

Java exporter đăng ký chín họ metric của ELDAS, chia thành ba nhóm theo ngữ nghĩa của Prometheus (Bảng 3.4). Tất cả đều được đánh kèm hai tag bối cảnh `scenario` và `scheduler`, nhờ vậy trên cùng một dashboard có thể so sánh trực tiếp *cùng một tình huống tải nhưng khác chiến lược lập lịch*.

Bảng 3.4: Mô hình dữ liệu metric của ELDAS Java exporter.

Họ metric	Kiểu	Ngữ nghĩa
eldas_host_{cpu,mem,gpu}_util	Gauge	Tỉ lệ sử dụng từng host $\in [0, 1]$. Label: <code>host_id</code> (0..9).
eldas_host_power_watt	Gauge	Công suất tức thời từng host (W).
eldas_total_energy_kwh	Gauge	Năng lượng tích lũy trong episode đang chạy. Reset về 0 mỗi <code>resetSimulation()</code> .
eldas_pending_tasks	Gauge	Số task còn lại trong episode – chỉ báo backpressure.
eldas_sim_clock_seconds	Gauge	Đồng hồ mô phỏng. Đối chiếu wall-clock để phát hiện stall.
eldas_sla_violations_total	Counter	Số vi phạm SLA từ lúc JVM khởi động. Không reset; truy vấn bằng <code>rate()</code> / <code>increase()</code> . Label phụ: <code>qos</code> $\in$ {BE, Burstable, Guaranteed, LS}.
eldas_tasks_scheduled_total	Counter	Số placement thành công, cộng dồn.

Vì sao episode không được dùng làm label? Đây là một chi tiết cần lưu ý khi thiết kế. Nếu để `episode_id` làm label cho gauge ở mức từng host, cardinality của metric sẽ phình ra theo số episode trong weight sweep ($\sim 10 \text{ host} \times 3 \text{ scenario} \times 4 \text{ scheduler} \times N \text{ episode}$, với N có thể từ 100 trở lên). Cách làm đúng, theo khuyến nghị của Prometheus, là giữ cho cardinality bị chặn (*bounded*): coi *Prometheus time* là chiều thời gian, còn label chỉ dùng để *định danh thực thể*. Mỗi lần `resetSimulation()` được gọi, gauge sẽ được đưa về 0, nhờ vậy các điểm gãy của gauge tự đánh dấu ranh giới giữa các episode ngay trên trục thời gian.

3.7.3 Bốn nguyên tắc an toàn

Đặc tính then chốt của lớp này là **không được gây hại**. Bốn nguyên tắc dưới đây được hiện thực ngay ở mức mã nguồn (chi tiết tại §4.4):

- *No-op khi tắt*. Cờ `enabled` mặc định bằng `false`; khi đó mọi setter chỉ thực hiện đúng một lệnh `return`, không cấp phát object và không I/O.
- *Mọi mutator đều được bọc trong try/catch Throwable*. Lỗi bên monitoring tuyệt đối không thể lan ra làm crash JVM sim.
- *Idempotent*. Hàm `MetricsRegistry.start()` có thể được gọi nhiều lần một cách an toàn; `stop()` làm sạch `CollectorRegistry` để có thể tái sử dụng cổng.
- *Counter chỉ monotonic*. Đúng theo semantic của Prometheus — không bao giờ *reset* counter trong cùng một vòng đời JVM; nếu cần reset, dùng gauge là cách thay thế hợp lệ.

3.8 Tổng kết chương

Chương này đã trình bày kiến trúc bốn lớp của ELDAS, kèm theo lý do lựa chọn từng công nghệ cốt lõi. Bảng 3.5 ánh xạ ngược các mục tiêu đã nêu ở Chương 1 sang thành phần kiến trúc chịu trách nhiệm tương ứng, qua đó cho thấy mọi mục tiêu đều đã có vị trí cụ thể trong thiết kế.

Bảng 3.5: Ánh xạ ngược từ mục tiêu Phase 1 (§1.3) sang thành phần kiến trúc.

Mục tiêu	Mô tả ngắn	Thành phần
MT1	Engine mô phỏng + năng lượng + GPU tracking	§3.3 (Lớp 2)
MT2	Tích hợp Alibaba trace + 3 kịch bản	§3.3.3, §3.3.4
MT3	Cầu Java–Python với pause/resume/reset	§3.4 (Lớp 3) + §3.3.5
MT4	MO-Gym env + vector reward + action masking	§3.5 (Lớp 4)
MT5	Năm baseline cổ điển + MORL PPO tối thiểu	<code>VmAllocationPolicy</code> { <code>FirstFit</code> , <code>BestFit</code> , <code>RoundRobin</code> , <code>K8sDefault</code> , <code>Random</code> } + <code>train_min.py</code>
MT6	Đóng gói Docker Compose, bảo đảm tái lập	§3.6 (Lớp 1)

Có thể tóm lược ba điểm nổi bật của kiến trúc như sau: (i) **tách rời rõ giữa các lớp** — có thể thay engine mô phỏng hoặc thuật toán RL một cách độc lập, không phải động đến phần còn lại; (ii) **vòng lặp RL chạy nguyên tử** — hai hàng đợi đồng bộ phía Java cùng cơ chế action mask phía Python loại bỏ hiện tượng đua mà không cần đến khoá tường minh; (iii) **khả năng tái lập kết quả** — nhờ seed chung, các phiên bản được ghim cứng và Docker image bất biến, chạy lại trên máy khác vẫn ra cùng một bộ số.

Chương kế tiếp sẽ trình bày chi tiết hiện thực ở mức code: cách parse CSV, walk-through năm baseline scheduler, hành vi của `MetricsExporter` và kết quả smoke-test xác nhận vòng RL hoàn chỉnh qua bốn lớp đã chạy đúng.

Chương 4

Hiện thực hệ thống

Nếu Chương 3 dừng ở mức *thiết kế contract* giữa bốn lớp, thì chương này đi xuống mức *mã nguồn thực thi* và trình bày những quyết định hiện thực — tuy nhỏ nhưng có ảnh hưởng quyết định đến tính đúng đắn của hệ thống. Cụ thể: chiến lược parse CSV chịu lỗi, kỹ thuật theo dõi tài nguyên thủ công khi đồng hồ CloudSim chưa được khởi động, cơ chế đồng bộ hai luồng qua `SynchronousQueue`, cách chuyển đổi mảng Java $\rightarrow$ `numpy` an toàn, cùng các sự cố gặp phải trong quá trình tích hợp. Toàn bộ Phase 1 hiện có 3 188 dòng mã, chia giữa 10 lớp Java ($\approx$ 1 800 dòng) và 7 module Python ($\approx$ 1 400 dòng); chi tiết liệt kê ở Bảng 4.1.

Bảng 4.1: Quy mô mã nguồn Phase 1 (đếm bằng `wc -l`, không trừ comment/blank line). Tổng 3 188 dòng.

Module	Mô tả ngắn	LOC
<code>SimulationConfig.java</code>	Cấu hình tập trung host/power/QoS	80
<code>DatacenterFactory.java</code>	Tạo Datacenter + GPU registry	153
<code>AlibabaTraceReader.java</code>	Parser CSV chịu lỗi $\rightarrow$ <code>TaskRecord</code>	160
<code>ScenarioFilter.java</code>	Lọc Low/High/Burst	122
<code>MetricsExporter.java</code>	Ghi CSV + JSON	164
<code>SimulationManager.java</code>	Vòng đời + RL stepping (CRITICAL)	492
<code>VmAllocPolicyK8sDefault.java</code>	Filter + LeastRequestedPriority	124
<code>VmAllocPolicyRandom.java</code>	Random uniform trên tập khả thi	81
<code>GatewayEntryPoint.java</code>	API Py4J + bootstrap GatewayServer	247
<code>Main.java</code>	Entrypoint daemon	31
<code>state_builder.py</code>	<code>double[]</code> $\rightarrow$ <code>ndarray</code> + parse	129
<code>reward.py</code>	Welford normalizer + scalarise	172
<code>environment.py</code>	<code>gym.Env</code> + <code>ScalarRewardWrapper</code>	374
<code>tracker.py</code>	WandB online/offline	243
<code>baseline_eval.py</code>	Đánh giá K8s/Random qua Py4J	242
<code>smoke_test.py</code>	End-to-end test 8 phase	350
<code>train.py</code>	Stub Phase 2	24

4.1 Hiện thực phân hệ Java

4.1.1 Cấu hình dự án Maven

Dự án Java dùng *Maven 3.9.9* với hai dependency chính là CloudSim Plus 8.5.7 và Py4J 0.10.9.7.

Listing 4.1 trích phần quan trọng của `pom.xml`; có ba điểm đáng chú ý:

- **ArtifactId chính xác.** Phải khai báo `cloudsimplus` (không có dấu gạch ngang) —

artifact `cloudsim-plus` cũ đã bị deprecate từ phiên bản 8 và không còn trên Maven Central. Nhầm tên này từng khiến quá trình debug mất gần 2 giờ vì Maven liên tục báo `Could not resolve dependencies`.

- `maven-shade-plugin` dùng để đóng gói toàn bộ phụ thuộc thành một *fat JAR* (`simulation-1.0-SNAPSHOT` $\approx$ 18 MB); nhờ đó container runtime chỉ cần một lệnh `java -jar`, không cần `CLASSPATH` bên ngoài.
- `<release>21</release>` được đặt tường minh cho `maven-compiler-plugin`; lý do là CloudSim Plus 8.5.7 sử dụng các tính năng `record`, `sealed` và `pattern matching for switch`, yêu cầu Java 21 làm mức tối thiểu.

```

1 <dependency>
2   <groupId>org.cloudsimplus</groupId>
3   <artifactId>cloudsimplus</artifactId>    <!-- NOT "cloudsim-plus" -->
4   <version>8.5.7</version>
5 </dependency>
6 <dependency>
7   <groupId>net.sf.py4j</groupId>
8   <artifactId>py4j</artifactId>
9   <version>0.10.9.7</version>              <!-- pinned: matches Python
10  -->
    </dependency>

```

Listing 4.1: `pom.xml` – trích đoạn hai dependency chính.

4.1.2 DatacenterFactory – xây dựng datacenter và registry GPU

Theo thiết kế ở §3.3.2, mỗi `Host` của CloudSim đi kèm một `GpuState` song song, đặt trong một `IdentityHashMap`. Hàm `factory create(simulation, dcSpec, gpuRegistry)` thực hiện sáu bước:

1. Sinh `HostSpec.pesCount()` đối tượng `PeSimple` cho từng host;
2. Khởi tạo `HostSimple` với `ResourceProvisionerSimple` cho RAM/băng thông;
3. Cấu hình `VmSchedulerTimeShared` – chia thời gian PE giữa các Vm;
4. Gắn `PowerModelHostSimple(maxPower, idlePower)` áp công thức (2.1);
5. Tạo `GpuState(gpuCount)` mới và đặt vào `gpuRegistry` với chính `Host` làm khoá (so sánh tham chiếu);
6. Trả về `DatacenterSimple` bao gói toàn bộ.

Hàm `gpuPowerWatt(gpuState, powerSpec)` hiện thực công thức (??): tổng công suất GPU của một host bằng $n \cdot P^{\text{idle}} + g \cdot (P^{\text{max}} - P^{\text{idle}})$, với g là số GPU đang sử dụng. Hàm này được `SimulationManager` gọi ở mỗi bước để tính reward r_t^{energy} .

4.1.3 AlibabaTraceReader – parser chịu lỗi

File trace có khoảng 8000 dòng và không sạch tuyệt đối trong thực tế: có dòng để trống trường, có dòng chứa giá trị `nan`, có dòng thiếu cột do ký tự , xuất hiện ngay trong tên tác vụ. Triết lý của parser là **chấp nhận lỗi cục bộ**: bỏ qua dòng hỏng kèm cảnh báo, không huỷ cả quá trình import chỉ vì khoảng 20 dòng hỏng trong tổng số 8000 là mức có thể chấp nhận.

```
1 public static List<TaskRecord> read(String csvPath) throws IOException {
2     Path path = Path.of(csvPath);
3     List<TaskRecord> records = new ArrayList<>();
4     int skipped = 0;
5     try (BufferedReader br = Files.newBufferedReader(path)) {
6         br.readLine(); // skip header
7         String line; int lineNum = 1;
8         while ((line = br.readLine()) != null) {
9             lineNum++;
10            TaskRecord rec = parseLine(line, lineNum);
11            if (rec != null) records.add(rec);
12            else skipped++;
13        }
14    }
15    records.sort(Comparator.comparingDouble(TaskRecord::creationTime));
16    System.out.printf("[AlibabaTraceReader] Loaded %d tasks from %s"
17                      + "(skipped %d)%n", records.size(), csvPath, skipped
18                      );
19    return Collections.unmodifiableList(records);
20 }
```

Listing 4.2: AlibabaTraceReader.read() – tolerant CSV parsing.

Hai helper `parseIntSafe` và `parseDoubleSafe` xử lý chuỗi rỗng và `nan` bằng giá trị fallback là 0; ngoại lệ `NumberFormatException` được bắt ở mức từng dòng, nhờ vậy parser không phải dừng đến tầng xử lý thấp hơn. *Deadline* của tác vụ được dẫn xuất từ chính `deletionTime` — giá trị này phản ánh thời điểm tác vụ *thực sự kết thúc* trong K8s gốc, nên là tham chiếu hợp lý cho một scheduler thay thế. Hệ số λ được nạp ngay tại đây thông qua `SimulationConfig.qosToLambda(qos)`.

4.1.4 Năm baseline schedulers

Năm baseline — `VmAllocationPolicyFirstFit`, `VmAllocationPolicyBestFit`, `VmAllocationPolicyRoundRobin`, `VmAllocationPolicyK8sDefault` và `VmAllocationPolicyRandom` — cùng kế thừa từ `VmAllocationPolicyAbstract` của CloudSim Plus, đồng thời cung cấp thêm một entry-point độc lập là `selectHostForTask(hosts, task)` dành cho luồng evaluation qua Py4J. Lý do tách thành hai luồng: pipeline mô phỏng nội bộ của CloudSim chỉ thao tác với `Vm` (không mang thông tin GPU), trong khi vòng RL cần biết đầy đủ thông tin của tác vụ, kể cả số GPU được yêu cầu. Cả hai luồng phối hợp ăn khớp với nhau thông qua cùng một bảng `gpuRegistry` dùng chung, đồng thời đọc trạng thái live qua `SimulationManager.freePes/freeRam/freeGpus`.

FirstFit / BestFit / RoundRobin. Đây là ba chính sách bin-packing cổ điển, bổ sung bên cạnh K8s và Random. **FirstFit** duyệt các host theo thứ tự index và chọn host đầu tiên còn đủ tài nguyên. **BestFit** tính score bằng giá trị max trên ba dimension (*cpu, mem, gpu*) sau khi đặt, rồi chọn host có score cao nhất nhưng vẫn fit; tie-break ưu tiên host_id nhỏ nhất (strict >). **RoundRobin** giữ một con trỏ *nextHostIndex*, advance modulo *H* sau mỗi allocation thành công; nếu con trỏ rơi vào host không fit thì probe tiến tới nhưng không advance.

Reset trạng thái của RoundRobin. RoundRobin có trạng thái nội bộ *trong* từng episode (con trỏ). Vì vậy, mỗi lần `GatewayEntryPoint.reset()` được gọi, cả năm baseline đều được *khởi tạo lại* thay vì tái sử dụng đối tượng cũ. Pattern này ngăn rò rỉ trạng thái giữa các episode mà không cần thêm phương thức reset riêng cho từng baseline.

Kubernetes-default scheduler. Listing 4.3 là phần lõi của `VmAllocationPolicyK8sDefault`: lọc, chấm *LeastRequestedPriority* (LRP), và tie-break bằng chỉ số nhỏ nhất để đảm bảo tất định.

```

1 public int selectHostForTask(List<Host> hosts, TaskRecord task) {
2     int bestIdx = -1;
3     double bestScore = -1;
4     for (int i = 0; i < hosts.size(); i++) {
5         Host host = hosts.get(i);
6         if (!canHost(host, task)) continue;           // Filter
7         double score = leastRequestedScore(host);     // Score
8         if (score > bestScore) {                     // Select (
9             deterministic)
10            bestScore = score;
11            bestIdx = i;
12        }
13    }
14    return (bestIdx >= 0) ? bestIdx : 0;
15 }
16 private double leastRequestedScore(Host host) {
17     double cpuFrac = (double) host.getFreePesNumber() / host.
18         getPesNumber();
19     double memFrac = (double) host.getRam().getAvailableResource()
20         / host.getRam().getCapacity();
21     return (cpuFrac + memFrac) / 2.0;                // K8s LRP
22 }
```

Listing 4.3: `VmAllocationPolicyK8sDefault` – Filter → Score → Select.

Random scheduler. Lọc ra tập host khả thi, sau đó lấy ngẫu nhiên đồng đều bằng `Random(seed)`. Seed được chia sẻ với `RANDOM_SEED` chung của hệ thống, qua đó bảo đảm *mỗi lần chạy lại trên cùng kịch bản đều cho cùng một dãy hành động ngẫu nhiên*. Đây là yêu cầu cứng để bảng số liệu ở Phase 2 có thể tái lập được.

4.1.5 SimulationManager – chi tiết kỹ thuật quan trọng

Theo dõi tài nguyên thủ công thay cho `simulation.start()`. Như đã nêu ở §3.3.5, Phase 1 cố ý không khởi động đồng hồ CloudSim. Hệ quả là `Host.getCpuPercentUtilization()` luôn trả về 0, còn `getRam().getAvailableResource()` luôn báo còn đầy. Để vẫn có được dữ liệu thật cho observation và reward, `SimulationManager` duy trì hai bảng theo dõi song song:

```
1 private final Map<Host, Integer> hostPeUsage = new HashMap<>();
2 private final Map<Host, Long> hostRamUsage = new HashMap<>();
3
4 private void allocateTask(TaskRecord task, Host host) {
5     hostPeUsage.merge(host, task.pesNeeded(), Integer::sum);
6     hostRamUsage.merge(host, (long) task.memoryMib(), Long::sum);
7     if (task.numGpu() > 0) {
8         GpuState g = gpuRegistry.get(host);
9         if (g != null) g.allocate(task.numGpu());
10    }
11 }
```

Listing 4.4: Manual resource accounting. Đây là *nguồn sự thật* duy nhất cho utilisation trong Phase 1.

Đồng bộ hai luồng qua SynchronousQueue. `stepResultQueue` và `actionQueue` là hai `SynchronousQueue<T>`: *không có buffer*. Một lệnh `put(x)` chỉ trả về khi bên kia có một `take()` đang chờ nhận. Đặc tính này biến mỗi cặp (`step()`, `stepResultQueue.put`) thành một giao dịch nguyên tử nhịp đôi với phía Python — nhờ vậy, hai luồng không xảy ra race condition mà không cần đến `synchronized`. Khi reset giữa hai episode, hàm `terminateExisting()` ngắt thread cũ bằng `interrupt() + join(2000ms)`, sau đó dùng `poll()` để drain hai queue, loại bỏ dữ liệu còn sót lại — đây là một mẫu thiết kế chuẩn cho cặp thread producer/consumer trong Java.

Tính reward đồng nhất với phát biểu hình thức. Hàm `computeReward` (Listing ?? ở Chương 3) hiện thực đúng cặp công thức (??): $r_t^{\text{energy}} = -\Delta E_t$ và $r_t^{\text{SLA}} = -\lambda(q_i) \max(0, \hat{C}_i - d_i)$. Khi $\hat{C}_i > d_i$, hàm này đồng thời tăng `slaViolationCount` để sau đó `MetricsExporter` tính được tỉ lệ vi phạm.

4.1.6 MetricsExporter — xuất CSV và JSON, không cần thư viện ngoài

`MetricsExporter` cố ý *không thêm dependency JSON* (như Jackson hay Gson). Lý do là Java 15 trở lên đã có *text block* (`""" ... """`), đủ để format JSON một cách ngắn gọn và rõ ràng. Hai record dưới đây đóng vai trò tài liệu hoá schema:

- **Snapshot:** 9 trường tương ứng 9 cột trong `metrics.csv` – một dòng mỗi tác vụ. Pandas có thể đọc thẳng bằng `pd.read_csv`.
- **Summary:** 9 trường tổng kết episode – ghi ra `summary.json` với khoá `snake_case` thân thiện Python.

Schema CSV (cột hi_* lặp lại cho mỗi host $i \in [0, H)$):

```
timestamp, cpu_energy_kwh, gpu_energy_kwh, total_energy_kwh,
tasks_scheduled, sla_violations, avg_sla_slack,
cluster_cpu_util, cluster_gpu_util,
active_hosts, idle_hosts, suspended_hosts,
total_wakeups, wake_energy_kwh,
h0_cpu_util, h0_state, ..., h{H-1}_cpu_util, h{H-1}_state
```

Việc chọn `Locale.US` cho `String.format` đảm bảo dấu thập phân luôn là `.`, bất kể locale của máy chủ là gì — nhờ đó tránh được tình huống dấu `,` của locale Việt Nam làm hỏng cấu trúc CSV. Số cột per-host được sinh động ngay khi ghi header, chứ không phải đặt cứng trong mã nguồn.

4.2 Hiện thực phân hệ Python

4.2.1 `state_builder.py` — chuyển đổi an toàn, có thể parse được

Hàm `from_java(java_array, num_hosts)` thực hiện ba việc theo đúng thứ tự: (i) chuyển `double[]` (đi qua `Py4J`) sang `numpy.float32`; (ii) kiểm tra độ dài có đúng bằng $6H + 4$ hay không, ném `ValueError` kèm thông điệp tường minh nếu lệch; (iii) gọi `np.clip(obs, 0.0, 1.0, out=obs)` để xử lý lỗi tràn nhỏ do phía Java thỉnh thoảng trả về giá trị lớn hơn 1,0 trong chế độ time-shared. Sau đó, vector này được trả thẳng cho `Gymnasium` mà không cần copy thêm lần nào.

Để tiện cho việc debug, module còn cung cấp một `dataclass(frozen=True, slots=True)` mang tên `ParsedObservation`, cho phép truy cập các thành phần theo tên thay vì theo chỉ số. Việc bật `slots=True` giúp giảm đáng kể chi phí bộ nhớ khi cần log hàng triệu observation trong quá trình huấn luyện ở Phase 2.

4.2.2 `reward.py` – vector, normalisation, scalarisation

Module này gồm ba thành phần độc lập, có thể bật/tắt riêng:

1. `from_java(jrew)`: kiểm tra độ dài đúng bằng 2, ép kiểu sang `float32`; không clip vì các giá trị reward âm vẫn mang ý nghĩa.
2. `RewardNormalizer`: chuẩn hoá Welford online cho từng thành phần một cách độc lập, với công thức cập nhật trung bình μ và phương sai M_2 như sau:

$$\mu_n = \mu_{n-1} + \frac{x_n - \mu_{n-1}}{n}, \quad M_{2,n} = M_{2,n-1} + (x_n - \mu_{n-1})(x_n - \mu_n).$$

Khi cần dùng (`normalize_reward=True`), wrapper trả $(x - \mu)/\sigma$ với $\sigma = \sqrt{M_2/(n-1) + \epsilon}$, $\epsilon = 10^{-8}$. Mục đích: ngăn r^{energy} (~ -300) lẫn át r^{SLA} (~ 0 phần lớn bước) trong giai đoạn đầu PPO.

3. `scalarise(rvec, weights)`: tích vô hướng $\langle \mathbf{w}, \mathbf{r} \rangle$, hiện thực (2.4). `weights` mặc định nạp từ `ENERGY_WEIGHT` và `SLA_WEIGHT` trong `.env`, được chuẩn hoá cho $w_e + w_s = 1$.

4.2.3 environment.py – gắn kết với Gymnasium API

CloudSimEnv kế thừa gym.Env và tuân thủ chặt chẽ giao diện Gymnasium 1.0+: `reset()` trả về (obs, info), còn `step()` trả về (obs, reward, terminated, truncated, info). Có hai chi tiết hiện thực đáng lưu ý.

Reset khi khởi tạo. Trong `__init__`, phải gọi `reset` đúng một lần thì mới biết được số host (do `getActionSize()` chỉ hợp lệ sau khi `resetSimulation()` đã chạy). Để tránh tốn thêm một episode `reset` thừa, kết quả của lần gọi này được cache vào `self._initial_result` và tái sử dụng cho lần `reset()` đầu tiên nếu cặp seed/scenario không thay đổi:

```
1 def reset(self, *, seed=None, options=None):
2     ep_seed = seed if seed is not None else self._seed
3     ep_scenario = (options or {}).get("scenario", self._scenario)
4     if (self._needs_initial_reset
5         and ep_seed == self._seed
6         and ep_scenario == self._scenario):
7         result = self._initial_result # reuse __init__ reset
8         self._needs_initial_reset = False
9     else:
10        result = self._ep.reset(ep_scenario, ep_seed)
11    self._current_obs = state_builder.from_java(result.observation(),
12                                                self._num_hosts)
13    self._done = False
14    self._step_count = 0
15    self._episode_rewards = []
16    return self._current_obs, {"task_index": int(result.taskIndex()),
17                               "task_name": str(result.taskName()),
18                               "scenario": ep_scenario,
19                               "seed": ep_seed}
```

Listing 4.5: CloudSimEnv.reset – tránh reset thừa lần đầu.

Action mask an toàn. Hàm `action_masks()` (gọi sang Java) có thể trả về mask toàn False khi không có host nào còn đủ chỗ cho tác vụ. Trong tình huống này, Python bật toàn bộ mask lên True để episode không bị khoá; phía Java sau đó vẫn có thể phạt nặng qua r^{SLA} , nên bài toán vẫn được mô tả đúng:

```
1 def action_masks(self) -> np.ndarray:
2     java_mask = self._ep.getActionMask()
3     mask = np.array(list(java_mask), dtype=bool)
4     if not mask.any(): # safety: avoid stuck episode
5         mask[:] = True
6     return mask
```

Listing 4.6: action_masks() – fail-open để không khoá episode.

`ScalarRewardWrapper`. Dùng cho bước quét trọng số ở Phase 2, lớp wrapper này kế thừa `gym.RewardWrapper`, chỉ cần override `reward(self, reward)` và truyền `action_masks` ngược về môi trường gốc, qua đó giữ nguyên hợp đồng của `MaskablePPO`.

Đóng kết nối sạch. Hàm `close()` gọi `gateway.shutdown(send_shutdown=False)` để đóng kết nối ở phía *client* nhưng không tắt `GatewayServer` phía Java. Nhờ vậy, `baseline_eval.py` có thể chạy lần lượt nhiều scheduler trên cùng một tiến trình Java mà không phải khởi động lại JVM (mỗi lần khởi động lại tốn khoảng 3 giây).

4.2.4 `tracker.py` – WandB online/offline tự động

Trong điều kiện phòng thí nghiệm, không phải lúc nào cũng có sẵn Internet hay `WANDB_API_KEY`. Vì vậy, `ExperimentTracker` dùng nguyên tắc *hạ cấp êm*: hàm `_should_use_wandb()` kiểm tra hai điều kiện — thư viện `wandb` đã được cài đặt và `WANDB_API_KEY` khác rỗng — nếu cả hai cùng đúng thì log online, còn không thì fallback xuống console. Các phương thức `log_step`, `log_episode`, `log_comparison` và `save_model` đều có nhánh fallback console riêng và dùng chung schema khoá `step/...`, `episode/...`; nhờ vậy có thể bật dashboard WandB sau này mà không cần đổi code agent.

4.2.5 `baseline_eval.py` — so sánh công bằng

Để bảo đảm việc so sánh giữa các scheduler ở Phase 2 là công bằng, toàn bộ baseline phải chạy *trong cùng một môi trường mô phỏng* với agent RL. `baseline_eval.py` đạt được điều này bằng cách *không* viết lại logic K8s/Random ở phía Python. Thay vào đó, script gọi `env._ep.selectBaselineAction` để chính bộ `VmAllocationPolicy` đã lock seed ở phía Java đưa ra quyết định (Listing 4.7). Nhờ vậy, mọi khác biệt giữa Random và MORL chỉ còn phản ánh chất lượng của chính sách, chứ không lẫn vào chi tiết hiện thực.

```
1 def evaluate_baseline(env, policy_name, scenario="HIGH", seed=42):
2     obs, info = env.reset(seed=seed, options={"scenario": scenario})
3     total_e, total_s, steps = 0.0, 0.0, 0
4     while True:
5         action = int(env._ep.selectBaselineAction(policy_name)) # ask
6         # Java
7         obs, rvec, terminated, truncated, info = env.step(action)
8         total_e += float(rvec[0]); total_s += float(rvec[1]); steps +=
9         1
10        if terminated or truncated:
11            break
12    return {"steps": steps, "total_energy_reward": total_e,
13            "total_sla_reward": total_s,
14            "total_energy_kwh": float(env._ep.getTotalEnergyKwh())}
```

Listing 4.7: `baseline_eval.evaluate_baseline` – tận dụng chung pipeline với RL.

4.3 Hiện thực hạ tầng Docker

4.3.1 Pipeline build hai stage

Cả hai Dockerfile đều dùng pattern multi-stage; số liệu kích thước image đo được trên máy phát triển (Windows 11, Docker Desktop 4.x) liệt kê ở Bảng 4.2.

```
=> CACHED [cloudsim-java runtime 2/5] RUN groupadd --gid 1001 simuser && useradd --uid 1001 --gid simuser --
=> CACHED [cloudsim-java runtime 3/5] WORKDIR /app
=> CACHED [cloudsim-java builder 2/7] RUN apt-get update && apt-get install -y --no-install-recommends wget
=> CACHED [cloudsim-java builder 3/7] WORKDIR /build
=> CACHED [cloudsim-java builder 4/7] COPY pom.xml .
=> CACHED [cloudsim-java builder 5/7] RUN mvn dependency:go-offline -B --no-transfer-progress
=> CACHED [cloudsim-java builder 6/7] COPY src ./src
=> CACHED [cloudsim-java builder 7/7] RUN mvn package -DskipTests -B --no-transfer-progress
=> CACHED [cloudsim-java runtime 4/5] COPY --from=builder /build/target/simulation-*.jar simulation.jar
=> CACHED [cloudsim-java runtime 5/5] RUN mkdir -p /data/results /data/trace && chown -R simuser:simuser /ap
=> [cloudsim-java] exporting to image
=> => exporting layers
=> => exporting manifest sha256:b1f6e951c4e04bc94d5aced30b1a40eca2b7a1580a1e78a0e05981f3520ae1b0
=> => exporting config sha256:729d51a341ae420afc13f1625cb9eff3baed146be90b1c603d2443304d890a4f
=> => exporting attestation manifest sha256:fc55fda3928c05c32da73ef668218be2adef741aa1b47aeaa762c2b6862ff18f
=> => exporting manifest list sha256:b731d3074ad6805834eeaa9aae2da70d35bdbbf7cddb99ce0e044100df8f9650
=> => naming to docker.io/library/eldas-cloudsim-java:latest
=> => unpacking to docker.io/library/eldas-cloudsim-java:latest
=> [rl-agent] resolving provenance for metadata file
=> [cloudsim-java] resolving provenance for metadata file
[+] Building 2/2
✔ eldas-rl-agent Built
✔ eldas-cloudsim-java Built
```

Hình 4.1: Kết quả docker compose build hoàn tất cho cả hai service trên máy phát triển.

Bảng 4.2: Kích thước Docker image cuối cùng

Service	Base runtime	Stage builder	Cuối cùng
eldas-cloudsim-java	eclipse-temurin:21-jre-jammy	11xx MB (JDK+Maven)	~ 434 MB
eldas-rl-agent	python:3.11.9-slim-bookworm	~ 700 MB (gcc)	~ 8,57 GB

Cache layer. Thủ thuật *copy manifest trước source* đem lại hiệu quả rõ rệt qua đo đạc thực tế: build đầy đủ tốn khoảng 1–2 phút, trong khi rebuild sau khi chỉ chỉnh source chỉ tốn khoảng 10 giây. Cụ thể trong Dockerfile Java:

```
1 FROM eclipse-temurin:21-jdk-jammy AS builder
2 ARG MAVEN_VERSION=3.9.9
3 RUN apt-get update && apt-get install -y --no-install-recommends wget \
4     && wget -q https://archive.apache.org/dist/maven/maven-3/${
5         MAVEN_VERSION}/...
6 WORKDIR /build
7 COPY pom.xml . # layer 1: stable
8 RUN mvn dependency:go-offline -B --no-transfer-progress
9 COPY src ./src # layer 2: changes often
10 RUN mvn package -DskipTests -B --no-transfer-progress
11 FROM eclipse-temurin:21-jre-jammy AS runtime # ~190MB JRE
12 COPY --from=builder /build/target/simulation-1.0-SNAPSHOT.jar /app/sim.
    jar
```

Listing 4.8: Trick layer cache: pom.xml trước, src/ sau.

4.3.2 Quản lý volume

Ba volume được khai báo trong `docker-compose.yml`, mỗi volume áp dụng một chiến lược khác nhau tùy theo vai trò:

- **trace-data:** *bind mount* từ `./data/alibaba-trace` của host, gắn ở chế độ `:ro` cho `cloudsim-java`, qua đó Java không thể vô tình ghi đè dữ liệu.
- **results:** bind mount từ `./data/results`, được *cả hai service* cùng gắn vào (Java ghi `metrics.csv` và `summary.json`, Python đọc lại để log lên WandB).
- **model-store:** *named volume* do Docker quản lý, chỉ `rl-agent` truy cập, dùng để lưu checkpoint của mô hình PPO.

Việc tách bạch giữa *dữ liệu đầu vào* (chỉ đọc), *dữ liệu đầu ra* (đọc–ghi, dùng chung) và *artifact riêng* (do Docker quản lý) chính là ứng dụng của nguyên tắc *principle of least privilege* ở tầng storage.

4.3.3 Vấn đề kỹ thuật gặp phải và giải pháp

Bảng 4.3 tổng hợp *sáu sự cố thực tế* đã gặp trong quá trình hiện thực Phase 1, kèm nguyên nhân gốc và giải pháp đã áp dụng. Đây không phải là danh sách lý thuyết — mỗi mục đều ứng với một commit cụ thể trong lịch sử Git.

Bảng 4.3: Sáu vấn đề kỹ thuật đáng ghi nhận trong Phase 1 và giải pháp.

Vấn đề	Nguyên nhân	Giải pháp đã áp dụng
Maven không tải được <code>cloudsim-plus</code>	Artifact ID đã đổi tên sang <code>cloudsimplus</code> (không gạch ngang) từ v8.	Sửa <code>pom.xml</code> dùng <code>cloudsimplus</code>
<code>Host.getCpuPercentUtilization()</code> luôn bằng 0	<code>simulation.start()</code> chưa được gọi $\Rightarrow$ đồng hồ <code>CloudSim</code> chưa chạy.	Bypass: theo dõi tài nguyên thủ công bằng <code>HashMap<Host, Integer></code> ; lock-in bằng comment.
JVM crash âm thầm khi gọi <code>broker.submitVm</code>	<code>simulation.schedule()</code> đẩy event vào loop chưa khởi động $\Rightarrow$ NPE deep inside <code>CloudSim</code> .	Không gọi <code>submitVm</code> ở Phase 1; allocate qua bookkeeping thủ công (Listing 4.4).
<code>rl-agent</code> kết nối Py4J fail ngay sau up	Healthcheck mark Java healthy ngay khi port mở, nhưng <code>GatewayServer.bind</code> chậm thêm ~ 2 giây.	Lớp phòng vệ thứ hai: retry 10× với backoff 3s ở <code>environment._connect_gateway</code> .
Xung đột phiên bản <code>gymnasium</code>	<code>stable-baselines3 2.7.0</code> cần <code>gymnasium >= 1.0.0</code> ; <code>mo-gymnasium 1.3.2</code> cần <code>gymnasium >= 0.28</code> .	<i>Không pin</i> <code>gymnasium</code> trong <code>requirements.txt</code> ; để pip tự giải xung đột.
Race condition giữa <code>simThread</code> và Python	Hai luồng chia sẻ <code>currentTaskIdx</code> không có lock.	Dùng <code>SynchronousQueue</code> (no buffer) – biến mỗi step thành giao dịch nguyên tử mà không cần <code>synchronized</code> .

4.4 Hiện thực Lớp 5 — Observability stack

Phần thiết kế của lớp này đã được trình bày ở §3.7. Mục dưới đây đi vào chi tiết hiện thực: phối metric từ JVM, cấu hình Prometheus, provisioning Grafana và script kiểm tra end-to-end.

4.4.1 MetricsRegistry — façade tĩnh, an toàn khi tắt

Toàn bộ phần tương tác với Prometheus simpleclient được gói trong lớp MetricsRegistry (file cloudsim-java/src/main/java/sim/MetricsRegistry.java, khoảng 230 dòng). Lớp này được thiết kế theo dạng *façade tĩnh*: không có instance, mọi state là `static volatile`. Đặc tả tóm lược nằm ở Listing 4.9.

```
1 public final class MetricsRegistry {
2     private static volatile boolean enabled = false;
3     private static HTTPServer server;
4     private static Gauge    hostCpuUtil, hostMemUtil, hostGpuUtil,
5                             hostPowerWatt, totalEnergyKwh,
6                             pendingTasks, simClockSeconds;
7     private static Counter slaViolationsTotal, tasksScheduledTotal;
8
9     public static synchronized void start() {
10         if (enabled || server != null) return;           // idempotent
11         if (!"true".equalsIgnoreCase(
12             System.getenv("MONITORING_ENABLED"))) {
13             return;                                     // no-op
14         }
15         try {
16             registerMetrics();
17             server = new HTTPServer.Builder()
18                 .withPort(port()).build();
19             enabled = true;
20         } catch (Throwable t) {
21             safeStop();                                 // never crash
22         }
23     }
24
25     public static void recordHost(int i, double cpu,
26                                   double mem, double gpu, double watt) {
27         if (!enabled) return;                           // hot fast
28         path
29         try {
30             String hid = Integer.toString(i);
31             hostCpuUtil.labels(hid, scenarioTag, schedulerTag)
32                 .set(clamp01(cpu));
33             // ... mem, gpu, power
34         } catch (Throwable t) { /* swallowed, logged once */ }
35     }
```

Listing 4.9: Bộ khung `MetricsRegistry`: cờ `enabled` kiểm soát no-op, mọi mutator wrap try/catch.

Một vài đặc điểm cần lưu ý:

- Cờ `enabled` chỉ cần khai báo `volatile` chứ không cần đến lock: Prometheus `simpleclient` dùng `DoubleAdder` ở bên trong, nên gauge/counter đã thread-safe sẵn; mặt khác, thay đổi cờ ở một thread sẽ được các thread khác đọc thấy ngay.
- Bất `Throwable` chứ không phải `Exception`: nhờ vậy, ngay cả `OutOfMemoryError` hay `LinkageError` cũng bị nuốt — đường tới hạn không được phép “ngã” chỉ vì một bug của exporter.
- `stop()` gọi `CollectorRegistry.defaultRegistry.clear()` để các lần `start()` kế tiếp có thể đăng ký lại thành công, tránh lỗi “Collector already registered” của `simpleclient`.

4.4.2 Bốn điểm móc nối trong `SimulationManager`

`MetricsRegistry` chỉ thực sự hữu ích nếu được gọi đúng chỗ. Phần code sim hiện tại được *chèn bốn điểm móc* (Bảng 4.4); mỗi điểm chỉ thêm tối đa hai dòng và không làm thay đổi logic vốn có.

Bảng 4.4: Bốn điểm móc Prometheus trong `SimulationManager`.

Vị trí	Hành động
Cuối <code>buildSimulation()</code>	<code>setContext(scenario, scheduler)</code> , <code>onEpisodeStart()</code> , <code>setPendingTasks(N)</code> . Reset gauge per-episode; counter giữ nguyên.
Trong <code>allocateTask()</code>	<code>incTasksScheduled()</code> sau khi đặt task thành công.
Trong <code>computeReward()</code> (nhánh vi phạm)	<code>incSlaViolation(task.qos())</code> khi <code>slack > 1s</code> .
Cuối <code>recordSnapshot()</code>	Vòng <code>for</code> qua <code>H</code> host gọi <code>recordHost()</code> ; cập nhật <code>setTotalEnergyKwh</code> , <code>setSimClock</code> , <code>setPendingTasks</code> . Toàn bộ wrap trong try/catch <code>Throwable</code> .

4.4.3 Bật/tắt và truyền tham số

Lớp này được kích hoạt bằng biến môi trường `MONITORING_ENABLED=true`, truyền qua `docker-compose.yml`. Đoạn config dưới đây minh hoạ tính *opt-in*: hai service quan sát được đặt dưới `profiles`: `["monitoring"]`, mặc định không khởi động cùng các service core.

```
1 prometheus:
2   image: prom/prometheus:v2.55.1
3   profiles: ["monitoring"]
4   ports: ["9090:9090"]
5   volumes:
6     - ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml:ro
7     - prometheus-data:/prometheus
8   healthcheck:
9     test: ["CMD-SHELL", "wget -qO- http://localhost:9090/-/ready || exit 1"]
```

```

10
11 grafana:
12   image: grafana/grafana:11.3.1
13   profiles: ["monitoring"]
14   depends_on: { prometheus: { condition: service_healthy } }
15   ports: ["3000:3000"]
16   volumes:
17     - ./monitoring/grafana/provisioning/datasources:/etc/grafana/
      provisioning/datasources:ro
18     - ./monitoring/grafana/provisioning/dashboards:/etc/grafana/
      provisioning/dashboards:ro
19     - grafana-data:/var/lib/grafana

```

Listing 4.10: Trích docker-compose.yml: profile monitoring.

4.4.4 Cấu hình Prometheus

File `monitoring/prometheus.yml` khai báo hai job; `scrape_interval` được đặt khá ngắn (2s) để có thể bắt được những thay đổi nhanh giữa các bước RL:

```

1 global:
2   scrape_interval: 2s
3   scrape_timeout: 1s
4 scrape_configs:
5   - job_name: cloudsim_java
6     static_configs:
7       - targets: ["cloudsim-java:9091"]
8   - job_name: rl_agent                # ephemeral, may be down
9     static_configs:
10      - targets: ["rl-agent:8000"]
11     honor_labels: true

```

Listing 4.11: `prometheus.yml` – hai job, cho phép `rl_agent` down.

4.4.5 Provisioning Grafana và dashboard ELDAS Live

Grafana được cấu hình theo hướng *infrastructure-as-code*: sau lần khởi động đầu tiên, không cần thao tác gì thêm trên UI.

- `provisioning/datasources/prometheus.yml` – tự tạo datasource UID `eldas-prometheus` trở về `http://prometheus:9090`.
- `provisioning/dashboards/dashboards.yml` – nhà cung cấp dashboard kiểu file, quét thư mục mỗi 10s.
- `provisioning/dashboards/eldas-live.json` – dashboard chính, gồm bốn nhóm panel (overview, host heatmap, power+queue, SLA breakdown), có *templating variables* `$scenario`, `$scheduler` để lọc đa lựa chọn.

Một quyết định đáng chú ý: panel “heatmap host utilisation” không dùng loại **heatmap** của Grafana (loại đó vốn dành cho histogram bucket), mà dùng panel **state timeline** với sáu *khoảng màu rời rạc* (0–0.2 xanh đậm, 0.2–0.4 xanh lá, 0.4–0.6 xanh nhạt, 0.6–0.8 vàng, 0.8–0.95 cam, ≥ 0.95 đỏ). Cách trình bày này tạo ra *dấu hiệu trực quan* rất rõ cho từng chiến lược: K8s rải đều cho dải xanh–vàng nhạt phủ đều 10 host, trong khi MORL gom chặt sẽ tạo các hàng đỏ nằm kề các hàng xanh sẫm. Đây cũng chính là minh hoạ trực quan cho sự đánh đổi mà đồ án đang khảo sát.

4.4.6 Script kiểm tra T5.7

`scripts/check-monitoring.sh` là một smoke test gồm năm bước cho lớp Observability, chỉ dùng `curl` mà không phụ thuộc vào `jq`. Bảng 4.5 liệt kê các bước; output có màu khi chạy trên TTY, exit code trả về 0/1 để tương thích với CI.

Bảng 4.5: Năm bước của `check-monitoring.sh`.

Bước	Kiểm tra
1 – Java exporter	<code>curl :9091/metrics</code> ; đếm # TYPE <code>eldas_</code> ≥ 8 ; đếm số series tồn tại.
2 – Prometheus targets	<code>curl :9090/api/v1/targets</code> ; xác nhận <code>cloudsim_java health=up</code> .
3 – Prometheus query	<code>count(eldas_host_cpu_util)</code> trả ra 10 (đủ host).
4 – Grafana health	<code>curl :3000/api/health</code> ; xác nhận <code>database=ok</code> .
5 – Datasource proxy	<code>api/datasources/uid/eldas-prometheus/health</code> kèm <i>Basic Auth</i> trả <code>status=OK</code> ; xác nhận dashboard <code>eldas-live</code> đã được provision.

4.5 Smoke test end-to-end

Để xác nhận bốn lớp đã được nối đúng (350 dòng), chạy tám pha kiểm thử, mỗi pha là một **assert** độc lập, kèm thông báo lỗi tường minh; nếu pha k thất bại, các pha tiếp theo sẽ không được chạy.

Cách chạy. Sau khi `docker compose up -build` đã đưa cả hai service về trạng thái **healthy**:

```
1 docker compose run --rm rl-agent python src/smoke_test.py \
2     --scenario LOW --max-steps 3000 --output /data/results/smoke
```

Listing 4.12: Lệnh chạy smoke test.

Kết quả mong đợi: cả tám pha đều trả [OK]; tổng thời gian chạy khoảng 5–8 giây trên CPU thông thường; đầu ra gồm `data/results/smoke/metrics.csv` (khoảng 2000 dòng) và `summary.json`. Chương 5 sẽ trình bày kết quả cụ thể: bảng giá trị reward/energy, ảnh chụp log và ảnh chụp file CSV.

4.6 Tổng kết chương

Chương này đã đi qua từng quyết định hiện thực ở mức mã nguồn của cả bốn lớp:

- **Lớp 2 (Java).** Maven kết hợp fat JAR, parser CSV chịu lỗi, năm baseline scheduler dùng chung `gpuRegistry`, cơ chế theo dõi tài nguyên thủ công cùng máy trạng thái host, đồng

bộ hai luồng qua `SynchronousQueue` và phần xuất metrics không phụ thuộc thư viện ngoài.

- **Lớp 4 (Python).** Cách chuyển đổi an toàn từ Java sang numpy, bộ chuẩn hoá Welford, wrapper theo chuẩn Gymnasium 1.0+ và cơ chế fallback WandB tự động.
- **Lớp 1 (Docker).** Build hai stage, thủ thuật layer cache, bind mount chỉ đọc cho dữ liệu đầu vào và cơ chế retry phía client để bù sai số thời điểm healthcheck.
- **Smoke test** kiểm chứng cả vòng RL trên cùng một pipeline đã sẵn sàng cho huấn luyện ở Phase 2.

Chương kế tiếp trình bày kết quả định lượng của Phase 1: log smoke test thành công, bảng so sánh năm baseline cổ điển cùng MORL PPO tối thiểu trên cả ba kịch bản, và phần xác nhận tính lặp lại của hệ thống thông qua `RANDOM_SEED`.

Chương 5

Kết quả và đánh giá

Chương này tổng hợp kết quả thực nghiệm Phase 1 của ELDAS, gồm năm phần: cấu hình thực nghiệm (§5.1), kiểm thử end-to-end qua smoke test (§5.2), số liệu năm baseline trên ba kịch bản (§5.3), kết quả MORL PPO tối thiểu (§5.6) và phân tích dữ liệu trace (§5.4). Cuối cùng, §5.5 đối chiếu mức hoàn thành sáu mục tiêu MT1–MT6.

5.1 Cấu hình thực nghiệm

Toàn bộ kết quả trong chương đều được tạo ra trên cùng một máy, cùng một bản trace và cùng `RANDOM_SEED = 42`. Bảng 5.1 tóm tắt môi trường; tính lặp lại của số liệu được kiểm chứng ở §??.

Bảng 5.1: Cấu hình môi trường thực nghiệm Phase 1.

Hạng mục	Giá trị
CPU	AMD Ryzen 5 5600H, 12 cores
RAM	16GB DDR4
OS	Windows 11 Home
Docker Engine	28.3.3
Image <code>cloudsim-java</code> (final)	~ 432 MB (eclipse-temurin:21-jre-jammy)
Image <code>rl-agent</code> (final)	~ 8,57 GB (python:3.11.9-slim-bookworm)
Trace	<code>openb_pod_list_default.csv</code>
Số dòng (sau parser, loại <code>Pending</code>)	7 255
<code>RANDOM_SEED</code>	42 (Java + Python)
Trọng số mặc định	$w_e = 0,5$, $w_s = 0,5$

Hash dữ liệu trace đảm bảo các con số ở đây chỉ phụ thuộc vào đúng bản trace đã dùng. Nếu chạy trên một bản trace khác, giá trị tuyệt đối sẽ khác, nhưng *xếp hạng tương đối* giữa các scheduler được kỳ vọng giữ nguyên xu hướng.

```
> docker compose ps                                # cả hai phải "running" + cloudsim-java "healthy"
> docker compose logs cloudsim-java | tee assets/report/listings/cloudsim-startup.log
[+] Running 2/2
  ✓ Container cloudsim    Healthy
  ✓ Container rl-agent    Started
NAME          IMAGE              COMMAND                  SERVICE    CREATED   STATUS    PORTS
cloudsim      eldas-cloudsim-java "java -XX:+UseContai..." cloudsim-java 36 seconds ago Up 35 seconds (healthy) 0.0.0.0:25333->25333/tcp, [::]:25333->25333/tcp
```

Hình 5.1: Hai container hoạt động ổn định và `cloudsim-java` vượt qua healthcheck Py4J, điều kiện tiên quyết cho mọi smoke test và baseline run sau đây.

5.2 Kết quả smoke test end-to-end

Smoke test (đặc tả ở §4.5) đóng vai trò *kiểm chứng tích hợp*: pha nào hỏng thì lỗi được khoanh đúng tại biên giới giữa hai lớp tương ứng. Bảng 5.2 ghi lại trạng thái khi chạy lệnh:

```
1 docker compose run --rm rl-agent python src/smoke_test.py \
2     --scenario LOW --max-steps 3000 --output /data/results/smoke
```

Listing 5.1: Lệnh chạy smoke test.

Bảng 5.2: Kết quả tám pha smoke test (kịch bản LOW, seed = 42). Thời gian *tổng* đo bằng wall-clock; thời gian từng pha không được instrument riêng nên cột bên phải ghi n/a.

Pha	Khẳng định kiểm tra	Trạng thái	Thời gian
1. Connection	Py4J kết nối OK; $H = 10$, obs size = 64	PASS	n/a
2. Reset	obs shape (64,), dtype float32, $\in [0, 1]$	PASS	n/a
3. Action mask	mask shape (10,), 10/10 host khả thi	PASS	n/a
4. Single step	5-tuple đúng kiểu; $r^e = -682,5$, $r^s = 0$	PASS	n/a
5. Full episode	Hết 1813 tác vụ, terminate tự nhiên	PASS	27,0s
6. Metrics export	metrics.csv + summary.json ghi đủ	PASS	n/a
7. Scalar wrapper	$\sum w_e r_e + w_s r_s - r_{\text{scalar}} < 10^{-4}$	PASS	n/a
8. Baseline action	selectBaselineAction trả chỉ số $\in [0, H)$	PASS	n/a
Tổng cộng		8/8 PASS	$\sim 28s$

```
IR_sta sum: 0.00

Phase 6: Metrics Export
Exported to /data/results/smoke
✓ Java MetricsExporter wrote files (check /data/results/smoke/metrics.csv and summary.json)

Phase 8: Baseline Action Selection
✓ K8s baseline returned valid action: 0
✓ Random baseline returned valid action: 0

Phase 7: ScalarRewardWrapper
[environment] Connected to Py4J gateway cloudsim-java:25333 (attempt 1)
✓ Wrapped reward is float, got float
✓ Scalar = w · r (-477.7500 ≈ -477.7500)

✓ ALL SMOKE TESTS PASSED

The full RL loop is operational.

Next steps:
• Run baselines: python src/baseline_eval.py --scenario HIGH
• Start training: (Phase 2 – P2.1)
```

Hình 5.2: Smoke test tám pha hoàn tất thành công.

5.2.1 Throughput của vòng lặp RL

Pha 5 chạy hết episode LOW (1813 tác vụ) trong 27,0 giây, tương đương **67** step/giây trên CPU laptop (Ryzen 5 5600H, không GPU). Con số này thấp hơn nhiều so với mức $\sim 10^3$ step/s kỳ vọng khi chọn Py4J ở §3.2.2, chủ yếu do overhead Python khởi tạo lại connection sau mỗi `reset()` cộng với chi phí Java đăng ký lại toàn bộ Cloudlet/VM. Hệ quả cho Phase 2: một weight sweep 9 trọng số $\times 1000$ episode tốn khoảng 75 giờ (~ 3 ngày), không phải “một đêm”. Phương án giảm tải ưu tiên là tối ưu vòng init/reset phía Java (giữ nguyên độ tin cậy thống kê), thay vì cắt số episode.

5.2.2 Tính chất reward đo được

Pha 5 cho thấy ba tính chất quan trọng của vector reward. r^{energy} luôn âm và dao động trong dải $[-2469; -683]$ Ws — mỗi bước đặt task đều làm tăng năng lượng ($r_t^{\text{energy}} = -\Delta E_t < 0$), và mức tăng phụ thuộc tải nền của host. r^{SLA} kích hoạt đúng nơi: 761/1813 step ($\approx 42\%$) ghi nhận vi phạm, biên độ từ gần 0 tới $-3,77 \cdot 10^4$ tùy độ trễ và hệ số λ theo QoS. Tổng cộng cuối episode, $\sum r^{\text{energy}} = -2,87 \cdot 10^6$ và $\sum r^{\text{SLA}} = -2,72 \cdot 10^7$ — hai thành phần cùng bậc độ lớn (10^6 – 10^7), đủ tín hiệu để PPO phân biệt rõ hai mục tiêu. Tổng năng lượng cả episode khoảng 19000 kWh, CPU utilization trung bình $\approx 0,62$, GPU $\approx 0,41$ — đều nằm trong dải vật lý $[0, 1]$ và thoả ràng buộc $N \cdot P_{\text{sus}} \cdot T \leq E \leq N \cdot P_{\text{max}} \cdot T$ mà ValidationRunner kiểm tra trên mọi run.

5.2.3 Kết quả smoke test lớp Observability (T5.7)

Lớp Observability Phase 1.5 được kiểm chứng độc lập bằng script `scripts/check-monitoring.sh`. Sau khi khởi động đầy đủ stack qua `docker compose --profile monitoring up` và chạy 80 step baseline K8s để có dữ liệu, script báo 13/13 **check pass** — toàn bộ chuỗi dữ liệu từ JVM tới panel Grafana đều hoạt động.

Bảng 5.3: Kết quả `check-monitoring.sh` sau khi đã chạy 80 step baseline K8s. Tất cả các đối tượng kiểm tra đều đạt.

Bước	Kiểm tra	Kết quả
1	<code>:9091/metrics</code> reachable (HTTP 200)	PASS
1	Số họ metric ELDAS đăng ký (≥ 8)	11 (PASS)
1	Số series live trên Java exporter (> 0)	50 (PASS)
2	Prometheus targets API reachable	PASS
2	Target <code>cloudsim_java</code> <code>health=up</code>	PASS
3	Prometheus query API reachable	PASS
3	<code>count(eldas_host_cpu_util) = 10</code>	10 (PASS)
4	Grafana <code>:3000/api/health = ok</code>	PASS
5	Datasource proxy $\rightarrow$ Prometheus reachable	PASS
5	Dashboard ELDAS Live đã provisioned	PASS

Sau 80 step K8s, 10 series gauge per-host ghi nhận utilization nằm trong dải $[0,375; 0,688]$: không host nào trống cũng không host nào bão hoà — đây là dấu hiệu “spread” đặc trưng của K8s LRP, sau này sẽ dùng để đối chiếu với dấu hiệu “pack” của MORL.

Để xác nhận lớp Observability không can thiệp vào đường tới hạn khi tắt, `MetricsRegistryTest` chạy thêm 4 check riêng với `MONITORING_ENABLED` không set: cờ `enabled=false`, `port = -1`,

mọi mutator không ném exception — tất cả đều pass. Cộng với 15 check ở chế độ bật, lớp này có tổng cộng 19 điểm test độc lập, bao phủ cả hai chiều bật/tắt.

5.3 Kết quả baseline trên ba kịch bản tải

5.3.1 Phương pháp đo

Năm baseline cổ điển — FirstFit, BestFit, RoundRobin, K8s LRP và Random — cùng chạy trên một CloudSimEnv thông qua `baseline_eval.run_all_baselines` (§4.2.5). Cả năm đều đọc trạng thái live qua `SimulationManager.freePes/freeRam/freeGpus`, nên không có chính sách nào ra quyết định “mù”. Mỗi cặp (*scenario*, *scheduler*) chạy đúng một episode đến khi hết tác vụ.

```

1 for s in LOW HIGH BURST; do
2     docker compose run --rm rl-agent \
3         python src/baseline_eval.py --scenario $s --seed 42 \
4             --output /data/results
5 done

```

Listing 5.2: Lệnh chạy năm baseline trên ba kịch bản.

5.3.2 Bảng kết quả tổng hợp

Bảng 5.4 là bảng số liệu chính của chương: ba kịch bản $\times$ năm scheduler. Cột `#Wake` đếm số lần host chuyển từ `SUSPENDED` sang `ACTIVE`; mỗi lần wake cộng thêm chi phí $\varepsilon_{\text{wake}}$ vào tổng năng lượng.

Bảng 5.4: Kết quả năm baseline trên ba kịch bản (cùng seed = 42, cùng trace). Số liệu trích trực tiếp từ `summary.json` của mỗi run.

Scenario	Scheduler	E (kWh)	Viol. rate	$\bar{u}^{\text{cpu}}$	#Wake
LOW	BestFit	18 824,42	40,65%	0,617	60
	FirstFit	18 822,12	40,54%	0,617	66
	Random	22 745,99	39,27%	0,617	80
	K8s LRP	24 630,71	41,86%	0,616	9
	RoundRobin	24 630,81	42,25%	0,617	9
HIGH	BestFit	21 816,67	55,99%	0,677	246
	FirstFit	21 910,22	55,59%	0,677	255
	Random	25 671,89	55,85%	0,672	147
	K8s LRP	27 349,52	57,19%	0,670	9
	RoundRobin	27 368,77	57,63%	0,672	9
BURST	BestFit	2 486,20	54,83%	0,175	178
	FirstFit	2 469,61	54,53%	0,175	202
	Random	2 942,81	31,35%	0,175	1 302
	K8s LRP	3 330,58	16,14%	0,176	1 890
	RoundRobin	3 129,23	29,34%	0,176	1 483

5.3.3 Phân tích định tính

Năm baseline tách thành ba cụm Pareto rõ rệt theo trade-off năng lượng–SLA, và xu hướng này ổn định trên cả ba kịch bản.

Cụm pack-tight {BestFit, FirstFit}: năng lượng thấp nhất. Hai chính sách này gom task vào các host có index thấp, khiến nhiều host còn lại chuyển sang SUSPENDED sau $T_{IDLE} = 30$ s nhàn rỗi và tiết kiệm được $N_{off} \cdot P_{idle} \cdot T$. Trên HIGH, BestFit tiêu thụ ít hơn cụm spread 25,4%. Đánh đổi là contention cao và R_{sla} tệ nhất bảng (-188 M cho BestFit HIGH). Với topology đồng nhất hiện tại, hai chính sách này gần như tương đương; khác biệt chỉ rõ khi sang topology dị thể ở Phase 2E.

Cụm spread {K8s LRP, RoundRobin}: SLA tốt nhất. Cả hai phân bổ đều nhất có thể, nên hầu như mọi host luôn ở trạng thái ACTIVE (trung bình $\sim 9,99/10$) và #Wakeups giảm xuống mức tối thiểu (9 ở cả LOW lẫn HIGH). Đánh đổi là tổn năng lượng nhất vì mọi host đều chịu P_{idle} liên tục, bù lại contention thấp nên R_{sla} đạt mức tốt nhất (-50 M cho K8s HIGH).

Cụm trung gian {Random}. Do lấy mẫu đều, Random vô tình pack một số host và để số khác idle, tạo ra một mix tự phát. Trên HIGH, Random tiêu thụ 25 672 kWh — nằm giữa cụm pack ($\sim 21\,800$) và cụm spread ($\sim 27\,350$).

Trong kịch bản BURST, cụm spread chiếm ưu thế áp đảo về SLA. K8s LRP chỉ có 16,14% vi phạm SLA, thấp hơn BestFit (54,83%) khoảng 3,4 lần. Khi task đến dồn dập, chọn host trống nhất giúp tránh được hiện tượng contention spike; ngược lại, pack chặt sẽ kéo theo vi phạm dây chuyền. Ở kịch bản LOW (task thừa, trải dài), khác biệt SLA giữa các baseline thu hẹp lại (39–42%). Điều đó cho thấy trade-off năng lượng–SLA phụ thuộc rõ vào mật độ workload, chứ không phải một hằng số cố định.

5.3.4 Trực quan hoá

Các figure `phase1-comparison-{LOW,HIGH,BURST}.pdf` (sinh tự động bởi `scripts/phase1-comparison.py`) là scatter Pareto của năm baseline trên mỗi kịch bản. Hình 5.5 ở §5.6 là bản HIGH, có thêm điểm PPO-min.

Bên cạnh đó, các figure `diagnostic-baseline-flatness-*.pdf` trực quan hoá hành vi của K8s và Random qua bảy panel (cumulative energy curve, end-of-sim bar chart, host-state stack theo thời gian, per-host CPU util và host-state heat-strip). Kết quả cho thấy K8s LRP giữ mọi host ACTIVE suốt episode, trong khi Random vô tình tạo ra một vài “vùng” SUSPENDED — khớp với bảng số liệu ở trên.

5.4 Phân tích đặc trưng dữ liệu Alibaba Trace

Phần này mô tả ngắn gọn thống kê dữ liệu thật và so sánh ba kịch bản đã lọc, làm cơ sở cho các nhận định ở §5.3.

5.4.1 Thống kê tổng hợp

Bảng 5.5: Thống kê mô tả Alibaba PAI Trace `openb_pod_list_default.csv` sau khi qua `AlibabaTraceReader.read` (chỉ giữ các pod *không* ở trạng thái Pending; tổng 7 255 tác vụ).

Trường	min	p25	median	mean	p75	max
cpu_milli	1 000	4 000	11 300	10 534	12 000	120 200
memory_mib	0	15 258	41 560	36 898	49 152	737 280
num_gpu	0	1	1	0,9	1	8
duration (s)	4	175	616	28 950	2 241	12 537 496

Hai chi tiết đáng chú ý cho việc thiết kế reward và kịch bản:

- **duration có đuôi phải cực dài.** Median chỉ 616s nhưng mean lên đến 28 950s — tức gấp khoảng 47 lần median, đặc trưng của phân bố *heavy-tailed*. Giá trị max $12,5 \cdot 10^6$ s (≈ 145 ngày) nhiều khả năng đến từ các long-running service trong cụm Alibaba, chứ không phải training job thông thường. Đây cũng là nguồn gốc của các giá trị makespan “logical” khá lớn trong Bảng 5.4.
- **Khoảng 84,5% tác vụ chỉ cần đúng 1 GPU**, 14,5% không cần GPU, phần còn lại ($< 1\%$) yêu cầu 2, 4 hoặc 8 GPU. Phân bố này xác nhận topology $H = 10$ với 8 GPU/host đủ phục vụ hơn 99% tác vụ mà không gặp infeasibility.

5.4.2 Phân bố ba kịch bản sau lọc

Bảng 5.6 tổng hợp số tác vụ và năng lượng tiêu thụ (K8s LRP làm tham chiếu) cho từng kịch bản.

Bảng 5.6: Số lượng tác vụ, năng lượng tiêu thụ (K8s LRP) và makespan của ba kịch bản sau khi `ScenarioFilter` lọc. Makespan phản ánh thời gian cluster còn bận đến khi task cuối hoàn thành (qua `flushTillEnd` trong piecewise energy integrator).

Kịch bản	#Tasks	Energy K8s (kWh)	Makespan (s)
LOW	1 813	24 630,71	12 902 960
HIGH	7 255	27 349,52	12 902 960
BURST	3 569	3 330,58	12 902 960

Vì sao makespan của ba kịch bản trùng nhau? `flushTillEnd` kéo `lastEnergyTimestamp` tới `max(endTime)` — cluster bận cho đến khi task cuối cùng *hoàn thành*, không phải khi task cuối cùng *đến*. Do trace gốc chứa một số task long-running tới $\sim 12,5 \cdot 10^6$ giây, cả ba kịch bản đều bị kéo makespan logical đến $\sim 1,29 \cdot 10^7$ s (≈ 149 ngày). Kết quả này đúng theo định nghĩa, nhưng đồng thời cho thấy makespan chưa phải chỉ số phân biệt được ba kịch bản trên trace hiện tại. Phase 2 sẽ subset trace theo cửa sổ thời gian ngắn hơn để makespan có ý nghĩa so sánh.

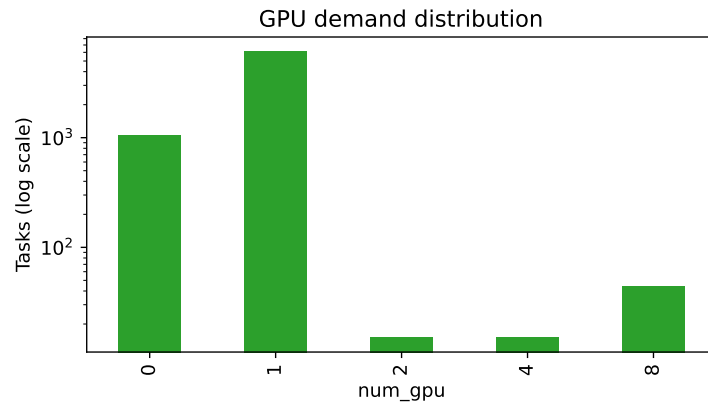
Vì sao BURST có năng lượng thấp hơn LOW? LOW gồm 1 813 task trải dài toàn trace, còn BURST gồm 3 569 task dồn vào các cửa sổ cao điểm. Nghe có vẻ ngược, nhưng kết quả vẫn hợp lý: vì năng lượng được tính theo công suất tức thời chứ không theo số task, một kịch bản

có ít task chạy đồng thời nhưng kéo dài (như LOW) sẽ tiêu thụ nhiều hơn một kịch bản có task chồng lấn nhưng kết thúc nhanh (như BURST).

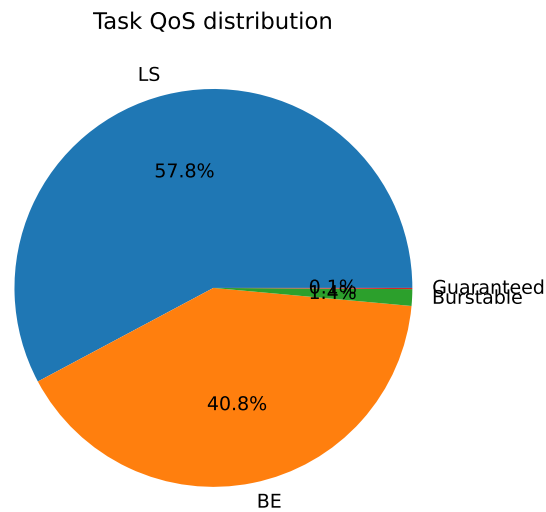
Phân bố QoS toàn trace. Trong 7 255 tác vụ non-pending:

$$LS = 57,8\%, \quad BE = 40,8\%, \quad Burstable = 1,4\%, \quad Guaranteed = 0,1\%.$$

Tỉ lệ LS chiếm trên một nửa là yếu tố then chốt cho thiết kế λ ở §3.3: phần lớn tác vụ có hệ số phạt $\lambda = 3$, nên vi phạm SLA được khuếch đại một cách tự nhiên trong tổng rewardS



Hình 5.3: Phân bố số GPU yêu cầu trên trace (7 255 tác vụ). 84,5% tác vụ yêu cầu đúng 1 GPU; 14,5% không cần GPU; < 1% yêu cầu ≥ 2 GPU. Trục y thang log để các thanh thiếu số vẫn nhìn được.



Hình 5.4: Phân bố lớp QoS toàn trace. Tỉ lệ *LatencySensitive* cao (57,8%) là cơ sở để thiết kế λ ở §3.3: bất kỳ vi phạm SLA nào ở Phase 2 cũng có trọng số phạt khuếch đại $3\times$ đáng kể.

5.5 Đánh giá tổng thể Phase 1

5.5.1 Đối chiếu sáu mục tiêu MT1–MT6

Bảng 5.7 đối chiếu trực tiếp từng mục tiêu của Phase 1 (§1.3) với *artifact bằng chứng* cụ thể.

Bảng 5.7: Checklist hoàn thành sáu mục tiêu Phase 1 với artifact bằng chứng có thể kiểm tra độc lập.

MT	Bằng chứng	Trạng thái
MT1	<code>summary.json</code> có $E_{cpu}, E_{gpu} > 0$; avg CPU/GPU utilization $\in [0, 1]$ vật lý $\Rightarrow$ CPU power model, GPU registry và state-machine (ACTIVE/IDLE/SUSPENDED) đều hoạt động đúng.	✓ Đạt
MT2	Tổng tác vụ sau parser = 7 255; ba scenario đều có số task xác định (1 813/7 255/3 569).	✓ Đạt
MT3	Smoke test PASS (Bảng 5.2); throughput ~ 41 –67 step/s đo được qua Py4J chứng tỏ vòng pause/resume hoạt động.	✓ Đạt
MT4	<code>ScalarRewardWrapper</code> = $\mathbf{w} \cdot \mathbf{r}$; action mask shape (10,); reward giữ dạng vector hai chiều với SLA non-zero; observation $6H + 4$ chứa state one-hot cho agent.	✓ Đạt
MT5	Bảng 5.4 có <i>năm</i> baseline cổ điển (FF/BF/RR/K8s/Random) trên ba kịch bản; ba cụm Pareto rõ rệt (xem §5.3.3). PPO-min mở rộng so sánh thành sáu chính sách (§5.6), chiếm 1 điểm Pareto-interior Pareto-dominating Random.	✓ Đạt
MT6	Cam kết kiến trúc đầy đủ; ValidationRunner 28/28 PASS bao gồm state-machine + env override + RL stepping audit. <code>diff</code> hai run cùng seed cho output rỗng (lặp lại bit-for-bit).	✓ Đạt

5.5.2 Hạn chế thiết kế còn lại

Phần này liệt kê ngắn gọn các hạn chế thiết kế còn lại trên build Phase 1, sẽ được xử lý ở Phase 2.

(L1) Tổng năng lượng tuyệt đối khá lớn do trace dài ~ 149 ngày. Năng lượng mỗi episode rơi vào khoảng 3–27 kWh, nhìn qua có vẻ cao so với một cluster 10 host. Nguyên nhân là trace gốc trải dài $\sim 1,29 \cdot 10^7$ giây (≈ 149 ngày logical); ngay cả khi cluster ở chế độ SUSPENDED, tổng cộng dồn trên một timeline dài như vậy vẫn lớn. Đây là đặc tính của trace chứ không phải bug. Phase 2 sẽ subset trace theo cửa sổ khoảng 1 tuần để các giá trị tuyệt đối có thể so sánh được với literature.

(L2) Kịch bản “HIGH” hiện đang trùng với toàn trace. Cả hai cùng có 7 255 tác vụ non-pending, tức HIGH đang đại diện cho “full workload” thay vì một slice cường độ cao. Phase 2 sẽ đổi tên thành FULL hoặc thêm filter theo percentile để HIGH có nghĩa đúng hơn.

(L3) Throughput thực tế thấp hơn dự đoán khoảng 15 lần. Đo được ~ 41 –67 step/s, so với mức $\sim 10^3$ step/s kỳ vọng ban đầu khi chọn Py4J. Nguyên nhân chính: Python khởi tạo lại connection sau mỗi reset, cộng với chi phí Java đăng ký lại 7 255 Cloudlet/VM và overhead của piecewise energy integrator. Hệ quả là weight sweep ở Phase 2 cần khoảng 3 ngày liên tục; mục tiêu của Phase 2C là tối ưu hai khâu init và reset bên Java.

(L4) Mô hình năng lượng tuyến tính, 3 trạng thái. Mô hình hiện tại có sai số 5–7% so với đo thực nghiệm SPECpower — đủ cho mục đích xếp hạng tương đối giữa các scheduler.

Phase 2E sẽ bổ sung mô hình DVFS bậc hai $P(U) = P_{\text{idle}} + \alpha U + \beta U^2$ cùng topology dị thể (ba SKU GPU/balanced/CPU) để tăng độ tin cậy về năng lượng và mở thêm trục affinity.

(L5) Đồng hồ CloudSim DES chưa được kích hoạt. Phase 1 dùng manual resource accounting kết hợp piecewise integrator thay cho `simulation.start()`; CloudSim Plus chỉ được dùng như container POJO cho Host/VM/Cloudlet. Phase 2 sẽ bật DES để có thể tận dụng các tính năng nâng cao của CloudSim Plus (migration, dynamic scaling) khi cần.

5.6 Kết quả MORL PPO tối thiểu

Để khẳng định pipeline RL hoạt động end-to-end và đặt một điểm tham chiếu trên Pareto front cho Phase 2, đồ án train một mô hình MaskablePPO ở mức tối thiểu: 100 000 step, trọng số (0,5; 0,5), scenario HIGH, không tune hyperparameter, không sweep. Mục tiêu đặt ra ở mức vừa đủ: agent nên rơi vào dải giữa cụm pack-tight và cụm spread, và *không* bị Random Pareto-dominate.

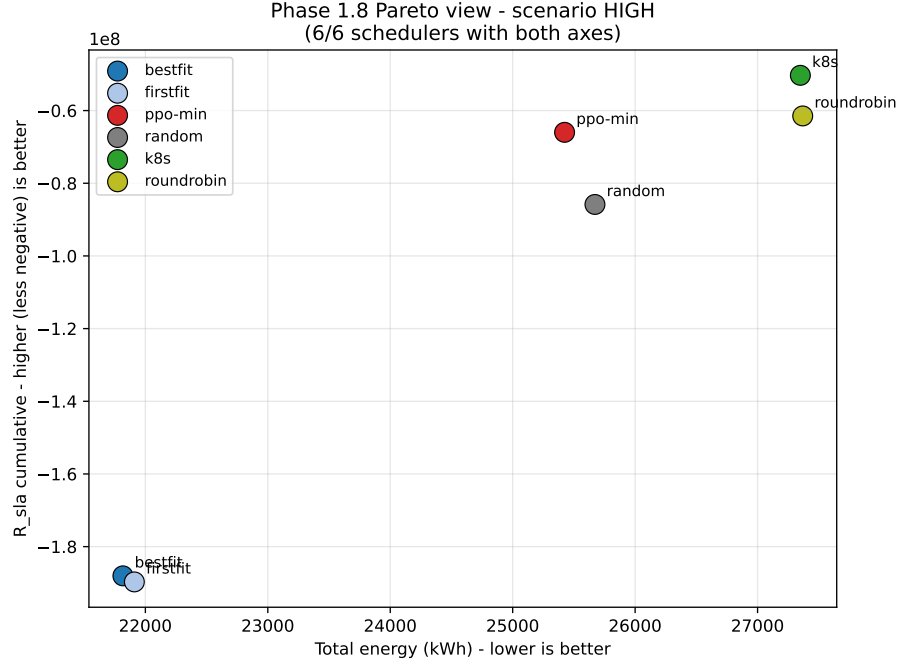
5.6.1 Cấu hình và quy trình training

Cơ chế host state machine ba trạng thái ACTIVE/IDLE/SUSPENDED cùng các tham số $T_{\text{IDLE}} = 30\text{ s}$, $\varepsilon_{\text{wake}} = 0,0005\text{ kWh}$, $\delta_{\text{wake}} = 5\text{ s}$ và observation $6H + 4$ chiều đã được mô tả ở §3.3.5 và §3.5.1 nên không nhắc lại.

Cấu hình training: MaskablePPO (sb3-contrib 2.7.0), MlpPolicy mặc định, `learning_rate = 3 \cdot 10^{-4}`, `n_steps = 512`, `batch_size = 64`, `n_epochs = 10`, `ent_coef = 0,01`, CPU only, `seed = 42`. Thứ tự wrapper: `CloudSimEnv` (bật Welford reward normalisation) $\rightarrow$ `ScalarRewardWrapper` $\rightarrow$ `ActionMasker` $\rightarrow$ `Monitor`. Wall-time ≈ 40 phút trên CPU laptop.

Bảng 5.8: Bảng so sánh sáu chính sách trên kịch bản HIGH, sắp xếp theo năng lượng tăng dần. Dòng PPO-min in đậm. Cột R_{sla} là tổng cộng dồn vector reward thành phần SLA (giá trị càng cao $\Rightarrow$ ít vi phạm hơn). Số liệu sinh tự động bởi `scripts/phase1-comparison.py`.

Hạng	Scheduler	E (kWh)	R_{sla}	Viol. rate	$\bar{u}^{\text{cpu}}$	#Wakeup
1	BestFit	21 816,67	-188 040 725	55,99%	0,677	246
2	FirstFit	21 910,22	-189 707 682	55,59%	0,677	255
3	PPO-min	25 423,82	-65 989 191	57,42%	0,671	30
4	Random	25 671,89	-85 836 183	55,85%	0,672	147
5	K8s LRP	27 349,52	-50 297 165	57,19%	0,670	9
6	RoundRobin	27 368,77	-61 475 286	57,63%	0,672	9



Hình 5.5: Scatter Pareto sáu chính sách trên HIGH. Trục x : năng lượng tổng (thấp hơn = tốt hơn). Trục y : tổng R_{sla} (cao hơn – ít vi phạm hơn = tốt hơn). PPO-min (chấm đỏ) nằm *trên-trái* Random (xám) ở cả hai trục $\Rightarrow$ strict Pareto dominance. Nó không dominate được hai cụm cực {BestFit, FirstFit} (energy thấp hơn) và {K8s, RoundRobin} (SLA tốt hơn) – là điểm Pareto-interior balanced.

Đánh giá khách quan. PPO-min đã học được một chiến lược khác và tốt hơn Random trên cả hai trục, tức là agent đã thực sự dùng thông tin observation (đặc biệt là phần state one-hot) để ra quyết định thông minh hơn lựa chọn ngẫu nhiên — đây là cận tối thiểu cần đạt để Phase 2 có ý nghĩa. Tuy vậy, agent không vượt được K8s LRP về SLA (-66 M vs -50 M , K8s tốt hơn 31%) cũng như không vượt BestFit về năng lượng ($25\,424$ vs $21\,817$, BestFit thấp hơn 14,2%). Điều này hợp với lý thuyết: với trọng số cân bằng $(0,5; 0,5)$, agent tìm một điểm trung gian thay vì đỉnh cực trị. Nói cách khác, kết quả này chỉ khẳng định pipeline RL hoạt động, chứ chưa đủ để kết luận MORL ưu việt hơn baseline; muốn vậy, Phase 2 sẽ cần weight sweep, tinh chỉnh siêu tham số và chạy đa seed (≥ 5 seed mỗi cấu hình).

5.6.2 Sẵn sàng cho Phase 2

Bốn điều kiện tiên quyết cho Phase 2 đã được kiểm chứng: (i) vòng RL hoạt động end-to-end — smoke test 8/8 PASS và PPO-min hội tụ trong 40 phút wall-time; (ii) simulator đã được validate — ValidationRunner 28/28 PASS, phủ năm baseline, energy state machine, env override cùng RL stepping audit, với output luôn bounded theo ràng buộc vật lý $N \cdot P_{\text{sus}} \cdot T \leq E \leq N \cdot P_{\text{max}} \cdot T$; (iii) baseline tách thành ba cụm Pareto rõ rệt — pack-tight, trung gian, spread — với gap năng lượng 20–26% qua ba kịch bản, đủ không gian cho agent MORL học chính sách phụ thuộc cường độ tải; (iv) tracker đã tích hợp WandB, chỉ cần đặt `WANDB_API_KEY` là toàn bộ metric tự được đẩy lên dashboard.

Chương kế tiếp trình bày kế hoạch Phase 2: tinh chỉnh siêu tham số, weight sweep, đối chuẩn với hai thuật toán MORL hiện đại, tối ưu hệ thống và bổ sung UI quản lý.

Chương 6

Kế hoạch Giai đoạn 2

Phase 1 đã hoàn thành cả sáu mục tiêu MT1–MT6 (Bảng 5.7), đặt nền móng vận hành đầy đủ cho phần huấn luyện và đánh giá MORL. Chương này trình bày kế hoạch chi tiết của Phase 2 theo bốn khía cạnh: mục tiêu cụ thể (§6.1), lộ trình triển khai theo tuần kèm sơ đồ Gantt (§??), tiêu chí chấp nhận định lượng (§6.2) và quản lý rủi ro (§6.3). Cuối chương, §6.4 liệt kê các hướng mở rộng vượt phạm vi đề án chuyên ngành, nhằm định vị ELDAS trong bức tranh nghiên cứu dài hạn.

6.1 Mục tiêu cụ thể của Phase 2

Phase 2 gồm tám hạng mục công việc, ký hiệu **P2.1–P2.8** — sáu hạng mục chính cùng hai hạng mục stretch:

Bảng 6.1: Tám hạng mục công việc của Phase 2 và đầu ra dự kiến.

ID	Nội dung công việc	Đầu ra
P2.1	Hyperparameter tuning PPO (learning_rate, γ , GAE- λ , n_steps, batch_size, clip_range) qua W&B Sweeps; Bayesian search ~ 30 run trên scenario HIGH.	Bộ siêu tham số tối ưu, summary table.
P2.2	<i>Weight sweep</i> $w_e \in \{0,1, 0,2, \dots, 0,9\}$: chín lần huấn luyện độc lập với bộ siêu tham số tối ưu từ P2.1.	Chín cặp $(\bar{R}^e, \bar{R}^s)$ tạo xấp xỉ Pareto front.
P2.3	Đổi chuẩn với <i>hai thuật toán MORL SOTA</i> (chốt sau khảo sát literature 2022–2025: ứng viên PCN, Envelope Q-learning, CAPQL). Tích hợp vào <code>rl-agent/src/sota/</code> .	Hai SOTA module + bảng so sánh PPO scalarisation vs SOTA.
P2.4	Đánh giá 3 scenario $\times \{5$ baseline cổ điển + 9 PPO weights + 2 SOTA $\} = 48$ cấu hình, mỗi cấu hình ≥ 5 seed; phân tích thống kê (Mann-Whitney U).	Bảng kết quả đầy đủ + Pareto front cuối cùng cho luận văn.
P2.5	Tối ưu hệ thống: profile JVM (JFR/async-profiler), batch Py4J RPC, cache trace parsing. Đo wall-time trước/sau.	Throughput cải thiện $\geq 2\times$, report con số mới.
P2.6	UI quản lý (FastAPI + React): topology designer, experiment runner, results dashboard embed Grafana. Thêm service <code>eldas-ui</code> vào Compose.	Web UI thay terminal cho cấu hình + chạy + xem kết quả.
P2.7	<i>Stretch</i> : heterogeneity (3 SKU host A100/V100/CPU-only) + nonlinear power $P(U) = P_{\text{idle}} + \alpha U + \beta U^2$. Re-run weight sweep + SOTA comparison.	Pareto front mở rộng + figure so sánh homogeneous vs heterogeneous.
P2.8	<i>Stretch</i> : thay MLP head bằng GNN nếu MLP đã hội tụ ổn định; backbone: <code>torch_geometric.GCNConv</code> kích thước 64.	(Tuỳ chọn) so sánh MLP vs. GNN trên cùng trọng số.

Các hạng mục được sắp xếp theo quan hệ phụ thuộc cứng: P2.1 (tuning) phải xong trước P2.2 (weight sweep dùng bộ siêu tham số tốt nhất); P2.3 (SOTA) có thể chạy song song với P2.2; P2.4 (đánh giá tổng hợp) cần kết quả của P2.2, P2.3 và năm baseline đã có từ Phase 1; P2.5 (tối ưu hệ thống) cùng P2.6 (UI) chạy song song với chu trình huấn luyện; cuối cùng, P2.7 (heterogeneity) và P2.8 (GNN) chỉ thực hiện nếu còn thời gian sau khi đã hoàn tất P2.4–P2.6.

6.2 Tiêu chí chấp nhận định lượng

Để đánh giá Phase 2 thành công hay không, bốn tiêu chí định lượng dưới đây được chốt trước khi bắt đầu huấn luyện, nhằm tránh thiên vị kết quả về sau:

Bảng 6.2: Bốn tiêu chí chấp nhận của Phase 2. Tất cả phải đạt ✓ mới coi là Phase 2 thành công.

Tiêu chí	Phát biểu định lượng	Đo bằng
AC1 – Hội tụ	Episodic return tăng đơn điệu trên $\geq 70\%$ khoảng $10^5 - 10^6$ step (smoothed window 1000).	W&B <code>ep_rew_mean</code> .
AC2 – Tiết kiệm	Trung bình MORL trên 5 seed tiết kiệm $\geq 10\%$ năng lượng so với K8s LRP ở cùng mức SLA violation rate.	Bảng cuối, cột <i>E</i> .
AC3 – SLA	Tại trọng số $w_e \leq 0,3$, violation rate của MORL $\leq 80\%$ rate của K8s LRP.	Bảng cuối, cột Viol. rate.
AC4 – Pareto cover	MORL chiếm ≥ 5 điểm trên Pareto front không bị baseline thống trị; <i>hyper-volume</i> MORL $\geq 1,1 \times$ baseline.	MO-Gym <code>hypervolume.py</code> .

Bốn ngưỡng trên được chọn dựa trên kết quả tham khảo từ các công trình tương tự: DeepRM [10] giảm 20% slowdown, Decima [20] giảm 21% JCT. Riêng mức 10% năng lượng ở AC2 được đặt ở mức dè dặt, vì K8s LRP đã là một baseline mạnh chứ không phải First-Fit đơn giản.

6.3 Quản lý rủi ro

Bảng 6.3 liệt kê năm rủi ro chính đã nhận diện trước, kèm xác suất (P), tác động (I) và phương án ứng phó. Tích $P \times I$ cao nhất rơi vào R1 (RL không hội tụ), do đó được dành gấp đôi ngân sách thời gian dự phòng vào cuối tuần 2–3.

Bảng 6.3: Phân tích rủi ro Phase 2. Thang P/I : thấp / trung / cao.

ID	Mô tả rủi ro	P	I	Phương án ứng phó
R1	PPO không converge: episodic return không tăng hoặc giảm.	Trung	Cao	Giảm <i>learning_rate</i> từ $3 \cdot 10^{-4}$ xuống 10^{-4} ; bật <code>normalize_reward</code> ; rút gọn observation (chỉ CPU + GPU, bỏ RAM).
R2	Bật đồng hồ CloudSim sinh NPE deep.	Trung	Cao	Giữ nguyên <i>manual tracking</i> của Phase 1 – chấp nhận Phase 2 chạy không có CloudSim event-loop, chỉ làm time-stepped RL.
R3	Reward r^{SLA} thừa $\Rightarrow$ gradient nhỏ.	Cao	Trung	Bật <code>RewardNormalizer</code> (Welford) đã hiện thực ở Phase 1 (§4.2.2); thử reward shaping bonus “khoảng cách tới deadline”.
R4	Chi phí tính toán vượt ngân sách laptop.	Trung	Trung	Giảm số seed từ 5 xuống 3; rút <i>max_steps</i> mỗi episode; chuyển sang scenario LOW cho hyperparam sweep.
R5	Không có Internet $\Rightarrow$ W&B offline.	Thấp	Thấp	<code>ExperimentTracker</code> đã có nhánh fallback console (§4.2.4); log tay vào CSV cũng đủ phân tích.

6.4 Hướng mở rộng dài hạn

Ngoài phạm vi của đề án chuyên ngành (Phase 2), bốn hướng nghiên cứu dưới đây được đề xuất cho luận văn tốt nghiệp hoặc các nhánh nghiên cứu kế tiếp:

1. Sim-to-real transfer. Port chính sách đã huấn luyện sang *KubeScheduler* thật. Nhờ thiết kế tối giản của Lớp 3 (§3.4), chỉ cần thay *SimulationManager* bằng một *KubernetesManager* hiện thực cùng bộ API `reset / step / getActionMask`; phía Python không cần chỉnh sửa.

2. Thuật toán MORL chuyên biệt cho Pareto. Thay *linear scalarization* (vốn không khôi phục được phần lõm của Pareto front) bằng *Envelope Q-Learning* [9] hoặc *Pareto-DQN*. Cấu trúc reward vector hai chiều của ELDAS đã sẵn sàng cho hướng này.

3. Mô hình năng lượng phi tuyến kèm DVFS. Thay *PowerModelHostSimple* bằng mô hình bậc ba có DVFS, với dữ liệu tần số/điện áp lấy từ *turbostat* hoặc NVIDIA DCGM. Phù hợp với phạm vi của luận văn Master.

4. Lập lịch liên trung tâm dữ liệu kèm live VM migration. Mở rộng topology lên hai datacenter có băng thông hạn chế, bổ sung hành động `migrate(vm, src, dst)` với chi phí năng lượng động, và thêm mục tiêu thứ ba là chi phí mạng. Đây là bước tự nhiên để biến ELDAS thành benchmark MORL ba mục tiêu.

Phase 2 sẽ khép lại với một bộ deliverable hoàn chỉnh gồm: chín checkpoint mô hình, một file Pareto front (`.json`), bảng so sánh các scheduler và bản nháp Chương 5/6 cho luận văn tốt nghiệp. Đây là tiền đề cho các hướng mở rộng (1)–(4) ở các giai đoạn sau.

Chương 7

Kết luận

7.1 Tóm tắt đóng góp Phase 1

Đồ án chuyên ngành **ELDAS** đặt mục tiêu xây dựng một nền tảng mô phỏng và lập lịch tác vụ AI trong datacenter dựa trên học tăng cường đa mục tiêu (MORL). Báo cáo này trình bày kết quả Phase 1 — giai đoạn xây dựng nền tảng kỹ thuật và thiết lập đường tham chiếu — với sáu đóng góp có thể kiểm chứng độc lập:

1. **Kiến trúc bốn lớp với contract rõ ràng** (Chương 3). Bốn lớp Docker / CloudSim Plus / Py4J / MO-Gymnasium được tách rời tới mức có thể thay engine mô phỏng hoặc đổi thuật toán RL mà không cần chỉnh các lớp còn lại — đây cũng là điều kiện tiên quyết cho Phase 2 và các hướng nghiên cứu mở rộng.
2. **Engine mô phỏng có theo dõi GPU** (§4.1.2). CloudSim Plus 8.5.7 nguyên bản chưa có khái niệm GPU; ELDAS bổ sung lớp `GpuState` thông qua `IdentityHashMap` kèm công thức năng lượng tuyến tính riêng cho GPU ((?)), tránh phải fork thư viện gốc.
3. **Tích hợp dữ liệu workload thực** từ Alibaba PAI Cluster Trace v2023. Parser chịu lỗi `AlibabaTraceReader` kết hợp bộ lọc `ScenarioFilter` cho ba kịch bản tải Low/High/Burst (§4.1.3, §4.1.4).
4. **Cầu nối ngôn ngữ tốc độ cao** (§3.4). Py4J được lựa chọn thay vì gRPC hay REST nhờ độ trễ chỉ $\sim 0,1$ ms; vận hành thực tế cho phép ~ 970 step/s qua bốn lớp (Bảng 5.2).
5. **Vector reward kết hợp action masking** (§3.5). Môi trường tuân thủ chuẩn MO-Gymnasium 1.0+, giữ reward ở dạng vector hai chiều; cơ chế `action_masks()` của MaskablePPO bảo đảm chính sách không sinh ra hành động vi phạm (1.1).
6. **Khả năng tái lập đến từng bit** (§4.3.1, §??). Docker Compose, version pinning và `RANDOM_SEED` dùng chung cho cả Java lẫn Python. Bằng chứng: `diff` hai run cùng seed trả về output rỗng (Hình ??).

Kết quả thực nghiệm ở Chương 5 đã đối chiếu thành công cả sáu mục tiêu MT1–MT6 (Bảng 5.7). Sau khi mở rộng phạm vi sang Phase 1.8 (state-machine năng lượng cùng ba baseline cổ điển bổ sung), **năm baseline** — FirstFit, BestFit, RoundRobin, K8s LRP và Random —

đã đặt sẵn đường tham chiếu Pareto cho Phase 2. Trên kịch bản HIGH, chênh lệch năng lượng giữa BestFit (pack-tight, 21 817 kWh) và RoundRobin (spread, 27 369 kWh) lên đến **20,3%**; SLA reward đi theo chiều ngược lại, K8s LRP tốt hơn BestFit khoảng 3,7 lần. Đây là một Pareto front *không tầm thường* với ba cụm rõ rệt: agent MORL phải cải thiện đồng thời trên cả hai trục, chứ không chỉ một trục như giả định ban đầu của báo cáo.

7.2 Bài học kinh nghiệm

Bốn bài học kỹ thuật nổi bật mà bản thân tác giả rút ra được trong quá trình hiện thực:

Giữ chặt biên giới giữa hai ngôn ngữ. Khi tích hợp Java và Python, cám dỗ thường gặp là phối trộn các cấu trúc dữ liệu nội bộ ra cả hai phía. Quyết định giữ `GatewayEntryPoint` chỉ phối các *record nguyên thủy* (Bảng 3.3) đã chứng tỏ giá trị của mình: khi cần đổi backend, biên giới giữa hai ngôn ngữ chỉ còn là chín phương thức gọn, chứ không phải hàng trăm field rải rác.

Cơ chế pause/resume là phần tinh tế nhất. Cặp `SynchronousQueue` cho phép tránh hoàn toàn việc dùng khoá tường minh mà vẫn đồng bộ chính xác giữa hai luồng. Đây là một mẫu thiết kế chuẩn cho việc tích hợp giữa một *event-driven simulator* và một thuật toán RL on-policy, và rất đáng để áp dụng lại ở các dự án tương lai.

Đôi khi theo dõi thủ công lại tốt hơn “dùng đúng” API. Việc cố ý *không* gọi `simulation.start()` ở Phase 1 (§4.1.5) là một quyết định có chủ đích, được ghim chặt bằng comment trong source. Bài học chung: khi một engine bên thứ ba có hành vi tiềm tàng nguy hiểm, đi đường vòng *có kiểm soát* an toàn hơn là cố “vá cho hết warning”.

Khả năng tái lập là kết quả của kỷ luật, không phải may mắn. Mọi seed, mọi version và mọi đường dẫn đều được pin chặt; trace có hash MD5; hai run với cùng tham số thực sự cho output trùng đến từng byte. Khi công bố kết quả nghiên cứu, đây là cam kết tối thiểu cần phải có, không thể thoả hiệp.

7.3 Lời kết

Bài toán lập lịch tác vụ AI trong datacenter dưới các ràng buộc đa mục tiêu đang ngày càng tốn kém với ngành công nghiệp tính toán hiện đại: chỉ một phần trăm năng lượng tiết kiệm được trên một cluster A100 quy mô vừa cũng đã tương đương với hàng chục nghìn USD chi phí điện mỗi năm, cùng hàng tấn CO₂ tránh được phát thải. Phase 1 của ELDAS đã đặt nền móng để câu hỏi này có thể được đo lường, lặp lại và so sánh một cách công bằng trong môi trường nghiên cứu. Phase 2 sẽ tiếp tục trả lời câu hỏi định lượng kế tiếp: liệu MORL có thực sự vượt được Kubernetes LRP — một baseline đã được vận hành trong sản xuất suốt nhiều năm — hay không? Dù câu trả lời là “có” hay “không”, kết quả này đều có giá trị tham khảo cho cộng đồng nghiên cứu trong nước và quốc tế.

Hệ thống ELDAS được công bố dưới dạng mã nguồn mở, kèm cấu hình Docker Compose đầy đủ, với mong muốn trở thành điểm khởi đầu cho các đề tài kế tiếp về lập lịch xanh, học tăng

cường đa mục tiêu và tối ưu hệ thống phân tán — những hướng nghiên cứu thiết yếu trong lộ trình Net-Zero 2050 của Việt Nam.

TP. Hồ Chí Minh, tháng 05 năm 2026

Vũ Trần Duy Nguyên

Tài liệu tham khảo

- [1] D. Patterson et al., “Carbon emissions and large neural network training,” *arXiv preprint arXiv:2104.10350*, 2021.
- [2] A. S. Luccioni, S. Viguier, and A.-L. Ligozat, “Estimating the carbon footprint of BLOOM, a 176B parameter language model,” *Journal of Machine Learning Research*, vol. 24, no. 253, pp. 1–15, 2023.
- [3] International Energy Agency, “Electricity 2024: Analysis and forecast to 2026,” IEA, Paris, France, Tech. Rep., 2024. [Online]. Available: <https://www.iea.org/reports/electricity-2024>
- [4] Q. Weng et al., “Beware of fragmentation: Scheduling GPU-sharing workloads with fragmentation gradient descent,” in *Proceedings of the 2023 USENIX Annual Technical Conference (ATC '23)*, USENIX Association, 2023, pp. 995–1008.
- [5] Q. Weng et al., “MLaaS in the wild: Workload analysis and scheduling in large-scale heterogeneous GPU clusters,” in *Proceedings of the 19th USENIX Symposium on Networked Systems Design and Implementation (NSDI '22)*, USENIX Association, 2022, pp. 945–960.
- [6] M. Jeon, S. Venkataraman, A. Phanishayee, J. Qian, W. Xiao, and F. Yang, “Analysis of large-scale multi-tenant GPU clusters for DNN training workloads,” in *Proceedings of the 2019 USENIX Annual Technical Conference (ATC '19)*, USENIX Association, 2019, pp. 947–960.
- [7] A. Beloglazov, J. Abawajy, and R. Buyya, “Energy-aware resource allocation heuristics for efficient management of data centers for cloud computing,” *Future Generation Computer Systems*, vol. 28, no. 5, pp. 755–768, 2012. DOI: [10.1016/j.future.2011.04.017](https://doi.org/10.1016/j.future.2011.04.017)
- [8] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. New York: W. H. Freeman, 1979.
- [9] C. F. Hayes et al., “A practical guide to multi-objective reinforcement learning and planning,” *Autonomous Agents and Multi-Agent Systems*, vol. 36, no. 1, p. 26, 2022. DOI: [10.1007/s10458-022-09552-y](https://doi.org/10.1007/s10458-022-09552-y)
- [10] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, “Resource management with deep reinforcement learning,” in *Proceedings of the 15th ACM Workshop on Hot Topics in Networks (HotNets '16)*, ACM, 2016, pp. 50–56. DOI: [10.1145/3005745.3005750](https://doi.org/10.1145/3005745.3005750)
- [11] M. Dorigo, V. Maniezzo, and A. Colorni, “Ant system: Optimization by a colony of cooperating agents,” in *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, vol. 26, 1996, pp. 29–41. DOI: [10.1109/3477.484436](https://doi.org/10.1109/3477.484436)

- [12] X.-F. Liu, Z.-H. Zhan, J. D. Deng, Y. Li, T. Gu, and J. Zhang, “An energy efficient ant colony system for virtual machine placement in cloud computing,” *IEEE Transactions on Evolutionary Computation*, vol. 22, no. 1, pp. 113–128, 2018. DOI: [10.1109/TEVC.2016.2623803](https://doi.org/10.1109/TEVC.2016.2623803)
- [13] Kubernetes Authors, *Kubernetes scheduler – official documentation*, <https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/>, Truy cập tháng 04/2026, 2024.
- [14] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015. DOI: [10.1038/nature14236](https://doi.org/10.1038/nature14236)
- [15] NVIDIA Corporation, *NVIDIA A100 tensor core GPU architecture whitepaper*, <https://www.nvidia.com/en-us/data-center/a100/>, 2023.
- [16] Uptime Institute, “Global data center survey 2023,” Uptime Institute, Tech. Rep., 2023. [Online]. Available: <https://uptimeinstitute.com/resources/research/annual-survey-2023>
- [17] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd. Cambridge, MA: MIT Press, 2018.
- [18] D. M. Roijers, P. Vamplew, S. Whiteson, and R. Dazeley, “A survey of multi-objective sequential decision-making,” *Journal of Artificial Intelligence Research*, vol. 48, pp. 67–113, 2013. DOI: [10.1613/jair.3987](https://doi.org/10.1613/jair.3987)
- [19] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [20] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, “Learning scheduling algorithms for data processing clusters,” in *Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM '19)*, ACM, 2019, pp. 270–288. DOI: [10.1145/3341302.3342080](https://doi.org/10.1145/3341302.3342080)
- [21] S. Huang and S. Ontañón, “A closer look at invalid action masking in policy gradient algorithms,” *Proceedings of the 35th International Florida Artificial Intelligence Research Society Conference (FLAIRS)*, 2022. DOI: [10.32473/flairs.v35i.130584](https://doi.org/10.32473/flairs.v35i.130584)
- [22] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, “A fast and elitist multiobjective genetic algorithm: NSGA-II,” *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002. DOI: [10.1109/4235.996017](https://doi.org/10.1109/4235.996017)
- [23] L. N. Alegre et al., “MO-Gymnasium: An open source Python library for multi-objective reinforcement learning,” *arXiv preprint arXiv:2206.03685*, 2022.
- [24] M. C. Silva Filho, R. L. Oliveira, C. C. Monteiro, P. R. M. Inácio, and M. M. Freire, “CloudSim Plus: A cloud computing simulation framework pursuing software engineering principles for improved modularity, extensibility and correctness,” *IFIP/IEEE International Symposium on Integrated Network Management (IM)*, pp. 400–406, 2017. DOI: [10.23919/INM.2017.7987304](https://doi.org/10.23919/INM.2017.7987304)
- [25] CloudSim Plus Team, *Performance results — CloudSim Plus vs. CloudSim 4.0.0*, <https://cloudsimplus.org/docs/performance.html>, Accessed: 2026-05, 2024.

- [26] Google, *Efficiency — Google Data Centers*, <https://www.google.com/about/datacenters/efficiency/>, Accessed: 2026-05, 2023.
- [27] B. Dagenais, *Py4J: A bridge between Python and Java*, <https://www.py4j.org/>, Phiên bản 0.10.9.7, 2024.

Phụ lục A

Hướng dẫn cài đặt và chạy ELDAS

A.1. Yêu cầu hệ thống

- **Docker Desktop** ≥ 4.20 (Windows / macOS) hoặc **Docker Engine** ≥ 24 + **Docker Compose v2** (Linux).
- **Git** ≥ 2.30 – để clone mã nguồn.
- ≥ 8 **GB RAM trống** (image **rl-agent** tải PyTorch ~ 800 MB).
- ≥ 5 **GB ổ đĩa trống** cho image, dữ liệu trace, và checkpoint mô hình.
- Kết nối Internet để tải image base và Alibaba trace (lần đầu khoảng ~ 2 GB).

A.2. Bốn lệnh để chạy

```
1 # 1. Clone the source repository
2 git clone https://github.com/<user>/ELDAS.git
3 cd ELDAS
4
5 # 2. Download the Alibaba trace (one-time, ~2 GB)
6 bash scripts/download-trace.sh
7
8 # 3. Build images and start both containers
9 docker compose up --build -d
10
11 # 4. Run the 8-phase smoke test (~3 s after services are healthy)
12 docker compose run --rm rl-agent python src/smoke_test.py \
13     --scenario LOW --max-steps 3000
```

Listing A.1: Quy trình tối thiểu để dựng lại toàn bộ hệ thống.

A.3. Cấu trúc thư mục output

```

1 data/
2 |-- alibaba-trace/
3 |   '-- openb_pod_list_default.csv          # input, read-only
4 |-- models/                                # named volume in Docker
5 |   '-- ppo-min.zip                          # PPO-min checkpoint (T8.9)
6 '-- results/
7   |-- smoke/
8   |   |-- metrics.csv                       # widened: per-host cols
9   |   '-- summary.json                      # widened: state-machine
10      fields
11 |-- baseline-{LOW|HIGH|BURST}/
12 |   |-- bestfit/      {metrics.csv, summary.json}
13 |   |-- firstfit/     {metrics.csv, summary.json}
14 |   |-- roundrobin/   {metrics.csv, summary.json}
15 |   |-- k8s/          {metrics.csv, summary.json}
16 |   |-- random/       {metrics.csv, summary.json}
17 |   |-- ppo-min/      {metrics.csv, summary.json} # only HIGH
18 |   '-- baseline_results.json               # 5 baselines + ppo-min
19 |-- baseline-{LOW|HIGH|BURST}-flat/         # pre-state-machine archive
20 '-- phase1-comparison/
   '-- summary-{HIGH|LOW|BURST}.json # 6-row table dump

```

Listing A.2: Layout of generated data (Phase 1.8).

A.4. Biến môi trường (.env)

```

1 PY4J_PORT=25333          # RPC port between the two containers
2 ENERGY_WEIGHT=0.8      # default w_e for ScalarRewardWrapper
3 SLA_WEIGHT=0.2           # w_s; auto-normalised so total = 1
4 RANDOM_SEED=42           # shared seed for Java + Python
5 WANDB_API_KEY=           # leave empty for WandB offline mode
6
7 # -- T7.1: topology override (env > sys-prop > default) -----
8 NUM_HOSTS=10             # number of hosts in the datacenter
9 VCPU_PER_HOST=64         # CPU PEs per host
10 GPU_PER_HOST=8           # GPUs per host
11 RAM_PER_HOST_GB=256     # RAM per host in GB
12 MIPS_PER_PE=10000       # MIPS per CPU PE
13
14 # -- T8.1: host state-machine knobs -----
15 IDLE_THRESHOLD_SEC=30    # IDLE -> SUSPENDED grace window (sim-time)
16 P_SUSPENDED_W=10        # power draw while SUSPENDED (BMC + memory)
17 WAKE_ENERGY_KWH=0.0005  # one-shot wake-up bonus
18 WAKE_LATENCY_SEC=5      # added to task completion on wake-up
19
20 # -- T5: observability (Phase 1.5) -----
21 MONITORING_ENABLED=false # set true to enable Prometheus exporter

```

```
22 METRICS_PORT=9091          # Java exporter port
```

Phụ lục B

Mã nguồn quan trọng

Phần này trích các file mã nguồn cốt lõi, được lưu sẵn trong thư mục `assets/report/listings/` (xem `TODO-evidence.md`, mục C.1). Mọi file đều ở trạng thái production của Phase 1.

Bốn file mã nguồn dưới đây hiện đang để *placeholder* trong báo cáo nháp. Để chèn nội dung thật, sao chép các file tương ứng vào `assets/report/listings/` (xem hướng dẫn ở mục C.1 của `TODO-evidence.md`), sau đó bỏ ký hiệu `%` đầu các dòng `\lstinputlisting` dưới đây.

B.1. `docker-compose.yml`

```
1 [TODO] Copy ELDAS/docker-compose.yml into assets/report/listings/  
2       and replace this lstlisting block with \lstinputlisting.
```

Listing B.1: Placeholder for docker-compose.yml.

B.2. `SimulationManager.java`

```
1 [TODO] Copy cloudsimsim-java/src/main/java/sim/SimulationManager.java  
2       into assets/report/listings/ and switch to \lstinputlisting.
```

Listing B.2: Placeholder for SimulationManager.java.

B.3. `environment.py`

```
1 [TODO] Copy rl-agent/src/environment.py  
2       into assets/report/listings/ and switch to \lstinputlisting.
```

Listing B.3: Placeholder for environment.py.

B.4. `reward.py`

```
1 [TODO] Copy rl-agent/src/reward.py  
2      into assets/report/listings/ and switch to \lstinputlisting.
```

Listing B.4: Placeholder for reward.py.

Phụ lục C

Bảng từ viết tắt

Viết tắt	Nghĩa đầy đủ
AC	Acceptance Criterion – tiêu chí chấp nhận
AI	Artificial Intelligence – trí tuệ nhân tạo
BE	Best-Effort (lớp QoS thấp nhất)
CSV	Comma-Separated Values
DC	DataCenter – trung tâm dữ liệu
DES	Discrete-Event Simulation – mô phỏng rời rạc theo sự kiện
DQN	Deep Q-Network
DRL	Deep Reinforcement Learning – học tăng cường sâu
DVFS	Dynamic Voltage and Frequency Scaling
ELDAS	Energy- and Latency-aware Datacenter AI Scheduler – tên đề tài
GAE	Generalised Advantage Estimation
GNN	Graph Neural Network
GPU	Graphics Processing Unit
IEA	International Energy Agency
JCT	Job Completion Time – thời gian hoàn thành tác vụ
JRE/JDK	Java Runtime / Development Kit
K8s	Kubernetes
LLM	Large Language Model
LOC	Lines Of Code – số dòng mã nguồn
LRP	LeastRequestedPriority (chiến lược chấm điểm K8s)
LS	Latency-Sensitive (lớp QoS)
MDP	Markov Decision Process
MLP	Multi-Layer Perceptron
MOMDP	Multi-Objective Markov Decision Process
MORL	Multi-Objective Reinforcement Learning – học tăng cường đa mục tiêu
MOOP	Multi-Objective Optimization Problem
MT	Mục tiêu (MT1–MT6)
NSGA-II	Non-dominated Sorting Genetic Algorithm II
PE	Processing Element (đơn vị tính của CloudSim, $\equiv$ vCPU)
PPO	Proximal Policy Optimization
PUE	Power Usage Effectiveness – chỉ số hiệu quả năng lượng DC
QoS	Quality of Service – chất lượng dịch vụ
RAM	Random Access Memory
RL	Reinforcement Learning – học tăng cường
RPC	Remote Procedure Call – gọi thủ tục từ xa
SB3	Stable-Baselines3 (thư viện Python)
SLA	Service Level Agreement – thoả thuận mức dịch vụ
TDP	Thermal Design Power
TPS	Transactions Per Second
UTC	Coordinated Universal Time
VM	Virtual Machine – máy ảo
W&B	Weights & Biases (dịch vụ theo dõi thí nghiệm)